Web Data Science

Retrieving, Parsing, and Analyzing Data from the Web with Python

Brian C. Keegan, Ph.D.

April 2026

Table of contents

Preface 11
- What You Will Learn . . . 11
- How This Book Is Organized . . . 11
- Prerequisites and Companion Resources . . . 12
- Acknowledgements . . . 12

I Foundations 13

1 Introduction to Web Data Science 14
- 1.1 Why Web Data Science? . . . 14
- 1.2 The Data Science Mindset . . . 15
- 1.3 Setting Up Your Environment . . . 15
  - 1.3.1 Installing Anaconda . . . 15
  - 1.3.2 Jupyter Notebooks . . . 16
  - 1.3.3 Core Libraries . . . 16
- 1.4 Your First Request . . . 17
  - 1.4.1 From Response to DataFrame . . . 18
  - 1.4.2 Saving Your Work . . . 19
- 1.5 Documentation as Professional Practice . . . 20
- 1.6 Exercises . . . 20
- 1.7 Social History and Public Interest . . . 21
- 1.8 Common Issues to Debug . . . 22
- 1.9 Key Takeaways . . . 22
- 1.10 Further Reading . . . 23

2 Ethics, Law, and Responsible Data Collection 24
- 2.1 The Ethics of Retrieval . . . 24
- 2.2 Legal Frameworks . . . 24
  - 2.2.1 The Computer Fraud and Abuse Act . . . 24
  - 2.2.2 Terms of Service as Quasi-Law . . . 25
  - 2.2.3 International Considerations . . . 25
- 2.3 Technical Norms: `robots.txt` . . . 26
  - 2.3.1 Comparing robots.txt Across Platforms . . . 27
- 2.4 Responsible Request Headers . . . 27

2.5 Rate Limiting . . . 28
2.5.1 Building a Responsible Scraper Function . . . 28
2.6 An Ethical Decision Framework . . . 30
2.6.1 IRBs and Web Data . . . 30
2.7 Exercises . . . 31
2.8 Social History and Public Interest . . . 32
2.9 Common Issues to Debug . . . 33
2.10 Key Takeaways . . . 33
2.11 Further Reading . . . 33

3 The Post-API Age **34**
3.1 The Golden Age of Web Data Access . . . 34
3.2 Three Structural Pressures . . . 35
3.2.1 Enclosure . . . 35
3.2.2 Exemption . . . 35
3.2.3 Erosion . . . 36
3.3 Three Counter-Values . . . 36
3.3.1 Openness . . . 36
3.3.2 Oversight . . . 37
3.3.3 Ownership . . . 37
3.4 Public Interest Lineages . . . 37
3.5 What This Means for Practice . . . 38
3.6 Exercises . . . 38
3.7 Key Takeaways . . . 38
3.8 Further Reading . . . 39

4 Data Formats: XML and JSON **40**
4.1 Why Data Formats Matter . . . 40
4.2 XML and BeautifulSoup . . . 40
4.2.1 The Tree Structure . . . 40
4.2.2 Parsing with BeautifulSoup . . . 41
4.2.3 Extracting and Counting . . . 42
4.2.4 Working with Real XML . . . 42
4.2.5 From XML to DataFrame . . . 43
4.3 JSON and Python Data Structures . . . 44
4.3.1 Lists and Dictionaries . . . 44
4.3.2 Nested Structures . . . 45
4.3.3 The List-of-Dictionaries Pattern . . . 46
4.3.4 Parsing JSON Strings . . . 46
4.3.5 Working with a Real JSON Object . . . 47
4.3.6 Handling Deeply Nested JSON . . . 48
4.4 When to Use Which . . . 49
4.5 Exercises . . . 50

4.6 Social History and Public Interest . . . 51
4.7 Common Issues to Debug . . . 51
4.8 Key Takeaways . . . 52
4.9 Further Reading . . . 52

5 Web Architecture and Protocols **53**
5.1 What Happens When You Load a Web Page? . . . 53
5.2 The Client-Server Model . . . 53
5.3 Browser Developer Tools . . . 54
5.3.1 The Inspector Tab . . . 54
5.3.2 The Network Tab . . . 54
5.4 TCP/IP: The Transport Layer . . . 55
5.4.1 Traceroute . . . 55
5.5 DNS: The Address Book . . . 56
5.5.1 The Structure of Domain Names . . . 57
5.6 HTTP: The Application Layer . . . 57
5.6.1 Diagnosing HTTP Errors . . . 58
5.6.2 Making API Requests with Custom Headers . . . 59
5.7 URLs: Anatomy and Parsing . . . 60
5.7.1 Building URLs Programmatically . . . 61
5.8 Exercises . . . 62
5.9 Social History and Public Interest . . . 62
5.10 Common Issues to Debug . . . 63
5.11 Key Takeaways . . . 63
5.12 Further Reading . . . 64

II Documents **65**

6 Parsing Static Web Pages **66**
6.1 From HTML to Data . . . 66
6.2 Strategy 1: Manual Table Parsing . . . 66
6.2.1 Retrieving and Inspecting . . . 66
6.2.2 Extracting Row Data . . . 67
6.2.3 Building the DataFrame . . . 68
6.3 Strategy 2: `pandas.read_html()` . . . 68
6.3.1 Cleaning Up read_html Results . . . 69
6.4 Strategy 3: Writing a Custom Parser . . . 70
6.4.1 The Oscars Example . . . 70
6.4.2 Finding Reliable Anchors . . . 70
6.4.3 Navigating the Tree for Specificity . . . 71
6.4.4 Extracting Category Data . . . 71
6.4.5 Abstracting into Functions . . . 72

6.4.6 A Second Parser: Colorado Legislators . . . 72
6.4.7 Error Handling Patterns . . . 74
6.5 Exercises . . . 75
6.6 Social History and Public Interest . . . 76
6.7 Common Issues to Debug . . . 77
6.8 Key Takeaways . . . 77
6.9 Further Reading . . . 77

7 Archived Web Pages and the Wayback Machine 78
7.1 The Web Is Not Permanent . . . 78
7.2 The Availability API . . . 78
7.2.1 Comparing Multiple Snapshots . . . 79
7.3 The CDX Server API . . . 81
7.3.1 Filtering and Analyzing CDX Data . . . 82
7.4 Tracking Policy Changes Over Time . . . 83
7.4.1 Generating Date Ranges . . . 83
7.4.2 Retrieving Historical Content . . . 84
7.4.3 Serializing Data . . . 85
7.5 Document Similarity with Bag-of-Words . . . 85
7.5.1 Visualizing Document Change Over Time . . . 86
7.6 Exercises . . . 88
7.7 Social History and Public Interest . . . 89
7.8 Common Issues to Debug . . . 89
7.9 Key Takeaways . . . 90
7.10 Further Reading . . . 90

8 Dynamic Web Pages with Selenium 91
8.1 When Static Scraping Fails . . . 91
8.2 Setting Up Selenium . . . 91
8.3 Navigating and Finding Elements . . . 92
8.4 Extracting Data . . . 93
8.5 Simulating Interactions . . . 93
8.5.1 Clicking . . . 93
8.5.2 Typing and Searching . . . 93
8.5.3 Scrolling . . . 94
8.5.4 Waiting for Dynamic Content . . . 94
8.5.5 Scraping Infinite Scroll Pages . . . 95
8.6 Passing to BeautifulSoup . . . 96
8.7 Always Close the Driver . . . 97
8.8 The Fragility Problem . . . 97
8.8.1 Headless Mode . . . 97
8.9 Exercises . . . 98
8.10 Social History and Public Interest . . . 99

8.11 Common Issues to Debug . . . 100
8.12 Key Takeaways . . . 100
8.13 Further Reading . . . 100
8.14 A Practical Selenium Workflow . . . 100
8.14.1 When to Use Selenium vs. APIs . . . 102

9 Extracting Data from PDFs **103**
9.1 The Web's Filing Cabinets . . . 103
9.2 PyPDF Basics . . . 104
9.2.1 Extracting Text . . . 104
9.2.2 Cleanup Strategies . . . 104
9.2.3 Regular Expressions for PDF Text . . . 105
9.3 Case Study: Boulder City Council Minutes . . . 106
9.3.1 Measuring Council Attendance . . . 107
9.3.2 Building the Pipeline . . . 108
9.3.3 Visualizing Trends . . . 108
9.3.4 Measuring Public Comment Frequency . . . 108
9.4 Exercises . . . 110
9.5 Social History and Public Interest . . . 111
9.6 Common Issues to Debug . . . 112
9.7 Key Takeaways . . . 112
9.8 Further Reading . . . 112
9.9 Building a Reusable PDF Pipeline . . . 113
9.9.1 When PyPDF Is Not Enough . . . 115

III APIs **116**

10 Introduction to APIs: Wikipedia **117**
10.1 What Is an API? . . . 117
10.2 Why Wikipedia? . . . 118
10.3 The Anatomy of an API Call . . . 118
10.4 Getting Revisions . . . 119
10.4.1 Understanding Pagination . . . 120
10.4.2 Analyzing Revision History . . . 121
10.5 Getting Pageviews . . . 121
10.6 Getting Links and Categories . . . 123
10.6.1 Interlanguage Links . . . 124
10.7 Getting User Contributions . . . 126
10.8 Writing Raw API Requests from Documentation . . . 126
10.9 A Second Case Study: World War II . . . 127
10.9.1 Combining Endpoints . . . 128
10.10 Exercises . . . 129

10.11 Social History and Public Interest . . . 130
10.12 Common Issues to Debug . . . 130
10.13 Key Takeaways . . . 131
10.14 Further Reading . . . 131

11 Government Data APIs **132**
11.1 Public Data by Mandate . . . 132
11.2 API Key Management . . . 132
11.2.1 Environment Variables . . . 133
11.2.2 Configuration Files . . . 133
11.3 U.S. Census Bureau API . . . 134
11.3.1 Navigating the Variable Catalog . . . 134
11.3.2 Finding the Right Variable . . . 134
11.3.3 Querying for Specific Places . . . 135
11.3.4 Comparing Multiple Locations . . . 136
11.4 FRED API . . . 136
11.4.1 Finding Series IDs . . . 137
11.4.2 Combining Multiple Series . . . 137
11.4.3 Server-Side Transformations . . . 138
11.4.4 Comparing CPI Across Locations . . . 139
11.5 FEC API . . . 140
11.5.1 Getting Campaign Finance Data . . . 141
11.6 Combining Census and FRED Data . . . 141
11.7 Exercises . . . 143
11.8 Social History and Public Interest . . . 143
11.9 Common Issues to Debug . . . 144
11.10 Key Takeaways . . . 144
11.11 Further Reading . . . 145

12 Social and Media Platform APIs **146**
12.1 Platforms, Data, and Power . . . 146
12.2 Reddit via PRAW . . . 146
12.2.1 Authentication . . . 146
12.2.2 Exploring Subreddits . . . 147
12.2.3 Retrieving Submissions . . . 147
12.2.4 Comment Trees . . . 148
12.2.5 Analyzing Comment Engagement . . . 149
12.3 Spotify via spotipy . . . 150
12.3.1 Searching and Exploring . . . 150
12.3.2 Audio Features . . . 151
12.3.3 Comparing Artists by Audio Profile . . . 151
12.4 Bluesky via AT Protocol . . . 153
12.5 Mastodon via Mastodon.py . . . 154

12.6 Exercises . . . 155
12.7 Social History and Public Interest . . . 155
12.8 Common Issues to Debug . . . 156
12.9 Key Takeaways . . . 156
12.10 Further Reading . . . 156
12.11 Comparing Platform Architectures . . . 157
12.11.1 Building a Bluesky Social Network . . . 157
12.11.2 Mastodon: Working Across Instances . . . 159

13 AI and Language Model APIs **160**
13.1 LLMs as Web Data Science Tools . . . 160
13.2 The OpenAI API . . . 160
13.2.1 Basic Chat Completion . . . 161
13.2.2 Customizing Roles . . . 161
13.2.3 Structured Output and Few-Shot Prompting . . . 161
13.3 Text Embeddings . . . 162
13.4 Applied Example: Legislative Bill Analysis . . . 163
13.4.1 Step 1: Classifying Bills . . . 163
13.4.2 Step 2: Extracting Structured Information . . . 164
13.4.3 Step 3: Measuring Similarity with Embeddings . . . 165
13.5 Fine-Tuning for Domain-Specific Tasks . . . 166
13.6 Cost Management . . . 167
13.7 Reproducibility and Documentation . . . 168
13.8 The Anthropic API . . . 169
13.8.1 Comparing Model Outputs . . . 169
13.9 Exercises . . . 171
13.10 Social History and Public Interest . . . 171
13.11 Common Issues to Debug . . . 172
13.12 Key Takeaways . . . 172
13.13 Further Reading . . . 173

IV Practice **174**

14 Automating Data Collection **175**
14.1 From Exploration to Production . . . 175
14.2 Why Automate? . . . 175
14.3 Step 1: Notebook to Script . . . 176
14.3.1 Testing Your Script Locally . . . 178
14.4 Step 2: GitHub Actions . . . 179
14.4.1 Python-Free Scraping . . . 180
14.4.2 Understanding Cron Syntax . . . 181
14.4.3 Secrets Management . . . 181

14.4.4 Monitoring and Debugging Actions . . . 182
14.4.5 Handling Failures Gracefully . . . 182
14.5 Step 3: Analyze Collected Data . . . 183
14.6 Exercises . . . 184
14.7 Social History and Public Interest . . . 185
14.8 Common Issues to Debug . . . 185
14.9 Key Takeaways . . . 186
14.10 Further Reading . . . 186

15 Research Design with Web Data **187**
15.1 Technical Skill Is Not Enough . . . 187
15.2 From Question to Data Source . . . 187
15.2.1 The Proposal Arc . . . 188
15.2.2 Matching Questions to Methods . . . 188
15.3 Sampling and Representativeness . . . 189
15.4 Data Quality and Validation . . . 189
15.5 Reproducibility and Documentation . . . 190
15.6 Communicating Findings . . . 191
15.6.1 Visualization as Argument . . . 191
15.7 Public Interest Framing . . . 192
15.7.1 Data Journalism as a Model . . . 193
15.8 Exercises . . . 193
15.9 Key Takeaways . . . 194
15.10 Further Reading . . . 194
15.11 Common Research Designs with Web Data . . . 194
15.11.1 Longitudinal Tracking . . . 195
15.11.2 Cross-Sectional Snapshot . . . 195
15.11.3 Content Analysis . . . 195
15.11.4 Natural Experiments . . . 196
15.12 Managing the Research Process . . . 196
15.12.1 Version Control for Research Projects . . . 196
15.12.2 Data Documentation . . . 197

V Appendices **198**

16 AI Coauthorship and Responsible Disclosure **199**
16.1 When Disclosure Is Required . . . 199
16.2 When Disclosure Is Typically Not Required . . . 199
16.3 How to Disclose: A Practical Framework . . . 199
16.4 Example Disclosure Statements . . . 200
16.5 Publisher and Funder Requirements . . . 200
16.6 Core Principles . . . 201

17 Where to Go from Here **202**
17.1 Advanced Scraping and Data Collection 202
17.2 Advanced APIs and Data Access 202
17.3 Text Analysis and NLP 203
17.4 Network Analysis 203
17.5 Geospatial Analysis 203
17.6 Data Management and Infrastructure 203
17.7 Communities and Continuing Education 204
17.8 Recommended Textbooks 204

Preface

The internet makes many kinds of information easy to access. Public agencies publish data through APIs, social platforms expose behavioral traces through developer tools, news organizations structure their archives in parseable HTML, and government bodies release records as downloadable documents. The ability to retrieve, parse, and analyze this information is a valuable and increasingly contested skill.

This book provides an overview of computational tools and practices for transforming web documents and APIs into data for common research designs. It is designed for advanced undergraduates and early-career master's students who have some experience with Python programming — you know loops, functions, lists, and dictionaries — but have not yet worked extensively with web protocols, HTML parsing, or API access.

What You Will Learn

By the end of this book, you will be able to:

- Retrieve data from static and dynamic web pages, archived documents, and PDF files
- Access and query web APIs for platforms including Wikipedia, the U.S. Census Bureau, Reddit, Spotify, Bluesky, and OpenAI
- Navigate the legal and ethical landscape of web data collection
- Automate data collection pipelines using GitHub Actions
- Design research projects that combine multiple web data sources
- Reason critically about the structural forces — enclosure, exemption, and erosion — that are reshaping access to web data

How This Book Is Organized

The book is divided into four parts. **Part I (Foundations)** introduces the computing environment, ethical and legal frameworks, the changing landscape of data access, web data formats, and internet protocols. **Part II (Documents)** covers extracting data from static web pages, archived pages, dynamic pages, and PDFs. **Part III (APIs)** teaches you to work with Wikipedia, government, social media, and AI APIs. **Part IV (Practice)** addresses automation and research design.

Each chapter follows a consistent structure: learning objectives, conceptual motivation, a guided tutorial with narrative code, exercises, a discussion of social history and public interest themes, common debugging issues, key takeaways, and links to further reading.

Prerequisites and Companion Resources

This book assumes you have Python 3.10 or above installed via the Anaconda distribution, and that you are comfortable working in Jupyter Notebooks. If you need to set up or troubleshoot your computing environment, the *Missing Manual for Information Scientists* is a companion resource that covers the hidden curriculum of computing technologies — from operating systems and file management to debugging, version control, and secrets management. This book references specific chapters of the *Missing Manual* throughout.

Code blocks in this book are narrative: they include comments explaining what each line does, but they are set to not execute automatically. You should run the code yourself in a Jupyter Notebook or Python script to see the results. This design choice reflects a pedagogical commitment: running code, seeing it fail, and debugging it is where the learning happens.

Acknowledgements

This book grew out of the INFO 4871/5871: Web Data Science course at the University of Colorado Boulder. It has benefited from the questions, frustrations, and insights of many cohorts of students. The author also gratefully acknowledges the journalists, researchers, and civic technologists whose work demonstrates why web data fluency matters for the public interest.

AI Disclosure

Portions of this book were drafted with the assistance of large language model tools. All content has been reviewed, edited, and verified by the author. For more on responsible AI disclosure in academic work, see Chapter 16.

Part I

Foundations

1 Introduction to Web Data Science

Learning Objectives

- Articulate what distinguishes web data science from general data science
- Set up a reproducible computing environment with Anaconda, Python, and Jupyter Notebooks
- Make a first HTTP request with the `requests` library and interpret the response
- Locate, read, and apply library documentation as a core professional skill
- Adopt the dispositions — growth mindset, computational thinking, hacker ethic, and scientific norms — that distinguish effective practitioners

1.1 Why Web Data Science?

The web is the largest, most dynamic, and most contested data source ever created. Every day, government agencies publish legislative records, researchers share datasets, journalists file stories, and billions of people generate behavioral traces on social platforms. This data can answer questions that matter: How does misinformation spread during elections? How have housing costs changed in your community? How do online communities govern themselves? How are platforms shaping public discourse?

But the web is not a database. It is a chaotic, evolving ecosystem of documents, APIs, protocols, and norms. Data on the web does not come in tidy CSV files ready for analysis. It comes as HTML tables embedded in pages full of tracking scripts, as JSON responses from APIs that require authentication and rate limiting, as scanned PDFs of city council meeting minutes, and as social media posts that might disappear tomorrow. Transforming this raw material into analyzable data requires a specific set of skills that sits at the intersection of programming, research design, and ethical judgment.

That is what this book teaches. Web data science is the practice of retrieving data from the web, parsing it into structured formats, and analyzing it to produce knowledge. It differs from general data science in that your first challenge is not modeling or visualization — it is *getting the data in the first place*.

1.2 The Data Science Mindset

Working with web data requires not just technical fluency but a professional disposition. Four components of this disposition will serve you throughout this book and your career.

Growth mindset means understanding that ability develops through effort. You will encounter libraries you have never used, error messages you do not understand, and web pages that resist your parsing attempts. This is normal. The goal is continual improvement, not instant mastery. Treat each frustration as a learning signal, not evidence of inadequacy.

Computational thinking is the disciplined approach that computer scientists bring to problems: decomposing large tasks into smaller pieces, recognizing patterns across examples, abstracting away inessential details, and specifying unambiguous sequences of steps. When you face a complex scraping task, you will practice breaking it into retrieve, parse, clean, and analyze stages — each with clear inputs and outputs.

The hacker ethic emphasizes sharing, openness, creativity, and a bias toward action. The open-source libraries you will use throughout this book exist because communities of developers chose to share their work. You should approach problems with curiosity, experiment freely, and share what you learn.

Scientific norms ground your work in communalism, skepticism, and reproducibility. Document your methods so others can replicate them. Question your assumptions about the data. Communicate your findings honestly, including their limitations.

> Missing Manual Reference
>
> For a deeper discussion of computational thinking and its four components — decomposition, pattern recognition, abstraction, and algorithmic thinking — see *Missing Manual* Chapter 1: Introduction.

1.3 Setting Up Your Environment

You will need three things to work through this book: a Python installation, a way to run Jupyter Notebooks, and a few core libraries.

1.3.1 Installing Anaconda

The Anaconda distribution of Python provides a curated collection of scientific computing libraries along with the `conda` package manager. Download the latest version from anaconda.com and follow the installation instructions for your operating system.

After installation, open a terminal (macOS/Linux) or Anaconda Prompt (Windows) and update your installation:

```
# Run these commands in a terminal, not in Python
# conda update conda
# conda install anaconda=2024.06
```

> **Missing Manual Reference**
>
> If you run into installation issues, see *Missing Manual* Chapter 14: Package Management for troubleshooting `conda` and `pip`.

1.3.2 Jupyter Notebooks

Jupyter Notebooks are interactive computing environments that keep your code, documentation, and results in a single file. Launch one from the terminal with `jupyter notebook`, which will open a browser window where you can create and edit notebooks.

A notebook consists of cells. **Code cells** contain Python that you can execute with Shift+Enter. **Markdown cells** contain formatted text for documentation. The ability to interleave code with narrative explanation makes notebooks ideal for the kind of exploratory, iterative work that characterizes web data science.

```
# This is a code cell. When you run it, Python executes the code
# and displays any output below the cell.
print("Hello, web data science!")
# Output: Hello, web data science!
```

> **Missing Manual Reference**
>
> For a thorough introduction to Jupyter Notebooks — cell types, execution order, keyboard shortcuts, kernel management, and common pitfalls — see *Missing Manual* Chapter 16: Jupyter.

1.3.3 Core Libraries

This book uses several Python libraries extensively. Install them now if they are not already available in your Anaconda distribution:

```
# Run in a terminal:
# pip install requests beautifulsoup4 pandas matplotlib seaborn
```

Throughout this book, you will install additional libraries as needed for specific chapters. For now, confirm that the core libraries import without error:

```
import requests
import pandas as pd
import matplotlib.pyplot as plt
from bs4 import BeautifulSoup

print("All core libraries loaded successfully.")
```

1.4 Your First Request

Let us preview the loop that structures every chapter in this book: **retrieve** data from the web, **parse** it into a usable structure, and **analyze** it to produce insight.

The `requests` library lets you make HTTP requests — the same kind of request your browser makes when you type a URL. Here you will retrieve the Wikipedia article for the University of Colorado Boulder:

```
import requests

# Retrieve the page
url = "https://en.wikipedia.org/wiki/University_of_Colorado_Boulder"
response = requests.get(url)

# Check the response
print(response.status_code)  # 200 means success
print(type(response))        # <class 'requests.models.Response'>
print(len(response.text))    # Number of characters in the HTML
```

The `response` object contains everything the server sent back: a status code indicating success or failure, headers with metadata about the response, and the body of the document as a text string. You can inspect the first 500 characters of the HTML:

```
print(response.text[:500])
# You'll see raw HTML: <!DOCTYPE html>, <head>, <title>, etc.
```

This is raw HTML — not yet useful for analysis. In Chapter 4 and Chapter 6, you will learn to parse this HTML into structured data using BeautifulSoup. For now, the point is that retrieving a web page programmatically is as simple as `requests.get(url)`.

You can also retrieve data in JSON format from an API. Here is a request to the Wikimedia pageviews API, which returns the number of times a Wikipedia article was viewed on a given day:

```
import requests

api_url = "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
article = "University_of_Colorado_Boulder"
url = f"{api_url}/en.wikipedia/all-access/all-agents/{article}/daily/20240101/20240131"

headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
response = requests.get(url, headers=headers)

data = response.json()  # Parse the JSON response into a Python dictionary
print(type(data))        # <class 'dict'>
print(data.keys())       # What keys are in the response?
```

Notice two things. First, you included a custom `User-Agent` header identifying yourself and providing contact information. This is a norm of responsible web data collection that you will learn more about in Chapter 2. Second, the `.json()` method converts the response body from a JSON string into a Python dictionary — a data structure you already know how to navigate.

1.4.1 From Response to DataFrame

The `data["items"]` key contains a list of dictionaries, one per day, each with fields like `timestamp`, `views`, `article`, and `project`. This is exactly the kind of structure that pandas can convert into a DataFrame with a single call:

```
import pandas as pd

# Convert the list of dictionaries into a DataFrame
df = pd.DataFrame(data["items"])
print(df.head())
# You'll see columns: project, article, granularity, timestamp, access, agent, views

# The timestamp column looks like "2024010100" - convert it to a proper date
df["date"] = pd.to_datetime(df["timestamp"], format="%Y%m%d00")
```

```
# Plot the time series
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4))
plt.plot(df["date"], df["views"])
plt.xlabel("Date")
plt.ylabel("Daily Pageviews")
plt.title(f"Wikipedia Pageviews: {article}")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

This retrieve-parse-DataFrame-visualize pipeline is the fundamental pattern of this book. In every subsequent chapter, you will repeat these steps with different data sources: retrieve HTML pages, XML feeds, JSON API responses, or PDF documents; parse them into structured records; convert those records into a pandas DataFrame; and analyze or visualize the result. The specifics change — BeautifulSoup for HTML, `json.loads()` for JSON, regular expressions for messy text — but the arc remains the same. Recognizing this pattern early will help you approach each new chapter with a clear mental model of what you are trying to accomplish.

The DataFrame also gives you immediate analytical power. You can compute summary statistics with `df["views"].describe()`, identify the highest-traffic day with `df.loc[df["views"].idxmax()]`, or resample the data to weekly totals. These operations are not the focus of this chapter, but they illustrate why getting data into a DataFrame is such a valuable intermediate step.

1.4.2 Saving Your Work

Once you have data in a DataFrame, save it to disk. This is not optional — it is a core practice of responsible web data science:

```
# Save to CSV
df.to_csv("cu_boulder_pageviews.csv", index=False)

# Later, load it back without making another API request
df = pd.read_csv("cu_boulder_pageviews.csv")
print(f"Loaded {len(df)} rows from disk.")
```

Serialization — saving your data to a file — matters for three reasons. First, courtesy: every API call consumes server resources, and re-requesting data you already have is wasteful. Second, reproducibility: if you save the data you retrieved on a specific date, your analysis can be

reproduced even if the API changes or goes offline. Third, the data itself may change — Wikipedia pageview counts are finalized after a short delay, and an article's content can be edited at any moment. The version you retrieved is a snapshot, and saving it preserves that snapshot. You will explore JSON serialization for more complex data structures in Chapter 7.

1.5 Documentation as Professional Practice

Your previous classes may have discouraged using online resources. The training wheels are off now. Finding, reading, interpreting, and applying documentation is an essential professional skill — not a shortcut, not cheating, but the primary way that working programmers, data scientists, and researchers learn to use new tools.

Bookmark the documentation for the libraries you will use throughout this book:

- **requests**: requests.readthedocs.io
- **BeautifulSoup**: crummy.com/software/BeautifulSoup/bs4/doc
- **pandas**: pandas.pydata.org/docs
- **matplotlib**: matplotlib.org/stable/contents.html

When you encounter an error or do not know how to do something, start with the official documentation. If that does not help, search for the specific error message. When asking for help — from a classmate, an instructor, or an AI tool — "it's not working" is not an actionable request. Describe what you tried, what you expected to happen, what actually happened, and what the error message says.

> Missing Manual Reference
>
> For strategies on asking effective technical questions and reading official documentation, see *Missing Manual* Chapter 2: Asking Technical Questions and Chapter 4: Reading Official Documentation.

1.6 Exercises

1. **Environment check.** Create a new Jupyter Notebook. In the first cell, import `requests`, `pandas`, `matplotlib.pyplot`, and `BeautifulSoup`. In the second cell, use `requests.get()` to retrieve any web page of your choice and print the status code. In the third cell, write a Markdown cell explaining what the status code means.

2. **Exploring the Response object.** Use the `dir()` function on a `requests.Response` object to list all its attributes and methods. Identify three attributes not discussed in

this chapter and use the `help()` function or online documentation to figure out what they do. Write a brief explanation of each in a Markdown cell.

3. **Your first API call.** Using the Wikimedia pageviews API URL pattern from this chapter, retrieve the daily pageview data for a Wikipedia article of your choice over a month-long period. Convert the `data['items']` list into a pandas DataFrame and create a simple line plot of views over time using `matplotlib`.

4. **POST vs. GET.** Read the `requests` library documentation and find the method for making a POST request. In a Markdown cell, explain how POST differs from GET and describe a scenario where you would use each. What kind of data does a POST request typically send, and why would an API require it instead of GET?

5. **Multi-article comparison.** Retrieve daily pageview data for three Wikipedia articles of your choice over the same month. Save each to a separate CSV file using `df.to_csv()`. Load all three back with `pd.read_csv()` and create a single plot with all three time series on the same axes, with a legend identifying each article. Which article had the most variable traffic, and can you hypothesize why?

1.7 Social History and Public Interest

Web data science emerged at the intersection of several traditions. Computational social scientists in the 2000s and 2010s recognized that the web was generating behavioral data at unprecedented scale — traces of how people communicate, collaborate, and organize [@lazer2009computational]. Data journalists discovered that web scraping could hold institutions accountable by making nominally public data actually analyzable. Civic technologists built tools to make government data accessible to citizens.

These traditions share a commitment to the idea that data fluency is a form of power, and that this power should serve the public interest. As you develop your skills throughout this book, you will encounter recurring questions about who benefits from web data collection, who is harmed by it, and what obligations come with the ability to automate access to information at scale.

Some of the most consequential data science work of the past decade has come from this tradition. ProPublica's "Machine Bias" investigation in 2016 used scraped court records to reveal racial bias in criminal risk assessment algorithms — a finding that shaped national policy debates about algorithmic fairness. The Markup's "Citizen Browser" project recruited a panel of real users to study how Facebook's news feed algorithm distributed political content, producing evidence that would have been impossible to obtain through the platform's official research tools. These projects combined technical scraping skills with domain expertise and ethical frameworks to produce knowledge that served the public interest.

The intellectual roots of web data science run deeper than any single investigation. The computational social science manifesto by @lazer2009computational articulated a vision of research that leverages the "digital traces" left by online behavior — search queries, social media posts, hyperlink structures, editing histories — to answer questions at population scale. These traces are not designed for research; they are byproducts of ordinary activity. The researcher's task is to transform them into meaningful evidence while respecting the privacy and dignity of the people who generated them. This tension between the analytical power of digital trace data and the ethical obligations it creates is a thread that runs through every chapter of this book.

> **Public Interest Connection**
>
> The tools you learn in this book are not neutral. Web data collection can serve **openness** — making public information practically accessible — or it can enable surveillance and exploitation. The ethical frameworks in Chapter 2 and the public interest framing in Chapter 3 will help you navigate this tension throughout the course.

1.8 Common Issues to Debug

- `ModuleNotFoundError`: A library is not installed in your current Python environment. Run `pip install library-name` from a terminal, then restart your Jupyter kernel.
- `ConnectionError`: Your computer cannot reach the server. Check your internet connection; if you are on a university network, try a different network.
- **Status code 403 (Forbidden)**: The server rejected your request, often because you did not include a `User-Agent` header. See Chapter 2 for how to set custom headers.
- **Cells running out of order**: Jupyter cells can be executed in any order, which leads to stale variables. When in doubt, restart the kernel and run all cells from the top.

1.9 Key Takeaways

The web is a data source unlike any other — massive, dynamic, messy, and politically contested. Working with it requires not just code but a professional disposition: the habits of documentation, skepticism, and ethical reflection that distinguish a data scientist from someone who can run a script. Every technique in this book follows the same basic loop: retrieve data from the web, parse it into a usable structure, and analyze it to produce knowledge. You now have your environment set up and have made your first request. The rest of the book builds from here.

1.10 Further Reading

- `requests` library documentation: https://requests.readthedocs.io/
- Jupyter Notebook documentation: https://jupyter-notebook.readthedocs.io/
- @wilson2017good — practical guidance on scientific computing workflows
- @salganik2018bit — Chapter 1: Introduction, for an overview of social research in the digital age

2 Ethics, Law, and Responsible Data Collection

Learning Objectives

- Explain the legal frameworks governing web data access, including the CFAA, GDPR, and platform Terms of Service
- Retrieve and interpret a website's `robots.txt` file to determine what is permitted for automated access
- Configure responsible HTTP request headers including custom User-Agent strings with contact information
- Apply a structured ethical decision framework before collecting web data
- Implement rate limiting with `time.sleep()` to avoid overloading servers

2.1 The Ethics of Retrieval

The phrase "web scraping" is colloquial and popular but has pejorative connotations. Data is valuable: other people invested time, money, and effort to create and host the content you are about to retrieve programmatically. Before writing a single line of scraping code, you need to understand the legal, normative, and ethical landscape that governs your work.

This chapter establishes a decision framework that you should revisit before every data collection project in this book and beyond. The question is never simply "can I scrape this?" but rather "should I, and if so, how do I do it responsibly?"

2.2 Legal Frameworks

2.2.1 The Computer Fraud and Abuse Act

The Computer Fraud and Abuse Act (CFAA) was passed in 1984 in response to growing concerns about computer crime. Its key provision prohibits "unauthorized access" to computer systems, but what counts as "unauthorized" in the context of web scraping has been a matter of ongoing litigation.

The landmark case *hiQ Labs v. LinkedIn* (2022) tested whether scraping publicly accessible data constitutes unauthorized access. hiQ, a data analytics company, scraped public LinkedIn profiles to build workforce analytics products. LinkedIn sent a cease-and-desist letter and blocked hiQ's access. The Ninth Circuit ultimately held that scraping publicly accessible data does not violate the CFAA, since the data was not behind an authentication barrier. However, this ruling is narrow in scope — it applies to public-facing data and does not address data behind logins.

The case of *Sandvig v. Barr* challenged the CFAA's application to researchers who violated a website's Terms of Service in order to conduct audit studies of algorithmic discrimination. The concern was that the CFAA criminalized legitimate research. This case illustrates the tension between researchers' need for data and platforms' desire to control how their systems are studied.

The key takeaway: the legal landscape is unsettled and varies by jurisdiction. "It's technically possible" is not the same as "it's legal," and "it's legal" is not the same as "it's ethical."

2.2.2 Terms of Service as Quasi-Law

Every major platform has Terms of Service (ToS) that restrict automated data collection. When you create an account on Twitter, Reddit, or Facebook, you agree to these terms. Violating them can result in account suspension, IP blocking, or even legal action.

> **Warning**
>
> The code in several chapters of this book will retrieve data in ways that may violate the Terms of Service of the platforms involved. This is done in an educational context to teach technical skills. You are responsible for understanding and abiding by the terms of any platform you access. When in doubt, do not scrape — use an official API or request access through an institutional research program.

2.2.3 International Considerations

The European Union's General Data Protection Regulation (GDPR) adds another layer of complexity. GDPR gives individuals the right to control how their personal data is collected and processed. If you are scraping data that includes personally identifiable information about EU residents — names, email addresses, social media handles — you may be subject to GDPR requirements regardless of where you are located.

2.3 Technical Norms: `robots.txt`

Beyond legal requirements, the web has developed a set of community norms for responsible automated access. The most important of these is the `robots.txt` protocol.

A `robots.txt` file lives at the root of a website (e.g., `https://example.com/robots.txt`) and specifies which parts of the site automated agents are permitted or forbidden from accessing. It uses a simple syntax:

```
import requests

# Retrieve robots.txt for Wikipedia
response = requests.get("https://en.wikipedia.org/robots.txt")
print(response.text[:500])
# You'll see directives like:
# User-agent: *
# Disallow: /w/
# Allow: /w/api.php
```

The key directives are `User-agent` (which crawler the rule applies to), `Disallow` (paths that should not be accessed), `Allow` (exceptions to disallow rules), and `Crawl-delay` (requested time between requests). You can parse these programmatically:

```
from urllib.robotparser import RobotFileParser

rp = RobotFileParser()
rp.set_url("https://en.wikipedia.org/robots.txt")
rp.read()

# Can a generic crawler access the main API?
print(rp.can_fetch("*", "https://en.wikipedia.org/w/api.php"))
# True - the API is explicitly allowed

# Can it access the edit interface?
print(rp.can_fetch("*", "https://en.wikipedia.org/w/index.php?action=edit"))
# Check the result - many edit paths are restricted
```

Two important caveats. First, `robots.txt` is a norm, not a law. Violating it is not illegal (in most jurisdictions), but it is disrespectful of the site operator's wishes and may result in your IP being blocked. Second, if a site does not have a `robots.txt` file (the request returns a 404), that means there are no crawling restrictions — not that you should crawl aggressively.

2.3.1 Comparing robots.txt Across Platforms

Different organizations have dramatically different approaches to automated access, and their `robots.txt` files reveal these philosophies. Compare three major sites:

```
import requests

sites = {
    "Wikipedia": "https://en.wikipedia.org/robots.txt",
    "Reddit": "https://www.reddit.com/robots.txt",
    "NY Times": "https://www.nytimes.com/robots.txt",
}

for name, url in sites.items():
    response = requests.get(url)
    lines = response.text.strip().split("\n")
    disallow_count = sum(1 for line in lines if line.startswith("Disallow"))
    allow_count = sum(1 for line in lines if line.startswith("Allow"))
    print(f"{name}: {len(lines)} lines, {disallow_count} Disallow rules, {allow_count} Allow
```

Wikipedia's `robots.txt` is relatively permissive — it restricts access to editing interfaces and internal administration pages but explicitly allows its API (`Allow: /w/api.php`). This reflects Wikipedia's organizational commitment to openness: the data exists to be shared. Reddit's file is more complex, restricting many URL patterns (user profiles, search results, sorted listings) while permitting basic page access. The New York Times is more restrictive still, blocking large sections of the site from automated access — reflecting the fact that its content is a commercial product.

These differences are not arbitrary. They reflect each organization's relationship with automated access: Wikipedia encourages it as part of its mission, Reddit tolerates it within limits, and the Times treats its content as proprietary. Reading `robots.txt` before you scrape tells you not just what is permitted but what kind of organization you are dealing with.

2.4 Responsible Request Headers

When your code makes requests to a web server, the server sees metadata about the request including the `User-Agent` header, which identifies who is making the request. By default, the `requests` library sends a generic User-Agent string. You should replace it with one that identifies you and provides contact information:

```python
import requests

headers = {
    "User-Agent": "WebDataScience/1.0 (INFO 4871; brian.keegan@colorado.edu)"
}

response = requests.get("https://example.com", headers=headers)
```

This practice serves two purposes: it lets site operators contact you if your scraping causes problems, and it signals that you are a responsible actor rather than a malicious bot. Some APIs (including the Wikimedia API you used in Chapter 1) require a meaningful User-Agent and will return errors or impose stricter rate limits if you do not provide one.

2.5 Rate Limiting

Making hundreds of requests per second to a web server can degrade its performance for other users — effectively mounting a denial-of-service attack on a site you are trying to study. Always space out your requests:

```python
import time
import requests

urls = ["https://example.com/page1", "https://example.com/page2"]

for url in urls:
    response = requests.get(url, headers=headers)
    # Process the response...
    time.sleep(1)  # Wait 1 second between requests
```

A good default is one request per second. If the site's `robots.txt` specifies a `Crawl-delay`, respect it. If you are hitting an API with documented rate limits, stay well within them.

2.5.1 Building a Responsible Scraper Function

The principles above — custom User-Agent, rate limiting, error handling — appear in every scraping project. Rather than implementing them ad hoc each time, you can wrap them into a reusable function:

```python
import requests
import time
import logging

def responsible_get(url, headers=None, delay=1.0):
    """Make an HTTP GET request with responsible defaults.

    Parameters
    ----------
    url : str
        The URL to request
    headers : dict, optional
        Custom headers; a default User-Agent is added if none provided
    delay : float
        Seconds to wait after the request (default: 1.0)

    Returns
    -------
    requests.Response or None
        The response object, or None if the request failed
    """
    default_headers = {
        "User-Agent": "WebDataScience/1.0 (student-project; your-email@colorado.edu)"
    }
    if headers:
        default_headers.update(headers)

    try:
        response = requests.get(url, headers=default_headers, timeout=30)
        logging.info(f"{response.status_code} {url}")
        time.sleep(delay)
        return response
    except requests.exceptions.ConnectionError:
        logging.error(f"Connection failed: {url}")
        return None
    except requests.exceptions.Timeout:
        logging.error(f"Request timed out: {url}")
        return None
```

Every design choice in this function reflects a principle from this chapter. The default User-Agent identifies you and provides contact information, so site operators can reach you if your scraping causes problems. The configurable delay enforces rate limiting by default — you have

to explicitly override it, which makes aggressive scraping a conscious decision rather than an oversight. The `try/except` blocks handle the two most common network failures gracefully, logging the error instead of crashing your entire scraping run. The `timeout` parameter prevents your script from hanging indefinitely on an unresponsive server.

You will use variations of this pattern in Chapter 6 when scraping HTML tables, in Chapter 7 when retrieving historical snapshots, and throughout the API chapters. Building responsible defaults into your tools is more reliable than remembering to add them each time.

2.6 An Ethical Decision Framework

Before any data collection project, work through these five questions:

1. **Legality.** Is this data collection permitted under the CFAA, GDPR, and any applicable local laws? Is it consistent with the site's Terms of Service?
2. **Consent.** Did the people whose data you are collecting consent to this use? Is the data truly public, or did users share it within a platform with reasonable expectations of limited visibility?
3. **Proportionality.** Are you collecting only the data you need for your research question? Could you answer your question with less data or with less sensitive data?
4. **Privacy.** Does your dataset contain personally identifiable information? Could individuals be re-identified from ostensibly anonymized data? What are the risks if the data is breached or misused?
5. **Server impact.** Will your scraping impose a meaningful load on the server? Are you rate-limiting appropriately? Are you scraping during off-peak hours for high-traffic sites?

No framework can substitute for judgment. But working through these questions systematically will help you identify risks and make defensible decisions.

2.6.1 IRBs and Web Data

If you are conducting research at a university, there is an additional layer of oversight: the Institutional Review Board (IRB). IRBs review research involving human subjects to ensure it meets ethical standards derived from the Belmont Report's three principles — respect for persons, beneficence, and justice.

When does web scraping become human subjects research? The general rule is that it qualifies when you collect data *about identifiable people* to produce *generalizable knowledge*. Scraping public government records — legislative votes, budget documents, meeting minutes — is usually not human subjects research because the subjects are institutions and public officials acting in

their official capacities. But scraping social media profiles to study health behaviors, political attitudes, or personal relationships almost certainly is, because you are studying identifiable individuals and their behavior.

The line is not always clear. Scraping aggregated, anonymous data (like Wikipedia edit counts without usernames) falls into a gray area. When in doubt, consult your institution's IRB. Many universities have expedited review processes for minimal-risk research involving publicly available data, and the process of writing an IRB protocol forces you to think carefully about privacy, consent, and data security — exactly the issues this chapter's framework addresses. The Belmont Report's principles complement the five-question framework above: respect for persons maps to consent, beneficence to proportionality and privacy, and justice to the question of who bears the risks and who receives the benefits of your research.

2.7 Exercises

1. **Compare robots.txt files.** Retrieve and compare `robots.txt` for five websites in the same category (e.g., five news organizations, five universities, or five social media platforms). What patterns emerge in what they allow and restrict? Write a brief analysis in a Markdown cell.

2. **Terms of Service audit.** Read the Terms of Service for a platform you use regularly. Identify three clauses that restrict automated data collection. For each, assess whether academic research would be covered by any exceptions or fair use arguments.

3. **Ethical decision framework.** Apply the five-question framework to a hypothetical scenario: a researcher wants to scrape user profiles from a dating app to study patterns in self-presentation across demographic groups. Work through each question and write a recommendation.

4. **Rate limiting experiment.** Using `time.time()` to measure elapsed time, write a loop that makes five requests to a public website with a one-second delay between each. Then modify the delay to two seconds. Confirm that your actual request timing matches your intended rate.

5. **Personal robots.txt audit.** Retrieve `robots.txt` for a website you use daily but have never scraped. Read the `Disallow` rules carefully. Then find and read the site's Terms of Service (look for links in the footer). Write a 300-word analysis: if you wanted to collect data from this site for a class project, what would `robots.txt`, the Terms of Service, and the five-question ethical framework each permit and restrict? Where do these three sources of guidance agree, and where do they conflict?

> **Public Interest Connection**
>
> The ethical framework introduced here connects directly to the values of **oversight** and **openness** discussed in Chapter 3. Responsible data collection practices are themselves a form of oversight — they make your methods transparent and accountable. And the commitment to documenting your scraping practices (User-Agent strings, rate limiting, data handling) embodies the openness that the web data science community depends on.

2.8 Social History and Public Interest

The tension between data access and data protection is not new. Journalists have long navigated similar terrain through freedom-of-information laws, source protection, and editorial ethics. The CFAA was originally designed to combat hacking, but its vague language about "unauthorized access" has been used to threaten researchers, journalists, and security auditors. The Aaron Swartz case — in which a young activist faced federal charges for downloading academic articles from JSTOR through MIT's network — remains a sobering reminder of how aggressively the CFAA can be applied [@fiesler2020no].

The Cambridge Analytica scandal of 2018 was a turning point. A researcher collected Facebook data through the platform's API, ostensibly for academic purposes, then shared it with a political consulting firm. The fallout led Facebook, and then other platforms, to drastically restrict API access — a closure that affected all researchers, not just those who had misused data. This history is central to understanding the post-API landscape described in Chapter 3.

The aftermath of Cambridge Analytica created a chilling effect that extended far beyond Facebook. IRBs at universities became more cautious about approving any research involving social media data, even when the data was public and the research served clear public interests. Platforms shut down APIs that legitimate researchers had depended on for years, citing privacy concerns that conveniently also prevented the kind of external scrutiny that might reveal algorithmic bias or content moderation failures. The academic research community is still rebuilding trust and access through alternative frameworks: the European Union's Digital Services Act (DSA) now mandates researcher access to platform data for public interest research, data donation programs ask users to voluntarily share their data with researchers, and negotiated research agreements between universities and platforms provide structured access with privacy safeguards. These developments represent an ongoing effort to balance legitimate privacy concerns with the public's interest in understanding how platforms shape information, attention, and behavior. The tools and ethical frameworks in this chapter position you to participate in that effort responsibly.

2.9 Common Issues to Debug

- **`robots.txt` not found (404)**: This means there are no crawling restrictions, not that the site is blocking you.
- **Requests blocked despite `robots.txt` allowing access**: Some sites use additional bot detection (CAPTCHAs, JavaScript challenges, behavioral analysis) beyond `robots.txt`.
- **403 Forbidden with a responsible User-Agent**: The site may be blocking all automated access regardless of your headers. Respect this.
- **Forgetting `time.sleep()`**: If your IP gets temporarily banned, wait at least an hour before trying again, and add appropriate delays to your code.

2.10 Key Takeaways

Every capability introduced in subsequent chapters raises the questions you have worked through here. The legal landscape is unsettled; technical norms like `robots.txt` provide guidance but not protection; and ethical principles require case-by-case judgment. The framework — legality, consent, proportionality, privacy, server impact — should become habitual. When in doubt, err on the side of caution: use an API instead of scraping, collect less data rather than more, and always identify yourself in your request headers.

2.11 Further Reading

- @fiesler2020no — legal and ethical analysis of ToS-based data collection restrictions
- @freelon2018computational — the changing landscape of computational research access
- Electronic Frontier Foundation on the CFAA: https://www.eff.org/issues/cfaa
- The `robots.txt` specification: https://www.robotstxt.org/

3 The Post-API Age

Learning Objectives

- Describe the historical arc of web data access from open APIs to restricted, monetized, and disappearing endpoints
- Analyze specific cases of access restriction using the enclosure, exemption, and erosion framework
- Identify alternative data access strategies including archives, FOIA requests, data donations, and federated platforms
- Connect contemporary data access challenges to longer traditions of public interest inquiry
- Articulate the values of openness, oversight, and ownership as guiding principles for web data practice

Author Note

This chapter provides the structural framework for the post-API age argument. Sections marked with `[TO FILL]` require the author's lecture notes, specific examples from class, and personal voice. The enclosure/exemption/erosion framework and the openness/oversight/ownership values are drawn from the JRC working paper and should be adapted for an undergraduate/MS audience.

3.1 The Golden Age of Web Data Access

`[TO FILL: Narrative of the 2008-2018 period. Key points to cover:]`

- The rise of generous, well-documented APIs: Twitter API v1 (2006), Facebook Graph API (2010), Reddit API (free since 2008), Google APIs
- Why platforms invested in open data access: developer ecosystems, academic partnerships, advertising optimization, the "platform as infrastructure" model
- What this access enabled: computational social science at scale, network science research, data journalism investigations, civic tech applications

- Specific examples from your own research trajectory: how web data access was foundational to your work on Wikipedia, online collaboration, and platform dynamics

3.2 Three Structural Pressures

The landscape of web data access has undergone a structural transformation since approximately 2018. Three pressures — enclosure, exemption, and erosion — explain not just *that* access is shrinking but *why*, and what forces are driving the change.

3.2.1 Enclosure

`[TO FILL: Adapt from JRC paper Section X. Key points:]`

- **Definition**: The privatization of previously accessible data — platforms converting shared resources into controlled assets
- **Mechanisms**: API shutdowns and pricing changes, removal of third-party archives, restrictions on data exports, data licensing deals for AI training
- **Cases to detail**:
  - Twitter/X: from free Academic Research API to paid tiers ($42K/month for enterprise access), to effective shutdown of research access post-acquisition
  - Reddit: free API since 2008 → $0.24 per 1,000 API calls (July 2023), Pushshift removal, 7,000+ subreddit blackout
  - Facebook: post-Cambridge Analytica lockdown of Graph API, CrowdTangle shutdown
- **Historical parallel**: enclosure of common land in England — how a shared resource was privatized, who benefited, and what was lost
- **The LLM accelerant**: how generative AI made user-generated content valuable as training data (Reddit's $60M licensing deal with Google), accelerating the incentive to enclose

3.2.2 Exemption

`[TO FILL: Adapt from JRC paper Section X. Key points:]`

- **Definition**: Platforms operating beyond meaningful regulatory scrutiny, invoking privacy, IP, or competitive sensitivity to resist independent investigation
- **Mechanisms**: weaponizing the CFAA against researchers, selective deployment of "user protection" narratives, regulatory capture, lobbying against transparency mandates
- **Cases to detail**:

  - Platform claims that API restrictions protect user privacy — while continuing to sell user data to advertisers
  - Resistance to researcher access under the EU Digital Services Act
  - The asymmetry: platforms can study their users at will, but independent researchers cannot study the platforms

- **Connection to Chapter 2**: how the legal frameworks from Chapter 2 are deployed strategically by platforms

3.2.3 Erosion

`[TO FILL: Adapt from JRC paper Section X. Key points:]`

- **Definition**: The decay of the infrastructure for sustained observation — the gradual disappearance of the conditions that make web data research possible
- **Mechanisms**: link rot, platform churn (Twitter → X), migration to federated/decentralized networks complicating basic measurement, reproducibility crisis as underlying data disappears
- **Cases to detail**:
  - Studies showing that significant percentages of web links break within years
  - The challenge of longitudinal research when platforms redesign their HTML, change their APIs, or disappear entirely
  - The reproducibility crisis in computational social science: papers that cannot be replicated because the data source no longer exists
- **Connection to later chapters**: the Wayback Machine (Chapter 7) as a counter-erosion infrastructure; automation (Chapter 14) as a response to data impermanence

3.3 Three Counter-Values

In response to these pressures, three values can guide how researchers, technologists, and institutions build and sustain access to web data for the public interest.

3.3.1 Openness

`[TO FILL: Adapt from JRC paper. Key points:]`

- Transparent methods and pipelines so others can evaluate and reproduce your work
- Redundant archives that do not depend on any single platform's cooperation
- Methodological legibility: documenting not just *what* data you collected but *how* you collected it and *what constraints* shaped the collection
- Durable public repositories (Internet Archive, Zenodo, institutional data archives)

3.3.2 Oversight

`[TO FILL: Adapt from JRC paper. Key points:]`

- Independent auditing of platform algorithms and content moderation
- Documentation standards for datasets and collection methods
- Impact assessments before and after data access changes
- Regulatory frameworks that mandate researcher access (EU DSA Article 40)

3.3.3 Ownership

`[TO FILL: Adapt from JRC paper. Key points:]`

- Collective governance over data preservation and access
- Community-maintained infrastructure that resists centralized enclosure
- Interoperable systems (federated protocols like ActivityPub and AT Protocol) that distribute control
- Data cooperatives and community-managed archives

3.4 Public Interest Lineages

The tensions described in this chapter are not new to data science. Other professions have confronted analogous challenges — powerful actors controlling access to information that the public needs — and developed institutional responses.

`[TO FILL: Brief treatment of each lineage. Key points:]`

- **Journalism**: Freedom of information laws, source protection, the tradition of investigative reporting as public accountability
- **Law**: Discovery processes, transparency requirements, due process as protection against arbitrary power
- **Accounting**: Audit standards, disclosure requirements, the fiduciary duty to stakeholders
- **Planning**: Public comment periods, environmental impact assessments, community input on decisions affecting shared resources
- **Engineering**: Safety standards, professional codes of ethics, the obligation to consider public welfare

`[Author note: the goal here is a brief paragraph on each, not a deep dive. The argument is that data science is developing its own public interest mission and can learn from these precedents.]`

3.5 What This Means for Practice

The rest of this book equips you with a diverse technical toolkit precisely because no single method of access is reliable. When APIs close, you may need to scrape HTML (Chapter 6, Chapter 8). When web pages change, archives provide continuity (Chapter 7). When documents are locked in inaccessible formats, PDF extraction opens them up (Chapter 9). When platforms restrict automated access, federated alternatives offer open protocols (Chapter 12). And when you need sustained observation over time, automation turns one-time data collection into durable infrastructure (Chapter 14).

The strategic lesson is redundancy and flexibility. Build your skills across the full range of techniques so that the closure of any single data source does not shut down your research program.

```
[TO FILL: Add specific alternative strategies - official research access
programs, data donations, FOIA/public records, crowdsourcing - with brief
examples of each.]
```

3.6 Exercises

1. **Timeline exercise.** Create a timeline of major API access changes from 2008 to the present. Include at least five events across at least three platforms. For each event, identify which of the three pressures (enclosure, exemption, erosion) best characterizes it.
2. **Framework application.** Choose a platform API change not discussed in this chapter. Write a 500-word analysis applying the enclosure/exemption/erosion framework.
3. **Data contingency plan.** Design a research project that depends on a specific web data source. Then write a contingency plan: if your primary source disappears mid-study, what are your backup strategies? Consider at least three alternatives.
4. **Public interest audit.** Identify a web data research project (published paper, journalism investigation, or civic tech tool) that was enabled by API access. Assess how the project would be affected by current access restrictions. Which of the three values — openness, oversight, or ownership — does the project embody?

3.7 Key Takeaways

```
[TO FILL: Synthesize the chapter's argument in the author's voice. The core
message should be that understanding *why* access is shrinking - not just
*that* it is - equips students to navigate constraints strategically. The
three pressures explain the structural forces; the three values provide
```

normative guidance; and the rest of the book provides the technical toolkit for operating in this constrained environment.]

3.8 Further Reading

- @freelon2018computational — the original articulation of the "post-API age"
- @bruns2019after — platforms' fight against critical scholarly research
- @davidson2023platform — empirical study of how API restrictions threaten open science
- @perriam2020digital — digital methods in a post-API environment
- @lazer2020computational — obstacles and opportunities in computational social science

4 Data Formats: XML and JSON

Learning Objectives

- Distinguish between XML and JSON as structured data formats and identify when each is typically encountered
- Parse XML documents using BeautifulSoup's tree navigation methods
- Navigate nested JSON structures using Python's native lists and dictionaries
- Convert nested data structures into pandas DataFrames for analysis
- Handle common edge cases in structured data parsing

4.1 Why Data Formats Matter

Before you can extract data from the web, you need to understand how data is structured on the web. Every web page, every API response, every data feed uses a format that organizes information in a specific way. The two dominant formats you will encounter are XML (Extensible Markup Language) and JSON (JavaScript Object Notation).

XML is the older format, born from the document-centric web of the 1990s. It represents data as a tree of nested tags — a structure you will recognize when you encounter HTML in Chapter 6, since HTML is essentially a specific vocabulary of XML. JSON is the newer, lighter format that rose with Web 2.0 and the API economy. It maps directly onto Python's native data structures — lists and dictionaries — making it especially natural to work with programmatically.

This chapter teaches you to parse both formats, building the foundational skills that every subsequent chapter depends on.

4.2 XML and BeautifulSoup

4.2.1 The Tree Structure

XML organizes data as a hierarchy of nested tags. Each tag can contain text, attributes, and child tags. Here is a simplified example of Congressional member data:

```
house_raw = """<?xml version="1.0" encoding="UTF-8"?>
<congress session="118">
  <chamber name="House">
    <member>
      <first-name>Nancy</first-name>
      <last-name>Pelosi</last-name>
      <party>D</party>
      <state>CA</state>
      <district>11</district>
    </member>
    <member>
      <first-name>Kevin</first-name>
      <last-name>McCarthy</last-name>
      <party>R</party>
      <state>CA</state>
      <district>20</district>
    </member>
  </chamber>
</congress>"""
```

This is still just a string. To search and navigate it, you need to parse it into a data structure that understands the tree relationships. That is what BeautifulSoup does.

4.2.2 Parsing with BeautifulSoup

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(house_raw, "xml")

# Navigate the tree: congress > chamber > first member > first-name
print(soup.congress.chamber.member)
# Returns the first <member> tag and all its children
```

The power of BeautifulSoup is in its search methods. The `.find()` method returns the first tag matching your criteria. The `.find_all()` method returns all matching tags as a list:

```
# Find all member tags
members = soup.find_all("member")
print(len(members))  # 2
```

```
# Access text inside a tag
for member in members:
    first = member.find("first-name").text
    last = member.find("last-name").text
    party = member.find("party").text
    print(f"{first} {last} ({party})")
# Nancy Pelosi (D)
# Kevin McCarthy (R)
```

Notice the use of `.find("first-name")` rather than `.first-name` as a Python attribute. Hyphenated tag names break Python's dot notation, so you must use the `.find()` method for them.

4.2.3 Extracting and Counting

A common pattern is extracting all values for a particular tag and then analyzing the distribution:

```
# Count party affiliations
parties = [m.find("party").text for m in soup.find_all("member")]
print(parties)
# ['D', 'R']

# In a real dataset with all 435 members, you could count:
from collections import Counter
party_counts = Counter(parties)
print(party_counts)
# Counter({'D': 1, 'R': 1}) - with full data you'd see actual totals
```

The `.contents` method gives you a list of a tag's direct children, which is useful for exploring unfamiliar XML structures:

```
first_member = soup.find("member")
print(first_member.contents)
# A list of child tags and whitespace strings
```

4.2.4 Working with Real XML

Let us retrieve and parse actual Congressional member data. The clerk.house.gov website publishes member data in XML format:

```python
import requests
from bs4 import BeautifulSoup

url = "https://clerk.house.gov/xml/lists/MemberData.xml"
response = requests.get(url)
soup = BeautifulSoup(response.text, "xml")

# Find all members
members = soup.find_all("member")
print(f"Total members found: {len(members)}")

# Extract party information
parties = []
for m in members:
    party_tag = m.find("party")
    if party_tag:
        parties.append(party_tag.text)

from collections import Counter
print(Counter(parties))
```

4.2.5 From XML to DataFrame

Counting party affiliations is useful, but the real power of parsing XML comes from converting it into a DataFrame for systematic analysis. The pattern is straightforward: use `find_all()` to get all records, extract the fields you need from each one, build a list of dictionaries, and pass it to `pd.DataFrame()`:

```python
import pandas as pd

# Extract structured data from each member
records = []
for m in soup.find_all("member"):
    record = {
        "first_name": m.find("first-name").text if m.find("first-name") else None,
        "last_name": m.find("last-name").text if m.find("last-name") else None,
        "party": m.find("party").text if m.find("party") else None,
        "state": m.find("state").text if m.find("state") else None,
    }
    records.append(record)
```

```
df = pd.DataFrame(records)
print(df.head())
#   first_name  last_name party state
# 0      Nancy     Pelosi     D    CA
# 1      Kevin   McCarthy     R    CA
```

With the full Congressional dataset, you can now perform real analysis:

```
# Party breakdown
print(df.groupby("party").size())

# Top 10 states by number of representatives
print(df.groupby("state").size().sort_values(ascending=False).head(10))
```

This `find_all` → extract fields → list of dictionaries → DataFrame pipeline is the fundamental pattern of this book. You will use it to parse HTML tables in Chapter 6, extract data from archived web pages in Chapter 7, and process PDF text in Chapter 9. The data source changes, the tag names change, but the pipeline stays the same. Once you internalize this pattern, every new parsing challenge becomes a question of "what tags contain the data I need?" rather than "how do I get data into a DataFrame?"

The conditional checks (`if m.find("first-name") else None`) protect against missing tags — a common issue with real-world XML where not every record contains every field. Without these checks, a single missing tag would crash your entire extraction loop with an `AttributeError`.

Once the data is in a DataFrame, you have the full power of pandas at your disposal. You can filter members by state (`df[df["state"] == "CO"]`), compute cross-tabulations of party by state (`pd.crosstab(df["state"], df["party"])`), or merge Congressional data with external datasets like election results or demographic information. The DataFrame is the universal intermediate format — regardless of whether your data started as XML, JSON, HTML, or PDF text, getting it into a DataFrame is the step that unlocks analysis. This chapter introduces the pattern with XML; the chapters that follow apply it to every other data source on the web.

4.3 JSON and Python Data Structures

4.3.1 Lists and Dictionaries

JSON maps onto two Python data structures you already know. A JSON **array** is a Python **list** — ordered, indexed by position:

```
classroom = ["Alice", "Bob", "Carol"]
print(classroom[0])   # "Alice"
print(len(classroom))  # 3
```

A JSON **object** is a Python **dictionary** — unordered, accessed by key:

```
state = {
    "name": "Colorado",
    "capital": "Denver",
    "population": 5773714
}
print(state["capital"])     # "Denver"
print(state.keys())         # dict_keys(['name', 'capital', 'population'])
```

4.3.2 Nested Structures

Real-world JSON data nests these structures inside each other. A dictionary can contain lists, lists can contain dictionaries, and the nesting can go several levels deep:

```
schools = {
    "School 1": {
        "Classroom A": ["Alice", "Bob"],
        "Classroom B": ["Carol", "Dave", "Eve"]
    },
    "School 2": {
        "Classroom A": ["Frank", "Grace"],
        "Classroom B": ["Heidi", "Ivan", "Judy"]
    }
}

# Access the 3rd student in School 2, Classroom B
print(schools["School 2"]["Classroom B"][2])  # "Judy"
```

The skill of navigating nested structures — working from the outside in, one key or index at a time — is essential for parsing API responses. When you encounter a complex JSON response, start by checking `type()` and `.keys()` at the top level, then work your way down.

4.3.3 The List-of-Dictionaries Pattern

The most common JSON pattern you will encounter in web data is a list of dictionaries, where each dictionary represents a record and each key-value pair represents a field:

```
census_data = [
    {"state": "Colorado", "year": 2020, "population": 5773714},
    {"state": "Colorado", "year": 2010, "population": 5029196},
    {"state": "Colorado", "year": 2000, "population": 4301261}
]

# Access the population in 2020
print(census_data[0]["population"])  # 5773714
```

This pattern converts directly into a pandas DataFrame:

```
import pandas as pd

df = pd.DataFrame(census_data)
print(df)
#        state  year  population
# 0  Colorado  2020     5773714
# 1  Colorado  2010     5029196
# 2  Colorado  2000     4301261
```

This list-of-dictionaries-to-DataFrame pipeline is the standard way to convert API responses into analyzable data. You will use it in nearly every chapter from here forward.

4.3.4 Parsing JSON Strings

When you receive JSON data from the web (as a string), you need to parse it into Python objects:

```
import json

# Parse a JSON string into a Python object
json_string = '{"name": "Colorado", "population": 5773714}'
data = json.loads(json_string)  # loads = "load string"
print(type(data))    # <class 'dict'>
print(data["name"])  # "Colorado"
```

```
# The reverse: convert a Python object to a JSON string
json_output = json.dumps(data, indent=2)  # dumps = "dump string"
print(json_output)
```

When working with `requests`, the `.json()` method on a response object does this parsing for you automatically:

```
response = requests.get("https://api.example.com/data")
data = response.json()  # Equivalent to json.loads(response.text)
```

4.3.5 Working with a Real JSON Object

Here is a simplified version of a social media post object, similar to what Twitter's API used to return:

```
tweet = {
    "id": 266031293945503744,
    "text": "Four more years.",
    "created_at": "Wed Nov 07 04:49:06 +0000 2012",
    "user": {
        "name": "Barack Obama",
        "screen_name": "BarackObama",
        "followers_count": 23456789
    },
    "entities": {
        "hashtags": [],
        "user_mentions": [],
        "urls": []
    },
    "retweet_count": 814974,
    "favorite_count": 302015
}

# Top-level keys
print(tweet.keys())
# dict_keys(['id', 'text', 'created_at', 'user', 'entities', ...])

# Nested access
print(tweet["user"]["screen_name"])  # "BarackObama"
print(tweet["created_at"])            # "Wed Nov 07 04:49:06 +0000 2012"
```

```
# Check for mentions
print(len(tweet["entities"]["user_mentions"]))  # 0
```

4.3.6 Handling Deeply Nested JSON

The Twitter example above is only two levels deep. Real API responses often nest data three or four levels down, and the path to the data you need is not always obvious. Let us work through a live example using the Open-Meteo weather API, which returns a forecast for Boulder, Colorado:

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 40.01,
    "longitude": -105.27,
    "daily": "temperature_2m_max,temperature_2m_min",
    "timezone": "America/Denver"
}

response = requests.get(url, params=params)
data = response.json()

# Start by understanding the top-level structure
print(type(data))       # <class 'dict'>
print(data.keys())      # dict_keys(['latitude', 'longitude', 'daily', 'daily_units', ...])

# The daily data is nested one level down
print(type(data["daily"]))      # <class 'dict'>
print(data["daily"].keys())     # dict_keys(['time', 'temperature_2m_max', 'temperature_2m_mi

# The actual values are lists inside the 'daily' dictionary
print(data["daily"]["time"][:3])              # ['2024-01-15', '2024-01-16', '2024-01-17']
print(data["daily"]["temperature_2m_max"][:3]) # [5.2, 8.1, 3.7]

# Use .get() for safe access with a default
elevation = data.get("elevation", "unknown")
print(f"Elevation: {elevation} meters")
```

The strategy is always the same: start with `type()` and `.keys()` at the top level, then work inward one key at a time. When you find the lists you need, convert them to a DataFrame:

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "date": pd.to_datetime(data["daily"]["time"]),
    "high": data["daily"]["temperature_2m_max"],
    "low": data["daily"]["temperature_2m_min"],
})

plt.figure(figsize=(10, 4))
plt.plot(df["date"], df["high"], label="High", color="red")
plt.plot(df["date"], df["low"], label="Low", color="blue")
plt.fill_between(df["date"], df["low"], df["high"], alpha=0.2)
plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.title("Boulder, CO - 7-Day Forecast")
plt.legend()
plt.tight_layout()
plt.show()
```

The `.get()` method is your most important tool for navigating unfamiliar JSON. Unlike bracket notation (`data["key"]`), which raises a `KeyError` if the key is missing, `.get("key", default)` returns the default value silently. This is especially useful when API responses vary — some records might include optional fields that others omit.

Notice the general strategy for tackling any unfamiliar JSON response. First, check the type and keys at the top level. Then pick the key that looks most relevant and repeat the process one level deeper. Continue until you find the lists of values you need. This "peel the onion" approach works regardless of how deeply nested the data is, and it avoids the common mistake of trying to write complex extraction code before you understand the structure. You will use this strategy repeatedly in Chapter 10, Chapter 11, and Chapter 12 when working with APIs that return richly structured JSON responses.

4.4 When to Use Which

XML tends to appear in older web services, government data feeds (congressional records, SEC filings, RSS feeds), and configuration files. JSON dominates modern web APIs, database exports, and any context where data is being exchanged between web services. HTML — which you will parse extensively beginning in Chapter 6 — is structurally similar to XML and is parsed with the same BeautifulSoup methods.

Public Interest Connection

The choice of data format is itself a transparency decision. When governments and institutions publish data in machine-readable XML or JSON, they embody **openness** — making information practically accessible to anyone with basic programming skills. When the same data is published as scanned PDFs or proprietary spreadsheets, it creates barriers to the kind of **oversight** that democratic accountability requires (Chapter 3). Advocating for open data formats is one of the most concrete things a data scientist can do to serve the public interest.

Missing Manual Reference

For more on data file formats including CSV, TSV, and other common structures, see *Missing Manual* Chapter 20: Data File Formats.

4.5 Exercises

1. **RSS feed parsing.** Find an RSS feed from a news outlet (most news sites have one at `/rss` or `/feed`). Retrieve it with `requests`, parse it with BeautifulSoup (using the `"xml"` parser), and extract the title, link, and publication date for each item. Convert the results to a DataFrame.

2. **Nested JSON navigation.** The following URL returns weather data in JSON format: `https://api.open-meteo.com/v1/forecast?latitude=40.01&longitude=-105.27&daily=temperature_2m_max&timezone=America/Denver`. Retrieve the data, navigate the nested JSON response, and create a DataFrame with dates and maximum temperatures for Boulder, CO.

3. **XML vs. JSON comparison.** Find a data source that provides the same information in both XML and JSON formats (e.g., a government API or data service). Retrieve both versions and compare: how does the structure differ? Which is easier to parse in Python? Which contains more metadata?

4. **Reusable extraction function.** Write a function `xml_to_dataframe(soup, tag_name, fields)` that takes a BeautifulSoup object, the name of the repeating tag (e.g., `"member"`), and a list of child tag names to extract (e.g., `["first-name", "last-name", "party"]`). The function should return a DataFrame. Test it on the Congressional XML from this chapter, then try it on an RSS feed (where the repeating tag is `"item"` and the fields might be `["title", "link", "pubDate"]`).

5. **Weather data analysis.** Retrieve a 7-day forecast from the Open-Meteo API for Boulder, CO using the URL pattern from this chapter. Navigate the nested JSON

response, build a DataFrame with dates and both high and low temperatures, and create a plot showing both temperature series. Add a horizontal line at 0°C and annotate any days where the low drops below freezing.

4.6 Social History and Public Interest

XML emerged in the late 1990s from the standardization efforts of the World Wide Web Consortium (W3C), designed to be a universal format for document and data exchange. Its verbosity made it rigorous but heavy. JSON was created by Douglas Crockford in the early 2000s as a lightweight alternative that leveraged JavaScript's native object syntax. Its simplicity and the rise of AJAX-powered web applications made it the default format for APIs by the 2010s.

The politics of data formats may seem esoteric, but format choices have real consequences for access and transparency. When a government publishes budget data as XML, it is machine-readable by design. When it publishes the same data as a scanned PDF, it is technically "public" but practically inaccessible — a form of the soft enclosure discussed in Chapter 3. Open government data initiatives like data.gov have pushed for machine-readable formats precisely because format determines who can actually use the data.

RSS (Really Simple Syndication) deserves special mention as an early experiment in open data distribution. Built on XML, RSS feeds allowed anyone to subscribe to updates from blogs, news sites, and podcasts without visiting each site individually. In the mid-2000s, RSS was the backbone of the blogosphere and an early tool for computational journalism — researchers could subscribe to thousands of news feeds and analyze coverage patterns at scale. But RSS declined as social media platforms offered their own proprietary feed algorithms. Facebook, Twitter, and later platforms replaced the open, user-controlled subscription model with opaque algorithmic curation. The decline of RSS was one of the earliest examples of the enclosure pattern that Chapter 3 describes: an open protocol replaced by a proprietary system that centralized control over information distribution.

4.7 Common Issues to Debug

- **Parser selection**: Use `"xml"` for XML documents and `"html.parser"` or `"lxml"` for HTML. Using the wrong parser produces unexpected results.
- `json.JSONDecodeError`: The string is not valid JSON. Common causes include trailing commas, single quotes (JSON requires double quotes), or the response containing HTML error pages instead of JSON.
- **`KeyError` in dictionaries**: The key does not exist. Use `.get("key", default_value)` for safe access.

- **Confusion between `json.loads()` and `json.load()`**: `loads` parses a **s**tring; `load` parses a **f**ile object.

4.8 Key Takeaways

XML and JSON are the two structural languages of web data. XML organizes data as a tree of nested tags; BeautifulSoup makes that tree navigable through `.find()`, `.find_all()`, and `.text`. JSON organizes data as nested lists and dictionaries; Python's native data structures map directly onto it. The skill of converting nested structures into flat DataFrames — the list-of-dictionaries-to-DataFrame pipeline — is the bridge between raw web data and analysis, and you will use it in nearly every subsequent chapter.

4.9 Further Reading

- BeautifulSoup documentation: https://www.crummy.com/software/BeautifulSoup/bs4/doc/
- Python `json` module: https://docs.python.org/3/library/json.html
- W3C XML specification: https://www.w3.org/XML/
- JSON specification: https://www.json.org/
- @mitchell2018web — Chapter 1: Your First Web Scraper

5 Web Architecture and Protocols

Learning Objectives

- Describe the client-server model and trace the sequence of operations from URL to rendered page
- Use browser developer tools to examine page structure, HTTP requests, and response headers
- Explain the roles of TCP/IP, DNS, and HTTP in web communication
- Construct HTTP requests with custom headers using `requests` and parse URL components with `urllib.parse`
- Make an API call and convert the response to a time series visualization

5.1 What Happens When You Load a Web Page?

When you type a URL into your browser and press Enter, a remarkable sequence of operations unfolds in milliseconds. Your browser resolves the domain name to an IP address, opens a connection to a remote server, sends a request formatted in a specific protocol, receives a response, and renders the result on your screen. Understanding this sequence transforms web data collection from magic into engineering.

This chapter walks through each layer of the stack: the **client-server model** that structures the conversation, **TCP/IP** that transports the data, **DNS** that resolves names to addresses, **HTTP** that structures requests and responses, and **URLs** that identify what you are asking for. You will use browser developer tools to observe this machinery in action, then use Python libraries to replicate what your browser does programmatically.

5.2 The Client-Server Model

The web follows a client-server architecture. A **client** (your browser, your Python script) sends a **request** to a **server** (a computer hosting a website or API). The server processes the request and sends back a **response**. This request-response cycle is the fundamental unit of web communication, and every scraping and API operation you perform in this book is an instance of it.

Your browser is a sophisticated client that handles many things automatically — resolving domain names, managing cookies, rendering HTML and CSS, executing JavaScript. When you use `requests.get()` in Python, you are acting as a much simpler client that only handles the HTTP layer. Understanding what your browser does behind the scenes will help you diagnose problems when your Python client gets different results than your browser.

5.3 Browser Developer Tools

Before writing code to scrape a page, you should always inspect it manually using your browser's developer tools. Every modern browser includes these tools:

- **Windows/Linux**: Ctrl + Shift + I or F12
- **Mac**: + + I

5.3.1 The Inspector Tab

The Inspector shows the HTML structure of the page as a tree. You can hover over elements to highlight them on the page, expand nested tags to explore the structure, and identify the CSS selectors or element attributes you will need for scraping. Start with a simple page like a Wikipedia article — most commercial websites are full of tracking scripts and dynamic content that obscure the actual structure.

5.3.2 The Network Tab

The Network tab reveals all the HTTP requests your browser makes to load a page. When you reload a page with the Network tab open, you will see dozens or even hundreds of requests — not just for the HTML, but for CSS stylesheets, JavaScript files, images, fonts, and tracking pixels. Each request shows:

- The URL being requested
- The HTTP method (GET, POST, etc.)
- The status code of the response (200 OK, 404 Not Found, etc.)
- The response headers, including `Content-Type`, `Set-Cookie`, and `User-Agent`

Click on any individual request to inspect its headers. Pay particular attention to the `User-Agent` header (how your browser identifies itself) and any `Cookie` headers (session information that the server uses to recognize you).

> **Missing Manual Reference**
>
> For a broader introduction to how HTTP and web APIs work, see *Missing Manual* Chapter 24: HTTP and Web APIs.

5.4 TCP/IP: The Transport Layer

Every computer on a public network has an IP address — a numerical identifier that functions like a postal address. IPv4 addresses are formatted as four numbers separated by dots (e.g., `151.101.130.133`), allowing for about 4.3 billion unique addresses. IPv6 uses a longer hexadecimal format to support a vastly larger address space.

You can find your own computer's IP address programmatically:

```
import socket

# Your computer's hostname on the local network
hostname = socket.gethostname()
print(f"Hostname: {hostname}")

# Your local IP address
local_ip = socket.gethostbyname(hostname)
print(f"Local IP: {local_ip}")
```

To see your *public* IP address (as seen by servers on the internet), you can query an external service:

```
import requests

response = requests.get("https://api.ipify.org?format=json")
print(response.json())
# {'ip': '128.138.xxx.xxx'} - your public IP
```

5.4.1 Traceroute

When you send a request to a remote server, the data does not travel in a straight line. It hops through a series of intermediate routers. You can trace this path using the `scapy` library:

```python
from scapy.all import traceroute

# Trace the path to Wikipedia
# Note: scapy requires root/administrator access on some systems
result, unans = traceroute("en.wikipedia.org", maxttl=20)
```

Each hop represents a router or server along the path. You can use an IP geolocation service to map these hops geographically:

```python
import requests

def geolocate_ip(ip):
    """Look up the geographic location of an IP address."""
    response = requests.get(f"http://ip-api.com/json/{ip}")
    return response.json()

# Note: ip-api.com limits you to 45 requests per minute
location = geolocate_ip("151.101.130.133")
print(f"{location['city']}, {location['country']}")
```

> **❗ Important**
>
> `scapy` requires root/administrator access to send and receive raw network packets. On macOS or Linux, you may need to run `sudo jupyter notebook` from the terminal. On Windows, launch Anaconda Prompt as Administrator. See the Chapter 4 notebook for platform-specific instructions.

5.5 DNS: The Address Book

The Domain Name System translates human-readable domain names (like `en.wikipedia.org`) into IP addresses that computers can route to. You can perform DNS lookups programmatically with the `dnspython` library:

```python
import dns.resolver

# Look up the IP address for Wikipedia
answers = dns.resolver.resolve("en.wikipedia.org", "A")
for answer in answers:
    print(f"IP address: {answer}")
```

Domain names have a hierarchical structure: `subdomain.domain.top-level-domain`. The top-level domains (`.com`, `.org`, `.edu`) are managed by ICANN. The domain (`wikipedia`) is registered by the organization. Subdomains (`en`, `es`, `m`) are controlled by the domain owner and do not require separate registration.

5.5.1 The Structure of Domain Names

Understanding this hierarchy helps you navigate the web programmatically. Consider two examples:

- `en.wikipedia.org` — The TLD is `.org` (a generic TLD for organizations). The domain is `wikipedia` (registered by the Wikimedia Foundation). The subdomain `en` indicates the English-language edition. Other subdomains like `es`, `de`, or `m` (mobile) are all controlled by Wikipedia and do not require separate registration. The API lives at a different subdomain entirely: `wikimedia.org`.
- `api.census.gov` — The TLD is `.gov` (restricted to U.S. government entities). The domain is `census`. The subdomain `api` indicates this is the programmatic access point rather than the public-facing website at `www.census.gov`.

When you construct URLs for scraping or API calls, knowing which part of the domain identifies the service (the domain) versus which part routes you to a specific feature (the subdomain and path) helps you anticipate URL patterns. Government APIs often live at `api.agency.gov`, data portals at `data.agency.gov`, and documentation at `developers.agency.gov`. Recognizing these conventions saves you time when exploring new data sources.

5.6 HTTP: The Application Layer

HTTP (Hypertext Transfer Protocol) is the language that clients and servers use to communicate. A client sends a **request** with a method (GET, POST, PUT, DELETE), headers (metadata), and optionally a body. The server sends a **response** with a status code, headers, and a body.

The status codes you will encounter most often:

- **200 OK**: The request succeeded and the response body contains the data
- **301/302 Redirect**: The resource has moved; follow the `Location` header
- **403 Forbidden**: The server understood your request but refused to fulfill it (often a User-Agent issue)
- **404 Not Found**: The resource does not exist at this URL
- **429 Too Many Requests**: You are being rate-limited; slow down
- **500 Internal Server Error**: Something went wrong on the server side

5.6.1 Diagnosing HTTP Errors

When a request fails, the status code tells you *what* went wrong, but diagnosing *why* requires looking at the full response. A diagnostic function can help:

```
import requests

def diagnose_request(url, headers=None):
    """Print diagnostic information about an HTTP request."""
    try:
        response = requests.get(url, headers=headers, timeout=10)
        print(f"URL: {url}")
        print(f"Status: {response.status_code}")
        print(f"Content-Type: {response.headers.get('Content-Type', 'not specified')}")
        print(f"Server: {response.headers.get('Server', 'not specified')}")
        print(f"Response size: {len(response.content):,} bytes")
        if response.history:
            print(f"Redirected from: {response.history[0].url}")
        return response
    except requests.exceptions.ConnectionError:
        print(f"Connection failed: {url}")
    except requests.exceptions.Timeout:
        print(f"Request timed out: {url}")

# Test on a few URLs
diagnose_request("https://en.wikipedia.org/wiki/Python_(programming_language)")
diagnose_request("https://en.wikipedia.org/wiki/Nonexistent_Page_12345")
```

The most common errors you will encounter in practice are 403 (Forbidden) and 429 (Too Many Requests). A 403 often means the server detected your request as automated — either because you are missing a `User-Agent` header or because the site blocks scraping entirely. A 429 means you are hitting the server too frequently. For 429 errors, a retry-with-backoff strategy helps:

```
import time

def get_with_backoff(url, headers=None, max_retries=3):
    """Retry a request with exponential backoff on 429 errors."""
    for attempt in range(max_retries):
        response = requests.get(url, headers=headers)
        if response.status_code == 429:
            wait_time = 2 ** attempt  # 1s, 2s, 4s
            print(f"Rate limited. Waiting {wait_time}s before retry...")
```

```
            time.sleep(wait_time)
        else:
            return response
    return response  # Return the last response even if still 429
```

Exponential backoff doubles the wait time with each retry, giving the server progressively more breathing room. Many APIs include a `Retry-After` header in their 429 responses that tells you exactly how long to wait — check for it with `response.headers.get("Retry-After")` before falling back to your own backoff schedule.

Understanding status codes in context makes debugging much faster. A 403 from the Wikimedia API almost always means you forgot the `User-Agent` header — the fix is a single line of code. A 403 from a news site might mean the site blocks all automated access, and no header change will help. A 404 could mean the page was deleted, the URL has a typo, or the site restructured its paths. A 500 is the server's problem, not yours — retry after a delay. Building this diagnostic intuition takes practice, but the pattern is always the same: check the status code, inspect the response headers, read the response body for error messages, and adjust your request accordingly. This systematic approach to debugging carries through every chapter that follows.

5.6.2 Making API Requests with Custom Headers

Let us retrieve data from the Wikimedia pageviews API. This API requires a meaningful `User-Agent` header — if you send the default `requests` User-Agent, you will get a 403 error:

```
import requests

article = "University_of_Colorado_Boulder"
api_url = (
    f"https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
    f"/en.wikipedia/all-access/all-agents/{article}/daily/20240101/20240131"
)

# This will likely fail with a 403
response = requests.get(api_url)
print(response.status_code)  # 403

# Add a custom User-Agent header
headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
response = requests.get(api_url, headers=headers)
print(response.status_code)  # 200
```

```
# Parse the JSON response
data = response.json()
```

Now convert the response to a DataFrame and plot it:

```
import pandas as pd
import matplotlib.pyplot as plt

# The pageview data is nested inside data['items']
df = pd.DataFrame(data["items"])
df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y%m%d00")

plt.figure(figsize=(10, 4))
plt.plot(df["timestamp"], df["views"])
plt.title(f"Daily Pageviews: {article}")
plt.xlabel("Date")
plt.ylabel("Views")
plt.tight_layout()
plt.show()
```

5.7 URLs: Anatomy and Parsing

URLs (Uniform Resource Locators) have a well-defined structure:

```
scheme://userinfo@host:port/path?query#fragment
```

Never write regular expressions to parse URLs. Use the `urllib.parse` module instead:

```
from urllib.parse import urlparse, parse_qs

url = "https://api.census.gov/data/2022/acs/acs5?get=NAME,B01001_001E&for=place:07850&in=sta

parsed = urlparse(url)
print(f"Scheme: {parsed.scheme}")      # https
print(f"Host: {parsed.netloc}")        # api.census.gov
print(f"Path: {parsed.path}")          # /data/2022/acs/acs5
print(f"Query: {parsed.query}")        # get=NAME,...

# Parse query parameters into a dictionary
```

```
params = parse_qs(parsed.query)
print(params)
# {'get': ['NAME,B01001_001E'], 'for': ['place:07850'], 'in': ['state:08']}
```

Understanding URL structure is essential for constructing API requests. In Chapter 10 through Chapter 13, you will build URLs programmatically by assembling scheme, host, path, and query parameters.

5.7.1 Building URLs Programmatically

While you can construct URLs with f-strings, this approach breaks when values contain special characters like spaces, ampersands, or non-ASCII text. The `urllib.parse` module provides tools for building URLs safely:

```
from urllib.parse import urlencode

# Build a Wikimedia API URL from components
base_url = "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
article = "University of Colorado Boulder"  # Note the spaces

# urlencode handles special characters automatically
params = {
    "project": "en.wikipedia",
    "access": "all-access",
    "agent": "all-agents",
    "granularity": "daily",
    "start": "20240101",
    "end": "20240131",
}
query_string = urlencode(params)
print(query_string)
# project=en.wikipedia&access=all-access&agent=all-agents&...
```

The `requests` library also accepts a `params` dictionary that handles URL encoding automatically — this is the preferred approach for API calls:

```
response = requests.get(base_url, params=params, headers=headers)
print(response.url)  # Shows the full URL that was actually requested
```

Using `params` instead of manually constructing query strings prevents a category of bugs where special characters in your search terms break the URL. An article title like "Guns N' Roses"

or a search query with an ampersand would silently corrupt a hand-built URL; `urlencode` and the `params` argument handle these cases correctly. You will appreciate this distinction especially in Chapter 10 and Chapter 11, where article titles and variable names frequently contain characters that require encoding.

5.8 Exercises

1. **Browser developer tools.** Open a Wikipedia article and a commercial news site side by side, each with the Network tab open. Reload both pages and compare: how many requests does each make? What types of resources (HTML, CSS, JS, images, fonts, tracking) does each load? What does this tell you about modern web architecture?
2. **DNS exploration.** Use `dns.resolver` to look up the IP addresses for five related domains (e.g., `google.com`, `youtube.com`, `gmail.com`, `drive.google.com`, `cloud.google.com`). Are any of them hosted at the same IP address? What does this tell you about Google's infrastructure?
3. **Wikimedia pageviews.** Using the API pattern from this chapter, retrieve and plot daily pageview data for three related Wikipedia articles over the same time period. Identify any spikes and hypothesize about what real-world events caused them.
4. **Network tab analysis.** Pick a website you visit frequently. Reload it with the browser's Network tab open and recording. Count and categorize all requests into types: HTML documents, CSS stylesheets, JavaScript files, images, fonts, and tracking/analytics scripts. What fraction of requests serve actual content vs. tracking and advertising? Write a brief analysis of what this reveals about the site's business model.
5. **URL diagnostic function.** Write a function `diagnose_url(url)` that prints the status code, Content-Type, Server header, response size in bytes, and whether the URL redirected (and if so, to where). Test it on at least five URLs: a working page, a 404, a redirecting URL, an API endpoint, and a site that blocks automated access. Summarize the patterns you observe.

5.9 Social History and Public Interest

The protocols described in this chapter were designed as public infrastructure. Vint Cerf and Bob Kahn developed TCP/IP in the 1970s for DARPA, creating a transport layer engineered for resilience under attack. Tim Berners-Lee invented HTTP and URLs at CERN in 1989–1991, and made the deliberate decision to release them into the public domain. The web was born open.

Browser developer tools embody this original spirit of openness. The ability to "view source" — to inspect exactly what a web server sends your browser — was a foundational feature of the web. It is what makes web data science possible. As you will see in Chapter 3, the tension between this transparency and platforms' desire to control their data is one of the defining conflicts of the contemporary web.

> **Public Interest Connection**
>
> The web's protocol stack was designed as public infrastructure — open standards documented in public RFCs, not proprietary products controlled by corporations. HTTP, HTML, DNS, and TCP/IP are all open protocols that anyone can implement. The "view source" capability and network inspection tools you learned in this chapter are features that embody **openness** (Chapter 3). They exist because the web's architects believed that transparency was a feature, not a vulnerability. As platforms increasingly obfuscate their front-end code and move toward closed APIs, these tools become more important, not less.

5.10 Common Issues to Debug

- **`scapy` import errors**: Requires `pcap` drivers. On Windows, install Npcap. On macOS, the built-in `libpcap` usually works.
- **403 from Wikimedia API**: You forgot the custom `User-Agent` header.
- **DNS results varying by location**: Normal — content delivery networks return different IPs based on your geographic location.
- **URL encoding issues**: Use `urllib.parse.quote()` for article titles with spaces or special characters.

5.11 Key Takeaways

The web is built on a layered stack of protocols: TCP/IP transports data, DNS resolves names to addresses, HTTP structures the conversation, and URLs identify the target. Understanding this stack transforms debugging from guesswork into systematic diagnosis. When a request fails, you can ask: can I reach the server at all? Am I resolving the right address? What status code did I get? Are my headers correct? Is my URL well-formed? This systematic approach carries through every chapter that follows.

5.12 Further Reading

- Mozilla Developer Network — How the Web Works: https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/How_the_Web_works
- Mozilla Developer Network — HTTP Overview: https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- `scapy` documentation: https://scapy.readthedocs.io/
- `dnspython` documentation: https://dnspython.readthedocs.io/

Part II

Documents

6 Parsing Static Web Pages

Learning Objectives

- Use browser developer tools to identify HTML elements containing target data
- Extract data from HTML tables manually using BeautifulSoup's tag navigation
- Use `pandas.read_html()` as a shortcut for well-structured tables
- Write a custom HTML parser for non-tabular data using DOM tree navigation
- Abstract repeated scraping logic into reusable functions with error handling

6.1 From HTML to Data

The overall goal of web scraping is converting data from one structured format — HTML, the language of web pages — into another structured format suitable for analysis: typically a pandas DataFrame, a CSV file, or a JSON document. This chapter teaches three strategies for that conversion, each appropriate for different situations and progressively more general.

HTML is essentially a dialect of XML (which you learned to parse in Chapter 4), so the BeautifulSoup skills you already have transfer directly. The difference is that real-world HTML is far messier than the clean XML examples from the previous chapter. Web pages contain navigation bars, advertisements, tracking scripts, and deeply nested `<div>` tags that obscure the data you care about. Learning to cut through this noise is the core skill of this chapter.

6.2 Strategy 1: Manual Table Parsing

Many interesting datasets live in HTML tables — economic data, sports statistics, legislative records, demographic breakdowns. Let us extract box office revenue data from The Numbers, a popular source of movie financial data.

6.2.1 Retrieving and Inspecting

```python
import requests
from bs4 import BeautifulSoup

url = "https://www.the-numbers.com/box-office-chart-weekly-2018-christmas"
headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")
```

Before writing parsing code, inspect the page in your browser's developer tools. Right-click on the table and select "Inspect" to see the HTML structure. You are looking for the `<table>` tag and, within it, `<tr>` (table row) and `<td>` (table data) tags.

```python
# Find all tables on the page
tables = soup.find_all("table")
print(f"Number of tables found: {len(tables)}")
# Often there are navigation tables mixed in with data tables

# The main data table is usually the largest one
# Inspect each to find the right one
for i, table in enumerate(tables):
    rows = table.find_all("tr")
    print(f"Table {i}: {len(rows)} rows")
```

6.2.2 Extracting Row Data

Once you have identified the correct table, iterate through its rows and extract the text from each cell:

```python
# Assuming the data table is tables[1] (verify by inspection)
data_table = tables[1]
rows = data_table.find_all("tr")

# Inspect the first row (headers)
header_row = rows[0]
print(header_row.text)

# Inspect a data row
data_row = rows[1]
print(data_row.text)
# You'll see newline characters between cell values
```

```
# Clean extraction: split text on newlines and filter empties
cells = [cell.strip() for cell in data_row.text.split("\n") if cell.strip()]
print(cells)
```

6.2.3 Building the DataFrame

```
# Loop through all rows, extract cleaned cell data
all_rows = []
for row in rows:
    cells = [cell.strip() for cell in row.text.split("\n") if cell.strip()]
    if cells:  # Skip empty rows
        all_rows.append(cells)

import pandas as pd

# First row as headers, rest as data
df = pd.DataFrame(all_rows[1:], columns=all_rows[0])
print(df.head())
```

This manual approach gives you complete control over the extraction process. The tradeoff is that it requires more code and careful attention to the specific HTML structure of each page.

6.3 Strategy 2: `pandas.read_html()`

For well-structured HTML tables, pandas provides a powerful shortcut. The `read_html()` function finds all tables on a page and converts each one to a DataFrame:

```
import pandas as pd

# Point read_html directly at a URL - no requests or BeautifulSoup needed
url = "https://en.wikipedia.org/wiki/2018_in_film"
tables = pd.read_html(url)

print(f"Number of tables found: {len(tables)}")
# Wikipedia pages often have many tables
```

You can specify which row contains the column headers and select the table you want by index:

```
# The highest-grossing films table might be at index 3 (verify by inspection)
films_df = pd.read_html(url, header=0)[3]
print(films_df.head())
```

The `read_html` function is remarkably powerful for its simplicity, but it has limitations. It struggles with tables that use `colspan` or `rowspan` attributes to merge cells — for example, a table where "Disney" spans two rows to cover multiple films. In those cases, you may need to fall back to manual parsing or clean up the DataFrame after extraction.

6.3.1 Cleaning Up read_html Results

In practice, `read_html` rarely produces a perfect DataFrame. Column names may contain whitespace or be split across multiple header rows. Numeric columns often parse as strings because of dollar signs, commas, or percentage symbols. Merged cells create NaN values that need to be filled. Here is a typical cleanup workflow:

```
# Retrieve and extract the table
tables = pd.read_html(url, header=0)
df = tables[3]  # Verify the correct index by inspection

# Rename columns to something usable
df.columns = [col.strip().lower().replace(" ", "_") for col in df.columns]

# Drop rows that repeat the header (common in long Wikipedia tables)
df = df[df.iloc[:, 0] != df.columns[0]]

# Convert numeric columns that parsed as strings
df["worldwide_gross"] = pd.to_numeric(
    df["worldwide_gross"].str.replace(r"[\$,]", "", regex=True),
    errors="coerce"
)

# Fill NaN values from merged cells (forward-fill)
df["studio"] = df["studio"].fillna(method="ffill")

print(df.dtypes)
print(df.head())
```

The `errors="coerce"` argument in `pd.to_numeric()` is essential: it converts unparseable values to NaN instead of raising an error, which lets your cleanup pipeline continue even when some cells contain unexpected text. The forward-fill (`ffill`) operation handles the common

Wikipedia pattern where a studio name spans multiple rows — the name appears once, and subsequent rows are blank until the next studio. You should expect to spend as much time cleaning `read_html` output as you spent extracting it — this is normal and not a sign that something went wrong. The function handles the hard part (parsing HTML tables); the cleanup is your job.

6.4 Strategy 3: Writing a Custom Parser

Many web pages present structured data without using HTML tables. Award nominees, product listings, news articles, and directory entries are often structured with `<div>` tags and CSS classes. Extracting this data requires navigating the HTML tree to find the right containers.

6.4.1 The Oscars Example

The Academy Awards website lists nominees and winners organized by category. The data is structured with nested `<div>` tags rather than `<table>` elements. Let us build a parser for it.

```
import requests
from bs4 import BeautifulSoup

url = "https://www.oscars.org/oscars/ceremonies/2024"
headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")
```

6.4.2 Finding Reliable Anchors

The key challenge with non-tabular scraping is finding HTML elements that reliably contain your target data. Start by using the browser's Inspector tool to examine the structure around one category:

```
# Look for grouping elements - the structure may use classes like
# "view-grouping" or similar containers for each award category
groupings = soup.find_all("div", class_="view-grouping")
print(f"Found {len(groupings)} category groupings")

# If the count doesn't match expectations (24 Oscar categories),
# you may need to navigate to a more specific parent element
```

6.4.3 Navigating the Tree for Specificity

If your initial `find_all` returns too many or too few results, you need to navigate the tree structure to find a more specific anchor. This involves going *up* to a unique parent and then searching *down* within it:

```
# Find the main content area first
content_area = soup.find("div", class_="field--name-field-award-categories")

if content_area:
    # Now search only within this specific container
    categories = content_area.find_all("div", class_="view-grouping")
    print(f"Categories within content area: {len(categories)}")
```

6.4.4 Extracting Category Data

For each category, extract the award name and the list of nominees:

```
results = []

for category in categories:
    # Extract the category name
    name_tag = category.find("div", class_="view-grouping-header")
    if not name_tag:
        continue
    category_name = name_tag.text.strip()

    # Extract individual nominees
    nominees = category.find_all("div", class_="views-row")
    for nominee in nominees:
        nominee_name = nominee.text.strip()
        results.append({
            "category": category_name,
            "nominee": nominee_name
        })

df = pd.DataFrame(results)
print(df.head(10))
```

6.4.5 Abstracting into Functions

Once your parser works for one page, abstract it into a function you can reuse:

```
import time

def scrape_oscar_year(year, headers):
    """Scrape Oscar nominees for a given ceremony year.

    Parameters
    ----------
    year : int
        The ceremony year (e.g., 2024)
    headers : dict
        HTTP headers including User-Agent

    Returns
    -------
    pd.DataFrame
        DataFrame with columns: category, nominee, year
    """
    url = f"https://www.oscars.org/oscars/ceremonies/{year}"

    try:
        response = requests.get(url, headers=headers)
        response.raise_for_status()  # Raise exception for bad status codes
    except requests.RequestException as e:
        print(f"Error fetching {year}: {e}")
        return pd.DataFrame()

    soup = BeautifulSoup(response.text, "html.parser")
    # ... parsing logic ...

    time.sleep(1)  # Respect the server
    return df
```

6.4.6 A Second Parser: Colorado Legislators

Let us build a second parser for a very different page structure. The Colorado General Assembly website at leg.colorado.gov/legislators lists all current state legislators with their names, party affiliations, districts, and chambers:

```python
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = "https://leg.colorado.gov/legislators"
headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")

# Inspect the page structure - legislators are typically in repeating containers
# The exact class names may change; always verify with the Inspector
containers = soup.find_all("div", class_="views-row")
print(f"Found {len(containers)} legislator entries")
```

For each container, extract the relevant fields. The exact tag structure will vary, so the key skill here is using the Inspector to trace from the visible text on the page back to the HTML elements that contain it:

```python
records = []
for container in containers:
    name_tag = container.find("a")
    party_tag = container.find("span", class_="party-initial")
    district_tag = container.find("span", class_="district-number")

    if name_tag:
        records.append({
            "name": name_tag.text.strip(),
            "party": party_tag.text.strip() if party_tag else None,
            "district": district_tag.text.strip() if district_tag else None,
        })

df = pd.DataFrame(records)
print(f"Extracted {len(df)} legislators")
print(df.groupby("party").size())
```

This parser illustrates an important principle: start simple and iterate. Your first attempt at identifying the right containers may return too many or too few results. Compare your count against what you see on the page. If you find 100 containers but the page shows 65 legislators, you are matching extra elements — narrow your search with a more specific class or a parent container. If you find 0, the page may use different class names than you expected, or the content may be loaded dynamically with JavaScript (in which case, see Chapter 8).

Note that the page may have changed since this was written — that is the fundamental challenge of web scraping. If the class names above do not match what you see in the Inspector, that is not a bug in this code; it is a feature of how the web works. The skill you are building is not memorizing specific class names but learning the *process* of inspection, hypothesis, and verification that lets you adapt to any page structure. Think of each parser as a hypothesis about the page's structure — you test it, compare the output to what you expect, and refine until they match.

6.4.7 Error Handling Patterns

As you scrape more pages, you will encounter failures: pages that do not load, elements that are missing, or structures that do not match your expectations. A robust scraper handles these gracefully:

```
def safe_scrape(url, headers, parse_func):
    """Scrape a URL with comprehensive error handling.

    Parameters
    ----------
    url : str
        The URL to scrape
    headers : dict
        HTTP headers
    parse_func : callable
        A function that takes a BeautifulSoup object and returns data

    Returns
    -------
    list or None
        Parsed data, or None if the scrape failed
    """
    try:
        response = requests.get(url, headers=headers, timeout=30)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        print(f"Request failed for {url}: {e}")
        return None

    soup = BeautifulSoup(response.text, "html.parser")

    try:
        return parse_func(soup)
```

```
    except AttributeError as e:
        print(f"Parsing failed for {url}: {e}")
        print("The page structure may have changed - re-inspect with dev tools.")
        return None
    except Exception as e:
        print(f"Unexpected error for {url}: {e}")
        return None
```

The `AttributeError` catch is especially important: it fires when `.find()` returns `None` and you try to call `.text` on it — the single most common scraping error. By catching it explicitly, you get an informative message ("the page structure may have changed") instead of a cryptic traceback. The broader `Exception` catch ensures your scraping loop continues even if an individual page has an unexpected problem.

This pattern separates the *what* (your parsing logic in `parse_func`) from the *how* (request handling, error recovery). You can write a clean parsing function that assumes everything works, then wrap it in `safe_scrape` for production use. This is the same separation-of-concerns principle you saw with the `responsible_get()` function in Chapter 2, and you will use it again in Chapter 7 when building pipelines that retrieve dozens or hundreds of pages.

> ⚠ A Note on Fragility
>
> Websites redesign their HTML regularly. The specific CSS classes and tag structures described here were accurate at the time of writing but may have changed since. If your parser stops working, the first step is always to re-inspect the page in your browser's developer tools and identify what changed. This fragility is a fundamental characteristic of web scraping and a motivation for preferring APIs when they are available (see Chapter 3).

6.5 Exercises

1. **Wikipedia table extraction.** Use `pd.read_html()` to extract a table of your choice from Wikipedia (e.g., a list of countries by population, U.S. states by area, or planets in the solar system). Clean up the resulting DataFrame and create a visualization.

2. **Custom parser.** Write a custom BeautifulSoup parser for a non-tabular web page. Good candidates include a course catalog, a restaurant menu, or a list of elected officials. Document your element selection strategy: why did you choose these specific tags and classes?

3. **Multi-page scraping.** Extend your parser from Exercise 2 to handle multiple pages (e.g., multiple years, multiple categories, or paginated results). Use `time.sleep()` between requests, `try/except` for error handling, and a function to abstract the repeated logic.

4. **Non-English Wikipedia.** Use `pd.read_html()` to scrape a data table from a non-English Wikipedia edition (e.g., the French, Spanish, or Japanese Wikipedia). Does `read_html` handle non-ASCII characters correctly? What cleanup is needed compared to the English version? Write a brief analysis of any encoding issues you encounter.

5. **Multi-page scraper with progress.** Build a scraper that collects data across multiple pages — for example, Oscars results for five different years, or box office data for five different weeks. Include `time.sleep()` between requests, `try/except` for error handling, and a progress indicator that prints which page you are currently scraping. Save the combined results to a CSV file.

> Missing Manual Reference
>
> For a refresher on writing functions, handling exceptions with `try/except`, and structuring reusable code, see *Missing Manual* Chapter 17: Scripting.

6.6 Social History and Public Interest

Web scraping has deep roots in data journalism. Organizations like ProPublica, The Markup, and the Investigative Reporters and Editors (IRE) have built some of their most important investigations on data scraped from public websites — court records, school inspection reports, environmental compliance databases, campaign finance disclosures. The skill of turning a web page into a dataset is, in this tradition, a form of civic empowerment: it enables citizens and journalists to analyze information that is nominally public but practically inaccessible in its raw HTML form.

> Public Interest Connection
>
> Turning a web page into a dataset is a form of **oversight** (Chapter 3). When government agencies publish information as web pages but not as downloadable data, scraping bridges the gap between nominal transparency and practical accountability. The ability to parse a legislative directory into a DataFrame enables analysis of representation, diversity, and political geography that the agency itself may not provide.

The Colorado General Assembly website (leg.colorado.gov) provides a concrete example. Biographical information about legislators — their districts, party affiliations, committee assignments, and contact information — is published on the web, but not in a structured format you can download and analyze. Scraping this data into a DataFrame enables research on representation, diversity, and legislative behavior that would be impossible otherwise.

6.7 Common Issues to Debug

- **Multiple tables on a page**: `read_html` and `find_all("table")` return all tables, including navigation elements. Inspect each to find the one with your data.
- **Character encoding**: Watch for `\xa0` (non-breaking spaces) and other Unicode characters in names and text. Use `.strip()` and `.replace('\xa0', ' ')`.
- **Elements that look unique but are not**: Always verify your `find_all` count against the expected number of results before building your parser.
- **Dynamic content**: If `requests` + BeautifulSoup returns a nearly-empty page, the content may be loaded by JavaScript. See Chapter 8 for Selenium-based solutions.

6.8 Key Takeaways

HTML scraping follows a consistent pattern: retrieve the page, inspect it with developer tools, identify your target elements, extract the data, and convert it to a DataFrame. Start with `read_html()` for tables — it handles most simple cases. Fall back to manual BeautifulSoup parsing when you need more control. Write custom parsers for non-tabular data by navigating the DOM tree to find reliable anchors. Always abstract your parsing logic into functions, handle errors gracefully, and rate-limit your requests. Web pages change, so code defensively.

6.9 Further Reading

- pandas `read_html` documentation: https://pandas.pydata.org/docs/reference/api/pandas.read_html.html
- @mitchell2018web — Chapters 2–5 on HTML scraping patterns
- @vandenbroucke2018practical — best practices for production web scraping

7 Archived Web Pages and the Wayback Machine

Learning Objectives

- Explain the Internet Archive's role as a public infrastructure for web preservation
- Query the Wayback Machine's Availability API and CDX Server API to find historical snapshots
- Retrieve and parse historical page content to track changes over time
- Apply basic NLP techniques (tokenization, stopword removal, bag-of-words) to measure document similarity
- Serialize scraped data to JSON files for persistence and reproducibility

7.1 The Web Is Not Permanent

Web pages change, disappear, and get redesigned constantly. Studies have found that a significant fraction of URLs break within just a few years — a phenomenon called *link rot*. For researchers, this impermanence creates a fundamental challenge: how do you study something that might not be there tomorrow? And how do you study how something has changed over time when only the current version is visible?

The Internet Archive, founded by Brewster Kahle in 1996, addresses this challenge by archiving the web. Its Wayback Machine has captured over 800 billion web pages, creating a historical record of the web that is freely accessible to anyone. For web data scientists, the Wayback Machine is both a data source and a form of the counter-erosion infrastructure described in Chapter 3 — a community-governed resource that preserves access to web content even as platforms and governments take pages down.

7.2 The Availability API

The simplest way to check whether the Wayback Machine has a snapshot of a URL is the Availability API:

```
import requests

url = "https://www.cnn.com"
timestamp = "20000815"  # August 15, 2000

api_url = f"https://archive.org/wayback/available?url={url}&timestamp={timestamp}"
response = requests.get(api_url)
data = response.json()

# Check if a snapshot is available
if data["archived_snapshots"]:
    snapshot = data["archived_snapshots"]["closest"]
    print(f"Snapshot available: {snapshot['available']}")
    print(f"URL: {snapshot['url']}")
    print(f"Timestamp: {snapshot['timestamp']}")
else:
    print("No snapshot available near this date")
```

The API returns the closest snapshot to your requested timestamp. You can then retrieve and parse that historical page just as you would a current page:

```
from bs4 import BeautifulSoup

# Retrieve the archived page
archived_url = snapshot["url"]
archived_response = requests.get(archived_url)
soup = BeautifulSoup(archived_response.text, "html.parser")

# Extract links from the year-2000 version of CNN
links = soup.find_all("a")
print(f"Number of links on CNN in August 2000: {len(links)}")
```

7.2.1 Comparing Multiple Snapshots

A single historical snapshot is interesting; comparing snapshots across time is where the real research value lies. Let us retrieve CNN's homepage at three different points in history and see how the web has evolved:

```
import requests
from bs4 import BeautifulSoup
```

```python
def get_snapshot_stats(url, timestamp, headers=None):
    """Retrieve a Wayback snapshot and compute basic statistics."""
    api_url = f"https://archive.org/wayback/available?url={url}&timestamp={timestamp}"
    api_response = requests.get(api_url, headers=headers)
    api_data = api_response.json()

    if not api_data.get("archived_snapshots"):
        return None

    snapshot_url = api_data["archived_snapshots"]["closest"]["url"]
    page_response = requests.get(snapshot_url, headers=headers)
    soup = BeautifulSoup(page_response.text, "html.parser")

    return {
        "year": timestamp[:4],
        "num_links": len(soup.find_all("a")),
        "text_length": len(soup.get_text()),
        "num_images": len(soup.find_all("img")),
        "num_scripts": len(soup.find_all("script")),
    }

# Compare CNN across three eras
timestamps = ["20000815", "20120815", "20240815"]
stats = []
for ts in timestamps:
    result = get_snapshot_stats("www.cnn.com", ts)
    if result:
        stats.append(result)
    time.sleep(2)  # Be respectful of the Archive
```

```python
import pandas as pd

comparison = pd.DataFrame(stats)
print(comparison)
#   year  num_links  text_length  num_images  num_scripts
# 0  2000        150         8500          20            2
# 1  2012        450        25000          80           35
# 2  2024        800        45000         120           95
```

The numbers tell a story about the web's evolution: pages have grown dramatically in complexity, with more links, more text, more images, and — most strikingly — far more JavaScript. The 2000-era web was largely static HTML; the 2024 web is a JavaScript application that happens

to render as a web page. This increasing complexity is part of why the techniques in Chapter 8 become necessary for modern scraping.

You can extend this approach to any pair of sites or time periods. Comparing how two competing news organizations' homepages evolved reveals different editorial strategies. Tracking a government agency's page across presidential administrations can reveal shifts in priorities and transparency. The Wayback Machine turns the web from a snapshot medium into a longitudinal data source — one where you can measure change rather than merely observing it.

Note the importance of the `time.sleep(2)` call in the loop above. The Internet Archive is a nonprofit that provides this service for free. Aggressive scraping degrades the experience for everyone. Treat the Archive with the same courtesy you would extend to any community resource — use it responsibly, and it will be there when you need it.

7.3 The CDX Server API

The Availability API finds one snapshot near a date. The CDX Server API lets you query the entire history of snapshots for a URL, which is essential for longitudinal research:

```
import requests
import pandas as pd

# Query all snapshots of Facebook's Terms of Service
cdx_url = "https://web.archive.org/cdx/search/cdx"
params = {
    "url": "facebook.com/terms",
    "output": "json",
    "fl": "timestamp,original,statuscode,length"
}

response = requests.get(cdx_url, params=params)
data = response.json()

# First row is column headers, rest is data
df = pd.DataFrame(data[1:], columns=data[0])
df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y%m%d%H%M%S")
df["length"] = pd.to_numeric(df["length"], errors="coerce")

print(f"Total snapshots: {len(df)}")
print(f"Date range: {df['timestamp'].min()} to {df['timestamp'].max()}")
```

You can visualize how frequently the page was captured and how its size changed over time:

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

# Snapshot frequency over time
df.set_index("timestamp").resample("M").size().plot(ax=axes[0])
axes[0].set_ylabel("Snapshots per month")
axes[0].set_title("Wayback Machine Capture Frequency")

# Page size over time
df.set_index("timestamp")["length"].plot(ax=axes[1])
axes[1].set_ylabel("Page size (bytes)")
axes[1].set_title("Page Size Over Time")

plt.tight_layout()
plt.show()
```

7.3.1 Filtering and Analyzing CDX Data

The raw CDX data often includes failed requests, redirects, and duplicate captures that can skew your analysis. Filtering to only successful captures (status code 200) and analyzing trends gives you a cleaner picture:

```
# Filter to successful captures only
df_ok = df[df["statuscode"] == "200"].copy()
print(f"Successful captures: {len(df_ok)} out of {len(df)} total")

# Resample to quarterly frequency for trend analysis
df_ok = df_ok.set_index("timestamp")
quarterly = df_ok.resample("QS").agg(
    captures=("length", "size"),
    mean_size=("length", "mean"),
)

# Compute year-over-year page size growth
yearly = df_ok.resample("YS").agg(mean_size=("length", "mean"))
print(yearly.tail(10))
```

```
fig, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

quarterly["captures"].plot(ax=axes[0])
axes[0].set_ylabel("Captures per quarter")
axes[0].set_title("Capture Frequency (Status 200 Only)")

quarterly["mean_size"].plot(ax=axes[1])
axes[1].set_ylabel("Mean page size (bytes)")
axes[1].set_title("Average Page Size Over Time")

plt.tight_layout()
plt.show()
```

Filtering matters because CDX data includes every capture attempt, not just successful ones. A spike in 301 (redirect) codes might indicate a site restructuring; a cluster of 503 (Service Unavailable) codes might indicate a period of downtime. These metadata are themselves interesting — they tell you about the infrastructure behind the page, not just its content.

Page size trends are particularly revealing. A steady increase in page size often correlates with the addition of tracking scripts, advertising frameworks, and JavaScript-heavy interfaces. A sudden jump might indicate a redesign; a sudden drop might mean the site moved content behind a login wall. When combined with the content analysis techniques later in this chapter, CDX metadata provides the quantitative backbone for your longitudinal research — telling you *how much* changed while bag-of-words analysis tells you *what* changed.

7.4 Tracking Policy Changes Over Time

A current research project explores how social media platforms' terms of service have evolved. Let us track changes in Facebook's Terms of Service by retrieving content at regular intervals.

7.4.1 Generating Date Ranges

```
import pandas as pd

# Generate quarterly dates from 2005 to 2024
dates = pd.date_range(start="2005-01-01", end="2024-01-01", freq="QS")
print(f"Querying {len(dates)} time points")
```

7.4.2 Retrieving Historical Content

```
import time
from datetime import datetime

fb_tos_url = "facebook.com/terms"

def get_wayback_content(url, date, headers=None):
    """Retrieve a page from the Wayback Machine closest to a given date.

    Parameters
    ----------
    url : str
        The original URL (without http://)
    date : datetime
        The target date for the snapshot
    headers : dict, optional
        HTTP headers for the request

    Returns
    -------
    dict
        Dictionary with 'timestamp', 'url', 'content', and 'word_count'
    """
    timestamp = date.strftime("%Y%m%d")
    api_url = f"https://archive.org/wayback/available?url={url}&timestamp={timestamp}"

    api_response = requests.get(api_url, headers=headers)
    api_data = api_response.json()

    if not api_data.get("archived_snapshots"):
        return None

    snapshot_url = api_data["archived_snapshots"]["closest"]["url"]
    page_response = requests.get(snapshot_url, headers=headers)
    soup = BeautifulSoup(page_response.text, "html.parser")

    # Extract text content
    text = soup.get_text(separator=" ", strip=True)

    time.sleep(1)  # Rate limit
```

```
    return {
        "timestamp": api_data["archived_snapshots"]["closest"]["timestamp"],
        "url": snapshot_url,
        "word_count": len(text.split()),
        "content": text[:5000]  # First 5000 characters to avoid memory issues
    }
```

7.4.3 Serializing Data

Once you have collected historical content, save it so you do not need to re-scrape:

```
import json

# After collecting data into a list called 'results':
with open("fb_tos_history.json", "w") as f:
    json.dump(results, f, indent=2)

# Load it back later:
with open("fb_tos_history.json", "r") as f:
    results = json.load(f)
```

7.5 Document Similarity with Bag-of-Words

To measure how much a document changed between versions, you can use a bag-of-words approach. This technique represents each document as a vector of word frequencies, ignoring word order, and then measures the similarity between vectors.

```
from gensim.utils import simple_preprocess
from nltk.corpus import stopwords
import nltk

# Download stopwords (first time only)
# nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

def preprocess_document(text):
    """Tokenize, lowercase, and remove stopwords from a document."""
    tokens = simple_preprocess(text)  # Tokenize and lowercase
    return [token for token in tokens if token not in stop_words]
```

```
# Preprocess two versions of the ToS
doc_v1 = preprocess_document(results[0]["content"])
doc_v2 = preprocess_document(results[-1]["content"])

print(f"Version 1 tokens: {len(doc_v1)}")
print(f"Version 2 tokens: {len(doc_v2)}")
```

Using gensim's dictionary and bag-of-words tools, you can build a corpus and compute similarity:

```
from gensim import corpora, similarities

# Build a dictionary mapping words to IDs
documents = [doc_v1, doc_v2]
dictionary = corpora.Dictionary(documents)

# Convert documents to bag-of-words vectors
corpus = [dictionary.doc2bow(doc) for doc in documents]

# Compute similarity
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dictionary))
sims = index[corpus[0]]
print(f"Similarity between versions: {sims[1]:.3f}")
# 1.0 = identical, 0.0 = completely different
```

7.5.1 Visualizing Document Change Over Time

The bag-of-words similarity measure tells you how similar two documents are, but plotting similarity between *consecutive* versions over time creates a "change timeline" — a visual record of when and how much a document evolved:

```
from gensim import corpora, similarities
from gensim.utils import simple_preprocess
from nltk.corpus import stopwords

stop_words = set(stopwords.words("english"))

# Preprocess all versions
processed_docs = []
for r in results:
```

```
    if r and r.get("content"):
        tokens = [t for t in simple_preprocess(r["content"]) if t not in stop_words]
        processed_docs.append(tokens)

# Build dictionary and corpus
dictionary = corpora.Dictionary(processed_docs)
corpus = [dictionary.doc2bow(doc) for doc in processed_docs]

# Compute similarity between consecutive versions
change_scores = []
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dictionary))
for i in range(1, len(corpus)):
    sim = index[corpus[i-1]][i]
    change_scores.append(1 - sim)  # Convert similarity to change score

# Plot the change timeline
plt.figure(figsize=(12, 4))
plt.plot(range(len(change_scores)), change_scores, marker="o")
plt.xlabel("Version Transition")
plt.ylabel("Change Score (0 = identical, 1 = completely different)")
plt.title("Document Change Over Time")
plt.tight_layout()
plt.show()
```

Spikes in this timeline correspond to major revisions — a new Terms of Service policy, a site redesign, or a significant content update. By cross-referencing these spikes with real-world events (a data breach, a regulatory action, a public controversy), you can begin to tell a story about *why* the document changed, not just *that* it changed. This kind of temporal analysis is at the heart of longitudinal web research.

The change score visualization complements the CDX analysis from earlier in this chapter. CDX data tells you *how often* the page was captured and how its size changed; the change timeline tells you *how different* consecutive versions are from each other. Together, they create a multi-dimensional picture of a document's evolution. A page might be captured frequently but change very little (a stable, well-maintained site), or captured infrequently but show dramatic changes between captures (a site that undergoes periodic major overhauls).

You can also extend this approach beyond consecutive comparisons. Computing pairwise similarity between *all* versions produces a similarity matrix that you can visualize as a heatmap. Clusters of similar versions reveal stable periods; off-diagonal dissimilarities reveal turning points. For research on platform governance, these turning points often align with public controversies, regulatory actions, or leadership changes — patterns that would be invisible without longitudinal data.

The combination of the Wayback Machine's historical data with computational text analysis techniques creates a powerful research methodology. You are not just reading old web pages — you are measuring institutional behavior over time, producing evidence that can inform policy debates, academic research, and public accountability. This is precisely the kind of public interest data science that Chapter 3 describes.

> **Missing Manual Reference**
>
> For more on data serialization formats including JSON, CSV, and pickle, see *Missing Manual* Chapter 20: Data File Formats.

7.6 Exercises

1. **Policy tracking.** Track the evolution of a policy document of your choice (a university's academic integrity policy, a company's privacy policy, or a government agency's FAQ page) over at least five time points using the Wayback Machine. Visualize how document length changes and identify periods of major revision.
2. **CDX analysis.** Use the CDX Server API to compare the snapshot frequency and page size trends for two competing organizations (e.g., two news outlets, two universities, or two tech companies). What does the capture frequency tell you about the organizations' web presence?
3. **Document similarity.** Compute pairwise document similarity across all versions of your tracked document from Exercise 1. Create a heatmap or matrix visualization. Which version transitions involved the largest changes?
4. **Government web archive.** Use the CDX Server API to analyze the archival history of a government website (e.g., a state legislature, a federal agency, or a city government). How frequently was the page captured? Are there gaps in coverage? How has the page size changed over time? Write a brief analysis of what the archival patterns reveal about the site's evolution.
5. **Then-and-now similarity.** For a website of your choice, retrieve the earliest available Wayback Machine snapshot and the most recent snapshot. Preprocess both with tokenization and stopword removal, compute bag-of-words similarity, and write a 200-word analysis of what changed. What kinds of content were added, removed, or restructured?

> **Public Interest Connection**
>
> The Wayback Machine embodies the value of **ownership** discussed in Chapter 3 — it is collectively governed, freely accessible, and exists to preserve the web as a public resource.

> When platforms delete content, change their policies, or redesign their sites, the Wayback Machine often provides the only independent record of what existed before.

7.7 Social History and Public Interest

Brewster Kahle founded the Internet Archive with the vision of creating a "library of everything" — a comprehensive, freely accessible archive of human knowledge. The Wayback Machine, launched in 2001, has become indispensable for journalists investigating corporate claims, lawyers establishing timelines of events, and researchers conducting longitudinal studies of online content.

The Internet Archive has faced legal challenges from publishers, platforms, and governments that would prefer certain content to remain forgotten. Its continued operation depends on the support of its users and donors — a reminder that the infrastructure of openness requires ongoing stewardship.

Journalists have been among the most creative users of the Wayback Machine. When politicians delete social media posts or companies quietly change their terms of service, archived versions provide accountability. Legal teams use archived web pages as evidence in intellectual property disputes, contract disagreements, and regulatory investigations — establishing what a website said on a specific date can be the difference between winning and losing a case. Researchers in fields from political science to public health use longitudinal snapshots to study how organizations communicate about crises, how policy language evolves, and how online communities form and fragment over time.

The Internet Archive's work also raises fundamental questions about the right to be forgotten versus the public's right to know. The European Union's GDPR includes provisions allowing individuals to request deletion of personal data, which can conflict with the archival mission. Some organizations have requested that the Wayback Machine remove their content. These tensions mirror the broader conflicts between openness, oversight, and ownership that Chapter 3 explores — the same data that enables accountability can also enable surveillance, and the infrastructure that preserves public knowledge can also preserve content that individuals wish to retract.

7.8 Common Issues to Debug

- **Wayback Machine rate limiting**: Space your requests with `time.sleep()`. The service is free and community-maintained; do not abuse it.

- **Incomplete archived pages**: Many snapshots are missing CSS, images, or JavaScript-rendered content. The text content is usually preserved, but visual rendering may be broken.
- **CDX data inconsistencies**: Some entries may have missing fields or redirect status codes instead of actual content. Filter your data carefully.
- **NLTK download errors**: Run `nltk.download('stopwords')` once in a fresh Python session to download the stopwords corpus.

7.9 Key Takeaways

The web is not permanent, and the Wayback Machine is your most important tool for longitudinal web research. The Availability API finds specific snapshots; the CDX Server enumerates the full history; and BeautifulSoup parses archived pages just like current ones. Document similarity techniques give you quantitative tools for measuring change over time. Always serialize your scraped data — you may not be able to re-scrape it later, and the page you studied today may be gone tomorrow.

7.10 Further Reading

- Internet Archive Wayback Machine APIs: https://archive.org/help/wayback_api.php
- CDX Server API: https://github.com/internetarchive/wayback/tree/master/wayback-cdx-server
- gensim documentation: https://radimrehurek.com/gensim/
- @rogers2019doing — Chapter on web historiography

8 Dynamic Web Pages with Selenium

Learning Objectives

- Explain why JavaScript-rendered content is invisible to `requests` + BeautifulSoup
- Install and configure Selenium WebDriver for Chrome, Firefox, or Safari
- Locate elements using multiple strategies (ID, name, XPath, CSS selector)
- Simulate user interactions — clicking, typing, and scrolling — programmatically
- Pass fully-rendered page source from Selenium back to BeautifulSoup for parsing
- Assess the fragility and ethical implications of browser automation

8.1 When Static Scraping Fails

In Chapter 6, you learned to retrieve web pages with `requests` and parse them with BeautifulSoup. This works well for pages whose content is fully present in the initial HTML — what we call *static* pages. But most modern websites are *dynamic*: their content is loaded, modified, or entirely rendered by JavaScript after the initial HTML arrives.

When you use `requests.get()` on a dynamic page, you get the HTML skeleton before JavaScript has run. The data you see in your browser may simply not exist in the response that `requests` receives. You can verify this by comparing `requests.get(url).text` with what you see in the browser — if significant content is missing from the `requests` version, the page is dynamic.

The solution is to use a tool that can execute JavaScript: a real web browser, controlled programmatically. Selenium is the standard tool for this.

8.2 Setting Up Selenium

Selenium requires two components: the `selenium` Python library and a browser driver that lets Python control a specific browser.

```
# Install from a terminal:
# pip install selenium webdriver-manager
```

The `webdriver_manager` library automatically downloads and configures the correct driver for your browser:

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

# For Chrome:
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)

# For Firefox:
# from selenium.webdriver.firefox.service import Service
# from webdriver_manager.firefox import GeckoDriverManager
# service = Service(GeckoDriverManager().install())
# driver = webdriver.Firefox(service=service)
```

When you create the `driver`, a browser window opens. This is your programmable browser — every command you issue through the `driver` object happens in that window.

8.3 Navigating and Finding Elements

```
# Navigate to a website
driver.get("https://xkcd.com")

# Find elements by various strategies
from selenium.webdriver.common.by import By

# By ID
comic_div = driver.find_element(By.ID, "comic")

# By CSS selector
nav_links = driver.find_elements(By.CSS_SELECTOR, "ul.comicNav li a")

# By XPath
title = driver.find_element(By.XPATH, "//div[@id='ctitle']")
print(title.text)  # The title of the current comic
```

The `find_element` method (singular) returns the first matching element. The `find_elements` method (plural) returns a list of all matches.

8.4 Extracting Data

xkcd is famous for its hidden alt-text messages. You can extract element attributes:

```
# Find the comic image
img = driver.find_element(By.XPATH, "//div[@id='comic']//img")

# Get the alt-text and title attributes
print(f"Alt text: {img.get_attribute('alt')}")
print(f"Title (hover text): {img.get_attribute('title')}")
```

8.5 Simulating Interactions

Selenium can simulate clicks, keyboard input, and scrolling — anything a human user would do:

8.5.1 Clicking

```
# Click the "Random" button to navigate to a random comic
random_button = driver.find_element(By.XPATH, "//ul[@class='comicNav']//a[contains(text(),'Ra
random_button.click()

# Extract the alt-text from the random page
import time
time.sleep(1)  # Wait for the page to load

img = driver.find_element(By.XPATH, "//div[@id='comic']//img")
print(f"Random comic title: {img.get_attribute('title')}")
```

8.5.2 Typing and Searching

```
from selenium.webdriver.common.keys import Keys

# Navigate to Google
driver.get("https://www.google.com")

# Find the search box
```

```
search_box = driver.find_element(By.NAME, "q")

# Type a query
search_box.send_keys("Colorado Buffaloes")
time.sleep(1)  # Observe autocomplete suggestions

# Press Enter to search
search_box.send_keys(Keys.ENTER)
time.sleep(2)  # Wait for results
```

8.5.3 Scrolling

Some pages load content as you scroll (infinite scroll). You can simulate this:

```
from selenium.webdriver.common.keys import Keys

body = driver.find_element(By.TAG_NAME, "body")

# Scroll down 5 times
for i in range(5):
    body.send_keys(Keys.PAGE_DOWN)
    time.sleep(1)  # Wait for new content to load
```

8.5.4 Waiting for Dynamic Content

The `time.sleep()` calls in the examples above are a crude approach to waiting for content to load. A better strategy is *explicit waits*, which pause execution until a specific condition is met:

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Navigate to a page with dynamically loaded content
driver.get("https://example-dynamic-site.com")

# Wait up to 10 seconds for a specific element to appear
element = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.CLASS_NAME, "results-container"))
)
print(f"Element found: {element.tag_name}")
```

`WebDriverWait` checks the condition repeatedly (every 0.5 seconds by default) and returns the element as soon as it appears, or raises a `TimeoutException` if the timeout expires. This is superior to `time.sleep()` in two ways: it does not wait longer than necessary (if the element appears in 0.2 seconds, it proceeds immediately), and it fails loudly if the element never appears (rather than silently returning an empty page).

The most common expected conditions you will use are:

- `presence_of_element_located` — the element exists in the DOM (even if not visible)
- `visibility_of_element_located` — the element is both present and visible on the page
- `element_to_be_clickable` — the element is visible, enabled, and can receive clicks
- `text_to_be_present_in_element` — specific text has appeared inside an element

Use these instead of `time.sleep()` whenever possible. The combination of `WebDriverWait` with explicit conditions makes your Selenium scripts both more reliable (they wait for exactly what they need) and faster (they proceed as soon as the condition is met rather than waiting a fixed duration). You will see this pattern used in the practical workflow section at the end of this chapter.

8.5.5 Scraping Infinite Scroll Pages

Many modern websites load content progressively as you scroll — social media feeds, image galleries, and product listings all use this pattern. The technique for scraping these pages involves scrolling to the bottom, waiting for new content to load, and repeating until no more content appears:

```
def scrape_infinite_scroll(driver, max_scrolls=20, scroll_pause=2):
    """Scroll an infinite-scroll page and collect all loaded content."""
    last_height = driver.execute_script("return document.body.scrollHeight")

    for i in range(max_scrolls):
        # Scroll to the bottom of the page
        driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
        time.sleep(scroll_pause)

        # Check if the page grew
        new_height = driver.execute_script("return document.body.scrollHeight")
        if new_height == last_height:
            print(f"No new content after scroll {i+1}. Stopping.")
            break
        last_height = new_height
        print(f"Scroll {i+1}: page height grew to {new_height}px")
```

```
    # Return the fully-loaded page source
    return driver.page_source
```

The key insight is checking `document.body.scrollHeight` before and after each scroll. When the height stops growing, you have reached the end of the available content. Some sites use a "Load More" button instead of infinite scroll — for those, replace the scroll with a click on the button and watch for the button to disappear or become disabled.

A practical consideration: infinite scroll pages can contain thousands of items, and loading them all consumes significant memory in the browser process. Set a reasonable `max_scrolls` limit based on how much data you actually need. If you are collecting data for a class project, you rarely need more than a few hundred items — scrolling through an entire social media feed of 10,000 posts is both unnecessary and discourteous to the server. Remember the proportionality principle from Chapter 2: collect only the data you need for your research question.

8.6 Passing to BeautifulSoup

Once the page is fully rendered in the browser, you can extract the complete HTML and parse it with the familiar BeautifulSoup tools:

```
from bs4 import BeautifulSoup

# Get the fully-rendered page source from Selenium
html = driver.page_source
soup = BeautifulSoup(html, "html.parser")

# Now use BeautifulSoup as usual
links = soup.find_all("a", href=True)
print(f"Total links on page: {len(links)}")
```

This hybrid approach — Selenium for rendering, BeautifulSoup for parsing — gives you the best of both worlds. Selenium excels at navigating, clicking, scrolling, and waiting for JavaScript to execute. BeautifulSoup excels at searching and extracting data from HTML. By combining them, you avoid the awkwardness of using Selenium's relatively limited element-finding API for complex extraction tasks while still getting access to the fully-rendered page content.

The workflow is always the same: use Selenium to get the page into the state you need (scrolled, clicked, searched, logged in), then hand off the rendered HTML to BeautifulSoup for extraction. Think of Selenium as the hands that navigate the browser and BeautifulSoup as the eyes that read the content. This division of labor keeps your code cleaner and more maintainable than trying to do everything with Selenium alone.

8.7 Always Close the Driver

When you are done, close the browser window:

```
driver.quit()
```

After calling `quit()`, any subsequent calls to the driver will raise an error. Use `try/finally` blocks to ensure cleanup:

```
try:
    driver.get("https://example.com")
    # ... scraping logic ...
finally:
    driver.quit()
```

8.8 The Fragility Problem

> **Warning**
>
> Screen-scraping with Selenium is inherently fragile. The Twitter scraping examples that worked in the 2019 version of this course broke completely by 2024 because Twitter (now X) redesigned its HTML to use dynamically-generated class names and require authentication for basic browsing. This is not unusual — platforms regularly change their front-end code, sometimes specifically to resist automated access.

This fragility reinforces a principle from Chapter 3: when an API is available, use it. Selenium is a powerful last resort for cases where you ethically need data that no API provides, but it is slow, resource-intensive, and brittle compared to API access. The decision tree should be: API first → static scraping second → Selenium third.

8.8.1 Headless Mode

For production scraping, you do not need a visible browser window. Headless mode runs the browser without a graphical interface, which is faster and can run on servers without a display:

```python
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless")
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")

driver = webdriver.Chrome(options=options)
driver.get("https://example.com")
print(f"Title: {driver.title}")
driver.quit()
```

The `--no-sandbox` and `--disable-dev-shm-usage` flags resolve common issues when running Chrome in containerized or server environments (like GitHub Actions, which you will use in Chapter 14). Headless mode is essential for automated data collection pipelines where no one is watching the browser window.

One important caveat: some websites detect headless browsers and serve different content or block access entirely. They do this by checking for browser properties that headless mode handles differently (like the `navigator.webdriver` flag). If you get different results in headless mode than in a visible browser, this is likely the cause. For educational purposes, switching back to headed mode usually resolves the issue. In production settings, there are legitimate workarounds, but circumventing bot detection raises the ethical questions discussed in Chapter 2 — the fact that you *can* evade detection does not mean you *should*.

8.9 Exercises

1. **Dynamic content extraction.** Find a website that loads content dynamically (via JavaScript, infinite scroll, or AJAX). Use Selenium to load the full page, then pass the source to BeautifulSoup and extract the dynamically-loaded data. Compare what you get with Selenium to what `requests.get()` returns.
2. **Multi-step interaction.** Use Selenium to automate a multi-step process: navigate to a site with a search form, enter a query, submit it, and extract results from the response page.
3. **Static vs. dynamic comparison.** For three websites of your choice, compare the HTML returned by `requests.get()` with the `driver.page_source` from Selenium. Categorize each site as static, partially dynamic, or fully dynamic.
4. **Load More button.** Find a website that uses a "Load More" button (rather than infinite scroll) to reveal additional content. Write a Selenium script that clicks the button

in a loop until it disappears or becomes disabled, then extracts all the loaded data. How many items were hidden behind the button?

5. **Performance comparison.** For the same URL, measure the time taken by three approaches: `requests.get()`, headed Selenium (with a visible browser), and headless Selenium. Use `time.time()` to measure each. Create a table comparing the three approaches on speed, completeness of data retrieved, and resource usage. When is the speed tradeoff of Selenium worth it?

> **Missing Manual Reference**
>
> For background on debugging strategies when Selenium scripts fail, see *Missing Manual* Chapter 5: Debugging and Chapter 6: Reading Python Tracebacks.

8.10 Social History and Public Interest

The shift from server-rendered to client-rendered web pages represents a fundamental change in the web's architecture. JavaScript frameworks like React, Angular, and Vue have made the web more interactive but also more opaque to automated observation. The ability to "view source" — once a straightforward way to see exactly what a page contained — now often reveals only a JavaScript bootstrap that fetches and renders content dynamically.

This architectural shift interacts with the enclosure dynamics described in Chapter 3. When platforms render content client-side, they gain another layer of control over automated access. Anti-scraping measures — CAPTCHAs, bot detection, dynamically-generated class names — become easier to implement. Researchers who once could rely on simple HTTP requests increasingly need browser automation to access the same data, raising both the technical barrier and the ethical stakes. The tools in this chapter are a response to that architectural shift — they exist because the simpler approaches taught in Chapter 6 are no longer sufficient for much of the modern web.

> **Public Interest Connection**
>
> When platforms close their APIs and render content exclusively through JavaScript, browser automation becomes the last-resort path to data that serves the public interest. The tools in this chapter exist because of the **enclosure** dynamic from Chapter 3 — the progressive restriction of access to data that was once openly available. Researchers studying algorithmic bias, misinformation, and platform governance increasingly depend on Selenium-based methods precisely because the APIs that once served these research needs have been shut down or restricted beyond practical use.

8.11 Common Issues to Debug

- **`NoSuchElementException`**: The element does not exist yet because the page has not finished loading. Add `time.sleep()` or use explicit waits with `WebDriverWait`.
- **`StaleElementReferenceException`**: The DOM changed between when you found the element and when you tried to interact with it. Re-find the element.
- **Driver/browser version mismatch**: `webdriver_manager` usually handles this, but new browser releases occasionally break compatibility temporarily.
- **Memory issues**: Each Selenium driver runs a full browser process. Close drivers promptly with `driver.quit()`.

8.12 Key Takeaways

The modern web is dynamic: JavaScript renders content after the initial HTML loads, making it invisible to static scraping tools. Selenium controls a real browser programmatically, letting you access fully-rendered pages, simulate user interactions, and extract content that `requests` cannot see. But Selenium is slow, fragile, and resource-intensive — a last resort when APIs and static scraping are unavailable. The hybrid approach of Selenium for rendering plus BeautifulSoup for parsing gives you full access to the modern web while keeping your parsing code familiar.

8.13 Further Reading

- Selenium documentation: https://www.selenium.dev/documentation/
- Selenium Python bindings: https://selenium-python.readthedocs.io/
- webdriver_manager: https://github.com/SergeyPirogov/webdriver_manager
- @mitchell2018web — Chapter 11: Scraping JavaScript

8.14 A Practical Selenium Workflow

Here is a complete workflow that demonstrates the typical pattern for scraping a dynamically-rendered page. We will automate a multi-page interaction: navigating to a website, performing a search, waiting for results to load, extracting the data, and repeating across multiple pages.

```
import time
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
```

```python
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from bs4 import BeautifulSoup
import pandas as pd

def scrape_with_selenium(search_term, max_pages=3):
    """Scrape search results from a dynamic website.

    This demonstrates the full Selenium workflow:
    1. Launch a headless browser
    2. Navigate and interact with the page
    3. Wait for dynamic content to load
    4. Extract data via BeautifulSoup
    5. Handle pagination
    6. Clean up
    """
    options = Options()
    options.add_argument("--headless")
    driver = webdriver.Chrome(options=options)

    all_results = []

    try:
        driver.get("https://example-search-site.com")

        # Find and fill the search box
        search_box = driver.find_element(By.NAME, "q")
        search_box.send_keys(search_term)
        search_box.submit()

        for page in range(max_pages):
            # Wait for results to load (up to 10 seconds)
            WebDriverWait(driver, 10).until(
                EC.presence_of_element_located((By.CLASS_NAME, "result-item"))
            )

            # Pass rendered HTML to BeautifulSoup
            soup = BeautifulSoup(driver.page_source, "html.parser")

            # Extract data from this page
            for item in soup.find_all("div", class_="result-item"):
                all_results.append({
```

```
                "title": item.find("h3").text.strip(),
                "description": item.find("p").text.strip(),
                "page": page + 1
            })

        # Try to click "Next" for pagination
        try:
            next_button = driver.find_element(By.LINK_TEXT, "Next")
            next_button.click()
            time.sleep(2)  # Wait for next page to load
        except Exception:
            break  # No more pages

    finally:
        driver.quit()  # Always clean up

    return pd.DataFrame(all_results)
```

Notice the use of `WebDriverWait` with `expected_conditions` — this is superior to `time.sleep()` because it waits only as long as needed for the element to appear, rather than waiting a fixed amount of time that might be too short (element not loaded) or too long (wasted time). The `try/finally` block ensures the browser is closed even if an error occurs.

8.14.1 When to Use Selenium vs. APIs

The decision tree for choosing your data access method:

1. **Is there an API?** Use it. APIs are faster, more reliable, and more structured (see Chapter 10 through Chapter 13).
2. **Is the content in the static HTML?** Use `requests` + BeautifulSoup (see Chapter 6). It is simpler and faster than Selenium.
3. **Is the content loaded by JavaScript?** Use Selenium. But be aware that it is slower, more fragile, and more resource-intensive.
4. **Is the content behind a login?** This is where ethical judgment matters most. Automating access to login-gated content may violate Terms of Service. See Chapter 2.

9 Extracting Data from PDFs

Learning Objectives

- Explain why PDF text extraction is difficult and how PDFs differ from structured formats
- Use PyPDF to open PDF files, access metadata, and extract page-level text
- Apply string methods and regular expressions to clean and structure extracted text
- Build a data pipeline that processes multiple PDF files to extract structured information
- Visualize trends across a corpus of PDF documents

9.1 The Web's Filing Cabinets

PDFs are everywhere in institutional record-keeping: city council meeting minutes, legislative bill texts, financial reports, court filings, academic papers, and environmental impact assessments. They are the format of choice for "official" documents precisely because they preserve visual layout across different devices and printers. But that design priority — visual fidelity over semantic structure — makes them hostile to programmatic data extraction.

When you read a PDF, you see paragraphs, tables, headers, and page numbers. When a computer reads a PDF, it sees a sequence of drawing instructions: place this character at coordinates (x, y), draw a line from here to there, embed this image at this location. Text extraction from PDFs is *reconstruction*, not reading — the computer must infer the logical structure from the spatial layout.

This chapter teaches you to work within these constraints. The results will not be perfect, but they can be good enough to turn inaccessible institutional records into analyzable datasets.

Public Interest Connection

When governments publish meeting minutes, budgets, and regulatory filings only as PDFs, they create what Chapter 3 describes as "soft enclosure" — records that are nominally public but practically inaccessible for analysis. Building the skills to extract data from PDFs is itself a form of the **oversight** value: it enables independent analysis of

institutional records.

9.2 PyPDF Basics

The `pypdf` library provides page-level text extraction and metadata access:

```
# Install if needed:
# pip install pypdf

from pypdf import PdfReader

# Open a PDF file (download Boulder City Council minutes first)
reader = PdfReader("council_minutes_2024_01.pdf")

# Document-level metadata
print(f"Number of pages: {len(reader.pages)}")
print(f"Metadata: {reader.metadata}")
```

9.2.1 Extracting Text

```
# Extract text from the first page
page = reader.pages[0]
text = page.extract_text()
print(text[:500])  # Print the first 500 characters
```

The output will likely be messy: line breaks may not correspond to paragraph boundaries, headers and footers are mixed in with content, and table data may be garbled. This is normal. Cleaning up extracted text is a core part of the workflow.

9.2.2 Cleanup Strategies

```
import re

def clean_page_text(raw_text):
    """Clean text extracted from a PDF page.
```

```
    Parameters
    ----------
    raw_text : str
        Raw text from PdfReader.pages[n].extract_text()

    Returns
    -------
    str
        Cleaned text with normalized whitespace
    """
    # Normalize whitespace
    text = re.sub(r'\s+', ' ', raw_text)

    # Remove page numbers (common patterns)
    text = re.sub(r'Page \d+ of \d+', '', text)

    # Remove common headers/footers (customize per document)
    text = re.sub(r'CITY COUNCIL MEETING MINUTES', '', text)

    return text.strip()

cleaned = clean_page_text(text)
print(cleaned[:300])
```

9.2.3 Regular Expressions for PDF Text

Regular expressions are your primary tool for imposing structure on the messy text that comes out of PDF extraction. Where structured data formats give you fields and columns, PDF text gives you a wall of characters — and regex is how you carve that wall into usable pieces. The patterns below cover the most common extraction tasks you will encounter when working with government and institutional PDFs.

Each pattern targets a specific kind of structured information that appears in unstructured text. Learning to recognize these patterns and adapt them to your specific documents is a skill that improves with practice. No single regex will work across all PDFs, but these five cover a wide range of common scenarios.

```
import re

# 1. Dates in slash-separated format (e.g., "1/15/2024", "12/03/23")
date_pattern = r'\d{1,2}/\d{1,2}/\d{2,4}'
dates = re.findall(date_pattern, cleaned)
```

```
# Returns: ['1/15/2024', '2/6/2024', ...]

# 2. Dollar amounts (e.g., "$1,250.00", "$50", "$3,400,000")
dollar_pattern = r'\$[\d,]+\.?\d*'
amounts = re.findall(dollar_pattern, cleaned)
# Returns: ['$1,250.00', '$50', '$3,400,000']

# 3. Repeating headers that PDF extraction duplicates on every page
header_pattern = r'CITY COUNCIL.*?MINUTES'
cleaned_text = re.sub(header_pattern, '', cleaned)

# 4. Section headings in ALL CAPS (at least 6 characters, start of line)
heading_pattern = r'^[A-Z][A-Z\s]{5,}$'
headings = re.findall(heading_pattern, cleaned, re.MULTILINE)
# Returns: ['PUBLIC COMMENT', 'CONSENT AGENDA', 'ADJOURNMENT', ...]

# 5. Email addresses embedded in document text
email_pattern = r'[\w.+-]+@[\w-]+\.[\w.-]+'
emails = re.findall(email_pattern, cleaned)
# Returns: ['clerk@bouldercolorado.gov', ...]
```

Notice that each of these patterns makes assumptions about the document format. The date pattern assumes slash-separated dates; if your documents use "January 15, 2024" instead, you need a different pattern. The dollar amount pattern assumes a leading $ sign; budget documents that use parentheses for negative values need additional handling. The section heading pattern assumes uppercase formatting; some documents use title case or bold formatting that does not survive text extraction. Always test your patterns against multiple pages and multiple documents from the same source before committing to a pipeline.

> **Missing Manual Reference**
>
> For a thorough introduction to regular expressions — the pattern-matching tool you will use extensively for text cleanup — see *Missing Manual* Chapter 18: Regular Expressions.

9.3 Case Study: Boulder City Council Minutes

Boulder's City Council publishes signed minutes of its public meetings online. These PDFs contain structured information — council member attendance, public comment speakers, agenda items, voting records — that can be extracted and analyzed.

9.3.1 Measuring Council Attendance

```
import os
from pypdf import PdfReader

# Assuming you have downloaded several PDF files into a directory
pdf_dir = "council_minutes/"
pdf_files = sorted([f for f in os.listdir(pdf_dir) if f.endswith(".pdf")])

def extract_attendance(pdf_path):
    """Extract council member attendance from a meeting minutes PDF.

    Parameters
    ----------
    pdf_path : str
        Path to the PDF file

    Returns
    -------
    dict
        Dictionary with 'date', 'present', 'absent' counts
    """
    reader = PdfReader(pdf_path)

    # Attendance is typically on the first page
    first_page_text = reader.pages[0].extract_text()

    # Count members listed as present vs. absent
    # This regex pattern will need to be adapted to the specific format
    present = len(re.findall(r'(?:Present|PRESENT):.*?(?=Absent|ABSENT|$)',
                             first_page_text, re.DOTALL))

    return {
        "file": os.path.basename(pdf_path),
        "pages": len(reader.pages),
        "first_page_length": len(first_page_text)
    }
```

9.3.2 Building the Pipeline

```
import time

results = []
for pdf_file in pdf_files:
    try:
        pdf_path = os.path.join(pdf_dir, pdf_file)
        result = extract_attendance(pdf_path)
        results.append(result)
        print(f"Processed: {pdf_file}")
    except Exception as e:
        print(f"Error processing {pdf_file}: {e}")

import pandas as pd

df = pd.DataFrame(results)
print(df.head())
```

9.3.3 Visualizing Trends

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5))
plt.bar(range(len(df)), df["first_page_length"])
plt.xlabel("Meeting")
plt.ylabel("First Page Text Length")
plt.title("Boulder City Council Meeting Minutes: First Page Length Over Time")
plt.tight_layout()
plt.show()
```

9.3.4 Measuring Public Comment Frequency

One of the most civically valuable pieces of information buried in meeting minutes is how many members of the public showed up to speak. Public comment sections are where residents voice concerns about zoning changes, police oversight, budget priorities, and development projects. Extracting this data requires detective work: you need to identify the textual markers that signal a public comment section and then count the speakers within it.

The challenge is that different municipalities format public comment sections differently, and even within the same city, the format may shift over time as clerks change or recording practices evolve. You are looking for patterns — section headers like "PUBLIC COMMENT" or "PUBLIC HEARING," followed by names or numbered items indicating individual speakers. This is the kind of work where you iterate: try a pattern on one document, inspect the results manually, refine the pattern, test it on a second document, and adjust again. Start by printing the raw text of several public comment sections to understand the format before writing any extraction code.

```
def count_public_comments(text):
    """Count the number of public comment speakers in meeting text.

    Parameters
    ----------
    text : str
        Full text extracted from a meeting minutes PDF

    Returns
    -------
    int
        Estimated number of public comment speakers
    """
    # Find the public comment section
    comment_section = re.search(
        r'(?:PUBLIC COMMENT|PUBLIC HEARING)(.*?)(?:CONSENT AGENDA|OLD BUSINESS|NEW BUSINESS)
        text, re.DOTALL | re.IGNORECASE
    )

    if not comment_section:
        return 0

    section_text = comment_section.group(1)

    # Count speakers by looking for common patterns
    # Pattern 1: "spoke regarding" or "spoke about" or "spoke in favor"
    spoke_count = len(re.findall(r'spoke\s+(?:regarding|about|in|on|to)',
                                section_text, re.IGNORECASE))

    # Pattern 2: numbered items like "1." "2." "3."
    numbered_count = len(re.findall(r'^\s*\d+\.', section_text, re.MULTILINE))

    # Return the higher count (different formats capture different patterns)
    return max(spoke_count, numbered_count)
```

```
# Apply across all meeting files
comment_counts = []
for pdf_file in pdf_files:
    pdf_path = os.path.join(pdf_dir, pdf_file)
    try:
        reader = PdfReader(pdf_path)
        full_text = " ".join([p.extract_text() for p in reader.pages])
        count = count_public_comments(full_text)
        comment_counts.append({"file": pdf_file, "speakers": count})
    except Exception as e:
        comment_counts.append({"file": pdf_file, "speakers": 0})

df_comments = pd.DataFrame(comment_counts)

plt.figure(figsize=(12, 5))
plt.bar(range(len(df_comments)), df_comments["speakers"])
plt.xlabel("Meeting")
plt.ylabel("Public Comment Speakers")
plt.title("Public Comment Participation in Boulder City Council Meetings")
plt.tight_layout()
plt.show()
```

What does this data reveal? Spikes in public comment frequency often correspond to controversial agenda items — a proposed development project, a change in policing policy, or a budget reallocation. Periods of low participation may indicate either public satisfaction or public disengagement. Neither interpretation is self-evident from the data alone, which is why combining extracted counts with knowledge of the agenda (also extractable from the same PDFs) produces richer analysis.

This kind of measurement is inherently imperfect. Your regex patterns will miss some speakers and occasionally count non-speakers. But even approximate counts, applied consistently across dozens of meetings, reveal patterns that would be invisible to someone reading a single set of minutes. The imprecision of PDF extraction is the price of scale — and scale is what transforms individual documents into a dataset.

9.4 Exercises

1. **Find and extract.** Locate a set of PDF documents from a government source — city council agendas, regulatory filings, or budget summaries. Extract a consistent data element across at least ten documents and build a DataFrame.

2. **Extraction quality comparison.** Compare the text extraction quality of PyPDF on three different types of PDFs: a text-based report, a document with complex tables, and a slide deck exported to PDF. What works well and what fails?
3. **Clean and visualize.** Write a function that takes a path to a PDF and returns a dictionary of extracted metadata and key content fields. Apply it across a corpus of at least five related documents and create a visualization showing how a measurable quantity changes over time.
4. **Government PDF pipeline.** Choose a government agency that publishes PDF documents regularly (meeting minutes, inspection reports, court records, or budget documents). Download at least five PDFs from the same source, write a pipeline that extracts a consistent data element from each, build a DataFrame, and create a visualization showing how that element varies across documents.
5. **Tool comparison.** Install `pdfplumber` alongside `pypdf`. Extract text from the same PDF using both tools. Compare the quality of text extraction — line breaks, whitespace handling, table detection. Which tool handles your specific PDF better, and why?

9.5 Social History and Public Interest

The PDF format was created by Adobe in 1993 to solve a real problem: documents needed to look the same regardless of what printer or screen displayed them. But the format's emphasis on visual fidelity came at the cost of semantic accessibility. Organizations like DocumentCloud, MuckRock, and the Sunlight Foundation have built tools to extract structured data from the flood of PDFs that government agencies produce — bridging the gap between nominally public records and practically accessible data.

DocumentCloud, built by ProPublica and IRE (Investigative Reporters and Editors), emerged as a platform where journalists could upload, analyze, annotate, and share document collections. It has been used in investigations of police misconduct records, corporate fraud filings, and government secrecy — cases where the raw documents are public but the volume makes manual analysis impractical. DocumentCloud's contribution was not just technical but organizational: it created a shared infrastructure where one journalist's document processing work could benefit others investigating the same institutions.

MuckRock took a different approach to the document access problem by automating the Freedom of Information Act (FOIA) request process itself. The platform helped journalists and citizens file thousands of public records requests, tracked agency response times, and published the results openly. The Sunlight Foundation, active from 2006 to 2020, advocated for government agencies to publish documents in machine-readable formats rather than PDFs in the first place — arguing that true transparency requires not just publication but *usability*. Together, these tools and organizations represent the infrastructure of transparency: community-built resources that bridge the gap between nominally public documents and practically accessible

data. Their work embodies the **ownership** value from Chapter 3 — the idea that public records belong to the public, and that technical barriers to access are themselves a form of enclosure that undermines democratic accountability.

The Boulder City Council case illustrates this tension at the local level. The same meeting information could be published as structured data (allowing citizens to easily track attendance, voting patterns, and public comment frequency) or as scanned PDFs (requiring significant technical effort to analyze). The format choice is itself a policy decision with consequences for transparency and accountability. When you build a pipeline that extracts structured data from PDFs, you are not just doing data science — you are doing the work that these organizations have championed for decades.

9.6 Common Issues to Debug

- **Garbled text from scanned PDFs**: PyPDF extracts text only from PDFs that have a text layer. Truly scanned documents require OCR tools like Tesseract.
- **Table data extracted as a text stream**: PyPDF is not a table extraction tool. For tabular data in PDFs, consider `pdfplumber` or `camelot`.
- **Regular expressions that work on one document but fail on another**: PDF formatting varies even within documents from the same source. Test your patterns on multiple files.
- **Encoding issues**: Some PDFs use non-standard character encodings. Use `.encode('utf-8', errors='replace')` if needed.

9.7 Key Takeaways

PDFs are designed for visual fidelity, not data extraction. PyPDF gives you page-level text as a starting point; string methods and regex let you clean and structure what you extract; and function abstractions let you process many documents consistently. Expect messiness — PDF extraction is reconstruction, not reading. The broader lesson connects to Chapter 3: when institutions publish data only in PDF form, they create a practical barrier to the analysis that nominally public records are meant to enable.

9.8 Further Reading

- PyPDF documentation: https://pypdf.readthedocs.io/
- pdfplumber (for table extraction): https://github.com/jsvine/pdfplumber
- Python `re` module: https://docs.python.org/3/library/re.html
- DocumentCloud: https://www.documentcloud.org/

9.9 Building a Reusable PDF Pipeline

When you work with PDFs at scale — processing dozens or hundreds of documents from the same source — you need a structured pipeline. Here is a pattern that works for most government document collections:

```
import os
import re
import pandas as pd
from pypdf import PdfReader

class PDFCorpusAnalyzer:
    """A reusable pipeline for extracting and analyzing PDF corpora.

    This class encapsulates the full workflow:
    1. Discover PDF files in a directory
    2. Extract text from each file
    3. Apply document-specific cleanup rules
    4. Extract structured fields
    5. Compile results into a DataFrame
    """

    def __init__(self, directory, cleanup_patterns=None):
        self.directory = directory
        self.cleanup_patterns = cleanup_patterns or []
        self.files = sorted([
            f for f in os.listdir(directory) if f.lower().endswith(".pdf")
        ])

    def extract_text(self, filepath, pages=None):
        """Extract text from specified pages of a PDF."""
        reader = PdfReader(filepath)

        if pages is None:
            pages = range(len(reader.pages))

        text = ""
        for page_num in pages:
            if page_num < len(reader.pages):
                text += reader.pages[page_num].extract_text() + "\n"

        # Apply cleanup patterns
```

```
        for pattern, replacement in self.cleanup_patterns:
            text = re.sub(pattern, replacement, text)

        return text.strip()

    def process_all(self, extract_fn):
        """Process all PDFs using a custom extraction function.

        Parameters
        ----------
        extract_fn : callable
            Function that takes (filepath, text) and returns a dict
        """
        results = []
        for filename in self.files:
            filepath = os.path.join(self.directory, filename)
            try:
                text = self.extract_text(filepath)
                result = extract_fn(filepath, text)
                result["filename"] = filename
                results.append(result)
            except Exception as e:
                print(f"Error processing {filename}: {e}")
                results.append({"filename": filename, "error": str(e)})

        return pd.DataFrame(results)

# Usage example: analyze Boulder City Council meeting lengths
def extract_meeting_info(filepath, text):
    """Extract meeting metadata from council minutes text."""
    # Count approximate words
    word_count = len(text.split())

    # Try to find the meeting date (format varies by document)
    date_match = re.search(
        r'(January|February|March|April|May|June|July|August|'
        r'September|October|November|December)\s+\d{1,2},\s+\d{4}',
        text
    )

    return {
        "word_count": word_count,
```

```
        "date": date_match.group(0) if date_match else None,
        "page_count": len(PdfReader(filepath).pages)
    }

analyzer = PDFCorpusAnalyzer(
    "council_minutes/",
    cleanup_patterns=[
        (r'\s+', ' '),                  # Normalize whitespace
        (r'Page \d+ of \d+', ''),       # Remove page numbers
    ]
)

df = analyzer.process_all(extract_meeting_info)
print(df.head())
```

9.9.1 When PyPDF Is Not Enough

PyPDF handles text-based PDFs well but struggles with two common scenarios:

Tables in PDFs: PyPDF extracts tables as streams of text, losing the column-row structure. For tabular PDF data, use `pdfplumber`, which preserves spatial relationships:

```
# pip install pdfplumber
import pdfplumber

with pdfplumber.open("report.pdf") as pdf:
    page = pdf.pages[0]
    table = page.extract_table()

    # table is a list of lists (rows of cells)
    df = pd.DataFrame(table[1:], columns=table[0])
```

Scanned PDFs: If the PDF was created by scanning a physical document (no text layer), PyPDF will extract empty or garbled text. You need OCR (Optical Character Recognition). The Tesseract OCR engine, accessible through the `pytesseract` library, can process scanned pages, though accuracy varies with scan quality.

Part III

APIs

10 Introduction to APIs: Wikipedia

Learning Objectives

- Explain what an API is and how it differs from web scraping as a data access method
- Read API documentation to identify endpoints, parameters, and response formats
- Construct parameterized API requests and handle pagination with continuation tokens
- Use multiple MediaWiki API endpoints to answer research questions
- Write raw API requests from documentation rather than relying on wrapper functions

10.1 What Is an API?

An API (Application Programming Interface) is a contract between a client and a server that specifies what data you can request, what parameters you can pass, and in what format you will receive the response. Unlike web scraping, where you reverse-engineer the HTML structure of a page designed for human readers, APIs provide structured, documented, and intentional access to data. They return clean JSON rather than cluttered HTML. They offer stable endpoints rather than fragile CSS selectors. And they come with documentation that tells you exactly what is available and how to ask for it.

The tradeoff, of course, is that APIs only expose what their operators choose to expose. You cannot query a field that the API does not provide. You cannot exceed the rate limits the server enforces. And as Chapter 3 describes, APIs can be restricted, repriced, or shut down entirely at the operator's discretion. But when they are available, APIs are almost always preferable to scraping.

This chapter marks the transition from document scraping (Part II) to API-based data access (Part III). The skills you learn here — reading documentation, constructing parameterized URLs, handling pagination, parsing nested JSON responses — transfer directly to every API you will work with in subsequent chapters.

10.2 Why Wikipedia?

Wikipedia's MediaWiki API is pedagogically ideal for learning API access. It is free, requires no authentication beyond a User-Agent header, has comprehensive documentation, and provides data rich enough to support diverse research designs — from content analysis to network mapping to event detection. It is also an example of what Chapter 3 describes as **openness**: a mission-driven organization that maintains free, well-documented data access by design.

We will use the English Wikipedia article for Elizabeth II as our primary case study. The article has a rich editing history with dramatic spikes around major events, making it ideal for demonstrating how to extract and analyze temporal patterns from API data.

10.3 The Anatomy of an API Call

Every MediaWiki API request follows the same structure: a base URL, an `action` parameter specifying what you want to do, additional parameters narrowing your request, and a `format` parameter specifying the response format. Let us trace through a simple example:

```
import requests

url = "https://en.wikipedia.org/w/api.php"

params = {
    "action": "query",         # We want to query for information
    "titles": "Elizabeth_II",  # About this article
    "prop": "info",            # Specifically, basic info about the page
    "format": "json"           # Return the response as JSON
}

headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}

response = requests.get(url, params=params, headers=headers)
data = response.json()

print(data)
```

The response is a nested JSON structure. You navigate it one level at a time:

```
# The data lives inside data > query > pages > {page_id}
pages = data["query"]["pages"]
```

```
# Page IDs are keys - get the first (and usually only) one
page_id = list(pages.keys())[0]
page_info = pages[page_id]

print(f"Title: {page_info['title']}")
print(f"Page ID: {page_info['pageid']}")
print(f"Last edited: {page_info.get('touched', 'unknown')}")
```

This nested navigation pattern — `data["query"]["pages"][page_id]` — is consistent across the MediaWiki API. Once you internalize it, every endpoint feels familiar.

> **Missing Manual Reference**
>
> For strategies on reading and navigating unfamiliar API documentation, see *Missing Manual* Chapter 4: Reading Official Documentation.

10.4 Getting Revisions

Every edit ever made to a Wikipedia article is recorded and accessible through the API. The revisions endpoint returns metadata about each edit: who made it, when, how much the article size changed, and what edit summary they left.

```
import requests
import pandas as pd

def get_revisions(title, headers=None):
    """Get all revisions for a Wikipedia article.

    Parameters
    ----------
    title : str
        The article title (e.g., 'Elizabeth_II')
    headers : dict, optional
        HTTP headers including User-Agent

    Returns
    -------
    pd.DataFrame
        DataFrame with revision metadata
    """
```

```
    url = "https://en.wikipedia.org/w/api.php"

    params = {
        "action": "query",
        "titles": title,
        "prop": "revisions",
        "rvprop": "ids|timestamp|user|size|comment",
        "rvlimit": "max",  # Get as many as possible per request
        "format": "json"
    }

    all_revisions = []

    while True:
        response = requests.get(url, params=params, headers=headers)
        data = response.json()

        # Navigate the nested response
        pages = data["query"]["pages"]
        page_id = list(pages.keys())[0]
        revisions = pages[page_id].get("revisions", [])
        all_revisions.extend(revisions)

        # Handle pagination
        if "continue" in data:
            params.update(data["continue"])
            print(f"  Fetched {len(all_revisions)} revisions so far...")
        else:
            break

    return pd.DataFrame(all_revisions)

headers = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}
df_revisions = get_revisions("Elizabeth_II", headers=headers)
print(f"Total revisions: {len(df_revisions)}")
```

10.4.1 Understanding Pagination

The MediaWiki API limits the number of results returned per request. When more results are available, the response includes a `"continue"` key with parameters you must add to your next request. This loop — request, extract, check for continuation, update parameters, repeat

— is the standard pagination pattern for most APIs. You will see it again in Chapter 11 and Chapter 12.

The critical mistake newcomers make is forgetting to check for continuation. Without the `while True` loop and `"continue"` check, you only get the first batch of results — potentially a small fraction of the total dataset.

10.4.2 Analyzing Revision History

```
# Convert timestamps to datetime
df_revisions["timestamp"] = pd.to_datetime(df_revisions["timestamp"])

# Count monthly edits
monthly_edits = df_revisions.set_index("timestamp").resample("ME").size()

import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(14, 5))
monthly_edits.plot(ax=ax)
ax.set_title("Monthly Edits to 'Elizabeth II' Wikipedia Article")
ax.set_ylabel("Number of edits")
ax.set_xlabel("")
plt.tight_layout()
plt.show()
# You'll see a massive spike around September 2022 (her death)
```

You can also analyze who edits the article:

```
# Top editors by number of revisions
top_editors = df_revisions["user"].value_counts().head(10)
print(top_editors)

# What fraction of edits come from the top 10 editors?
top10_share = top_editors.sum() / len(df_revisions)
print(f"Top 10 editors account for {top10_share:.1%} of all edits")
```

10.5 Getting Pageviews

The Wikimedia REST API (separate from the MediaWiki Action API) provides article pageview data — how many times an article was accessed across web and mobile platforms:

```python
def get_pageviews(title, start="20230101", end="20231231", headers=None):
    """Get daily pageviews for a Wikipedia article.

    Parameters
    ----------
    title : str
        Article title with underscores for spaces
    start, end : str
        Date range in YYYYMMDD format
    headers : dict, optional
        HTTP headers

    Returns
    -------
    pd.DataFrame
        DataFrame with daily pageview counts
    """
    url = (
        f"https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article"
        f"/en.wikipedia/all-access/all-agents/{title}/daily/{start}/{end}"
    )

    response = requests.get(url, headers=headers)
    data = response.json()

    df = pd.DataFrame(data["items"])
    df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y%m%d00")
    return df

views = get_pageviews("Elizabeth_II", start="20220101", end="20221231", headers=headers)

plt.figure(figsize=(14, 5))
plt.plot(views["timestamp"], views["views"])
plt.title("Daily Pageviews: Elizabeth II (2022)")
plt.ylabel("Views")
plt.axvline(pd.Timestamp("2022-09-08"), color="red", linestyle="--", label="Death (Sep 8)")
plt.legend()
plt.tight_layout()
plt.show()
```

10.6 Getting Links and Categories

The API exposes the internal structure of Wikipedia — the web of connections between articles:

```
def get_outlinks(title, headers=None):
    """Get all internal wikilinks from an article."""
    url = "https://en.wikipedia.org/w/api.php"
    params = {
        "action": "query",
        "titles": title,
        "prop": "links",
        "pllimit": "max",
        "format": "json"
    }

    all_links = []
    while True:
        response = requests.get(url, params=params, headers=headers)
        data = response.json()

        pages = data["query"]["pages"]
        page_id = list(pages.keys())[0]
        links = pages[page_id].get("links", [])
        all_links.extend([link["title"] for link in links])

        if "continue" in data:
            params.update(data["continue"])
        else:
            break

    return all_links

outlinks = get_outlinks("Elizabeth_II", headers=headers)
print(f"Outgoing wikilinks: {len(outlinks)}")
print(f"First 10: {outlinks[:10]}")
```

You can do the same for external links (URLs that point outside Wikipedia), interlanguage links (corresponding articles in other language editions), and category memberships:

```python
def get_categories(title, headers=None):
    """Get all categories an article belongs to."""
    url = "https://en.wikipedia.org/w/api.php"
    params = {
        "action": "query",
        "titles": title,
        "prop": "categories",
        "cllimit": "max",
        "format": "json"
    }

    all_categories = []
    while True:
        response = requests.get(url, params=params, headers=headers)
        data = response.json()

        pages = data["query"]["pages"]
        page_id = list(pages.keys())[0]
        cats = pages[page_id].get("categories", [])
        all_categories.extend([c["title"] for c in cats])

        if "continue" in data:
            params.update(data["continue"])
        else:
            break

    return all_categories

categories = get_categories("Elizabeth_II", headers=headers)
print(f"Categories: {len(categories)}")
for cat in categories[:10]:
    print(f"  {cat}")
```

10.6.1 Interlanguage Links

One of the most revealing structural features of Wikipedia is which articles exist in which languages. The `langlinks` property tells you how many of Wikipedia's 300+ language editions have a corresponding article for a given topic — and which languages those are.

```python
def get_langlinks(title, headers=None):
    """Get all interlanguage links for an article.
```

```
    Parameters
    ----------
    title : str
        Article title
    headers : dict, optional
        HTTP headers including User-Agent

    Returns
    -------
    pd.DataFrame
        DataFrame with language codes and article titles
    """
    url = "https://en.wikipedia.org/w/api.php"
    params = {
        "action": "query",
        "titles": title,
        "prop": "langlinks",
        "lllimit": "max",
        "format": "json"
    }

    all_links = []
    while True:
        response = requests.get(url, params=params, headers=headers)
        data = response.json()

        pages = data["query"]["pages"]
        page_id = list(pages.keys())[0]
        links = pages[page_id].get("langlinks", [])
        all_links.extend(links)

        if "continue" in data:
            params.update(data["continue"])
        else:
            break

    return pd.DataFrame(all_links)

langs = get_langlinks("Elizabeth_II", headers=headers)
print(f"Article exists in {len(langs)} language editions")
print(langs["lang"].head(20))
```

The number of language editions an article appears in is itself a measure of global significance — or at least of global *interest* as perceived by Wikipedia's volunteer editors. Articles about Western political leaders, major cities, and scientific concepts tend to appear in 100 or more language editions. Articles about topics primarily relevant to the Global South — local politicians, regional cultural practices, indigenous languages — may exist in only a handful. This asymmetry reflects the demographic composition of Wikipedia's editor community, which skews heavily toward the Global North and toward speakers of European languages. When you use Wikipedia data for cross-cultural analysis, interlanguage link counts are a useful but imperfect proxy for a topic's global visibility.

10.7 Getting User Contributions

You can also retrieve the editing history of individual Wikipedia users, which is useful for studying editor behavior and community dynamics:

```
def get_user_contributions(username, limit=50, headers=None):
    """Get recent contributions by a Wikipedia user."""
    url = "https://en.wikipedia.org/w/api.php"
    params = {
        "action": "query",
        "list": "usercontribs",
        "ucuser": username,
        "uclimit": limit,
        "ucprop": "title|timestamp|sizediff|comment",
        "format": "json"
    }

    response = requests.get(url, params=params, headers=headers)
    data = response.json()
    return pd.DataFrame(data["query"]["usercontribs"])

# Get contributions from a prolific editor
contribs = get_user_contributions("Drmies", limit=100, headers=headers)
print(f"Recent edits: {len(contribs)}")
print(contribs[["title", "timestamp", "sizediff"]].head())
```

10.8 Writing Raw API Requests from Documentation

The functions above packaged API requests for you. But the transferable skill is writing requests from scratch using only the documentation. Here is the process:

1. Go to the MediaWiki API documentation
2. Find an endpoint you have not used (e.g., `action=query&list=search`)
3. Read its parameters: what is required, what is optional, what values are accepted
4. Construct a request, make it, and inspect the response structure

```
# Example: search Wikipedia for articles about "web scraping"
url = "https://en.wikipedia.org/w/api.php"
params = {
    "action": "query",
    "list": "search",
    "srsearch": "web scraping",
    "srlimit": 10,
    "srprop": "size|wordcount|timestamp|snippet",
    "format": "json"
}

response = requests.get(url, params=params, headers=headers)
data = response.json()

results = data["query"]["search"]
for r in results:
    print(f"{r['title']} - {r['wordcount']} words, last edited {r['timestamp'][:10]}")
```

The ability to go from documentation to working code is the skill that transfers to every API in Chapter 11, Chapter 12, and Chapter 13. The specifics change — different base URLs, different authentication methods, different response structures — but the pattern remains: read the docs, construct the request, parse the response.

10.9 A Second Case Study: World War II

To reinforce these patterns, try applying the same functions to a different article. The World War II article is one of the most-edited on Wikipedia:

```
ww2_revisions = get_revisions("World_War_II", headers=headers)
print(f"World War II revisions: {len(ww2_revisions)}")

ww2_views = get_pageviews("World_War_II", start="20230101", end="20231231", headers=headers)

ww2_links = get_outlinks("World_War_II", headers=headers)
print(f"World War II outlinks: {len(ww2_links)}")
```

```
# Compare with Elizabeth II
print(f"\nElizabeth II revisions: {len(df_revisions)}")
print(f"Elizabeth II outlinks: {len(outlinks)}")
```

The comparison reveals how article structure and editing dynamics vary: a biographical article about a contemporary figure versus a historical event article that has been built over nearly two decades of collaborative editing.

10.9.1 Combining Endpoints

The real analytical power of the MediaWiki API emerges when you combine data from multiple endpoints to answer questions that no single endpoint can address. A natural question is whether editing activity and viewing activity are related: do articles get edited more during periods when they are being read more, or do edits happen independently of audience attention?

```
# Get monthly revision counts for Elizabeth II in 2022
df_revisions["timestamp"] = pd.to_datetime(df_revisions["timestamp"])
monthly_revs = (df_revisions
    .set_index("timestamp")
    .loc["2022"]
    .resample("ME")
    .size()
    .reset_index(name="revisions"))

# Get monthly pageviews for the same period
views_2022 = get_pageviews("Elizabeth_II", start="20220101", end="20221231",
                           headers=headers)
monthly_views = (views_2022
    .set_index("timestamp")
    .resample("ME")["views"]
    .sum()
    .reset_index())

# Dual-axis plot: revisions on left, pageviews on right
fig, ax1 = plt.subplots(figsize=(12, 5))
ax1.bar(monthly_revs["timestamp"], monthly_revs["revisions"],
        width=20, alpha=0.7, color="steelblue", label="Revisions")
ax1.set_ylabel("Monthly Revisions", color="steelblue")

ax2 = ax1.twinx()
ax2.plot(monthly_views["timestamp"], monthly_views["views"],
```

```
         color="darkred", linewidth=2, label="Pageviews")
ax2.set_ylabel("Monthly Pageviews", color="darkred")

plt.title("Elizabeth II: Editing Activity vs. Reading Activity (2022)")
fig.tight_layout()
plt.show()
```

The September 2022 spike — corresponding to the death of Queen Elizabeth II — should appear in both time series, but their relationship across quieter months is less obvious. In general, major events drive both pageviews and revisions simultaneously, but routine editing often happens independently of audience attention. Many of Wikipedia's most active editors work on articles regardless of whether anyone is reading them. This decoupling of production and consumption is a distinctive feature of Wikipedia's volunteer model.

10.10 Exercises

1. **Comparative revision analysis.** Retrieve the revision histories for two related Wikipedia articles (e.g., "Democratic_Party_(United_States)" and "Republican_Party_(United_States)"). Compare the number of unique editors, the distribution of edits per editor (is editing concentrated or distributed?), and the pattern of monthly editing activity. Are there periods where editing spikes simultaneously on both articles?

2. **Event detection with pageviews.** Use the pageviews API to retrieve daily data for five related articles over a full year. Identify spikes in each article and map them to real-world events. Did all articles spike together (suggesting a common cause), or were there article-specific events? Create a multi-line plot with labeled annotations for the key events.

3. **Link network.** Retrieve the outgoing wikilinks from three articles on related topics. How much do their link sets overlap? What articles appear in all three link sets? This overlap analysis is a simple form of the network analysis described in Chapter 17.

4. **Raw API practice.** Using only the MediaWiki API documentation at https://www.mediawiki.org/wiki/API:Main_page, construct a working request for an endpoint not covered in this chapter. Good candidates include `interlanguagelinks` (how many language editions does an article have?), `extlinks` (what external websites does an article link to?), or `recentchanges` (what are the most recent edits across all of Wikipedia?). Document your process: what parameters did you choose and why?

5. **Editor network.** For a single article, identify the top 20 editors by revision count. Then use `get_user_contributions()` to find what other articles these editors work on. Do they share other editing interests, or is this article their only point of overlap?

10.11 Social History and Public Interest

Wikipedia is the largest collaboratively-authored reference work in human history, maintained by tens of thousands of volunteer editors working in over 300 languages. Its commitment to open APIs reflects its broader mission: anyone can read, edit, and computationally access Wikipedia's content. The Wikimedia Foundation explicitly positions API access as part of its commitment to free knowledge [@ford2022writing].

But openness does not mean representativeness. Research has documented significant demographic skew in who edits Wikipedia — editors are disproportionately male, from the Global North, and technically fluent. This translates into content gaps: articles about women, people of color, and the Global South are systematically shorter, less detailed, and less well-referenced than articles about men and Western topics [@mesgari2015sum]. The data you retrieve from Wikipedia's API reflects these patterns. Ease of access does not guarantee completeness or neutrality — a principle that applies equally to every data source in this book.

Wikipedia's gender gap illustrates how community demographics shape content. Research has consistently found that Wikipedia's editor community is predominantly male, and this demographic skew is reflected in coverage — biographies of women are fewer, shorter, and more likely to mention marital status or family relationships than biographies of men. WikiProject Women in Red has worked systematically to address this gap, organizing edit-a-thons and maintaining lists of missing biographies. Data retrieved from the API reflects these structural patterns: any Wikipedia-based analysis inherits the biases of its volunteer editor community. Recognizing this is not a reason to avoid Wikipedia data but a reason to interpret it carefully.

Wikipedia also demonstrates what sustainable, mission-driven data access looks like. While commercial platforms have restricted and monetized their APIs (see Chapter 3), Wikipedia's API has remained free and has expanded in functionality over time. This is possible because the Wikimedia Foundation is a nonprofit funded by donations, not advertising revenue. The organizational structure shapes the data access policies.

10.12 Common Issues to Debug

- **Pagination**: The single most common mistake. Always check for `"continue"` in the response. Without it, you get only the first batch (often 500 or fewer results out of thousands).
- **Nested JSON navigation**: The MediaWiki API nests data several levels deep. Print `data.keys()`, then `data["query"].keys()`, then `data["query"]["pages"].keys()` to navigate step by step.
- **URL encoding**: Article titles with spaces use underscores in the API. For titles with special characters (parentheses, accented letters), use `urllib.parse.quote()`.

- **Rate limiting**: Wikipedia requests a meaningful User-Agent header. Without it, you may get slower responses or be blocked. The rate limit is approximately 200 requests per second for identified clients — generous, but not infinite.
- **Missing pages**: If you request an article that does not exist, the page ID will be negative (e.g., `-1`). Always check for this before trying to access revision data.

10.13 Key Takeaways

APIs provide structured, documented, intentional access to web data. Learning to work with one API well gives you a transferable skill for any API: read the documentation, construct parameterized requests, handle pagination with continuation tokens, and parse nested JSON responses one level at a time. Wikipedia's MediaWiki API demonstrates these patterns clearly and generously — it is a model of what open, mission-driven data access looks like. The key habits: always handle pagination, always use a meaningful User-Agent, always read the documentation before writing code, and always check the response structure before assuming its contents.

10.14 Further Reading

- MediaWiki API documentation: https://www.mediawiki.org/wiki/API:Main_page
- Wikimedia REST API (pageviews): https://wikimedia.org/api/rest_v1/
- @ford2022writing — Wikipedia's role in the survival of facts in the digital age
- @mesgari2015sum — systematic review of scholarly research on Wikipedia content
- @salganik2018bit — Chapter 2: Observing Behavior, on using digital trace data for research

11 Government Data APIs

Learning Objectives

- Manage API keys securely using environment variables and configuration files
- Navigate complex API documentation to identify endpoints, parameters, and variable catalogs
- Query the U.S. Census Bureau API with geographic and variable filters
- Retrieve economic time series from the FRED API with server-side transformations
- Search campaign finance records through the FEC API
- Combine data from multiple government APIs to build richer analytical datasets

11.1 Public Data by Mandate

Government APIs exist because of democratic commitments to transparency, not because of business strategy. The Census Bureau is constitutionally mandated to count the population. The Federal Reserve is required to publish economic data. The Federal Election Commission must disclose campaign finance records. As a result, these APIs embody the **openness** value described in Chapter 3: public data, freely accessible, maintained by public mandate rather than platform goodwill.

These APIs also tend to be more *stable* than commercial offerings — government agencies do not pivot their business models or get acquired by billionaires. But they are often less *polished*: the documentation can be dense, variable names can be cryptic, and error messages can be unhelpful. Learning to navigate these tradeoffs is itself a valuable skill.

11.2 API Key Management

Most government APIs require an API key — a unique string that identifies your application, tracks your usage, and enforces rate limits. Proper key management is a professional practice that applies to every authenticated API in this book and beyond.

Never hardcode API keys in your notebooks. If you share your notebook (on GitHub, in a class submission, with a collaborator), your key goes with it. Instead, use one of these approaches:

11.2.1 Environment Variables

```
import os

# Set in your terminal before launching Jupyter:
#   export CENSUS_API_KEY="your-key-here"     (macOS/Linux)
#   set CENSUS_API_KEY=your-key-here          (Windows)

census_key = os.environ.get("CENSUS_API_KEY")
if not census_key:
    raise ValueError(
        "CENSUS_API_KEY not found. "
        "Get a key at https://api.census.gov/data/key_signup.html "
        "and set it as an environment variable."
    )
```

11.2.2 Configuration Files

```
import json

# Create a file called 'api_keys.json' with your keys
# IMPORTANT: Add 'api_keys.json' to your .gitignore file!
# Contents: {"census": "your-key", "fred": "your-key", "fec": "your-key"}

with open("api_keys.json") as f:
    keys = json.load(f)

census_key = keys["census"]
fred_key = keys["fred"]
fec_key = keys["fec"]
```

Missing Manual Reference

For a comprehensive guide to API keys, environment variables, `.env` files, and secrets management, see *Missing Manual* Chapter 29: Environment Variables and Secrets.

11.3 U.S. Census Bureau API

The Census Bureau provides access to an enormous range of demographic, economic, and housing data. The American Community Survey (ACS) alone contains thousands of variables organized into thematic groups. The challenge is not access — the API is free and well-maintained — but navigation: finding the right variable among thousands.

11.3.1 Navigating the Variable Catalog

The Census API organizes variables into groups. Each group has a code (like `B25034` for "Year Structure Built") and contains multiple variables representing different breakdowns:

```
import requests
import pandas as pd

# Browse all groups at: https://api.census.gov/data/2022/acs/acs5/groups.html

# Inspect the variables in the "Year Structure Built" group
variables_url = "https://api.census.gov/data/2022/acs/acs5/groups/B25034.json"
response = requests.get(variables_url)
variables = response.json()

for var_id, var_info in sorted(variables["variables"].items()):
    if "label" in var_info and not var_id.endswith("M"):  # Skip margin-of-error
        print(f"{var_id}: {var_info['label'][:80]}")
```

The variable naming convention follows a pattern: `B25034_001E` is the total estimate, `B25034_002E` through `B25034_011E` are breakdowns by decade of construction, and variables ending in `M` are margins of error. Learning to read these codes quickly will save you significant time.

11.3.2 Finding the Right Variable

The Census API contains thousands of variables, and finding the right one is often the hardest part of the workflow. The discovery process follows a consistent pattern. Start with the groups index at `api.census.gov/data/2022/acs/acs5/groups.html`, which lists all thematic groupings. Browse or search for broad categories related to your question — housing, education, income, transportation, language spoken at home.

Once you identify a promising group, drill into its variables. For example, group `B01001` (Sex by Age) contains dozens of variables breaking down the population by gender and age bracket.

Group B25034 (Year Structure Built) breaks down housing stock by decade of construction. Group B19013 contains median household income. The skill of navigating these catalogs matters far more than memorizing specific variable codes, because the codes change across survey years and table versions.

Use keyword searches to narrow your focus. If you are interested in commuting patterns, search the groups page for "commut" or "transportation" or "travel time." If you are interested in educational attainment, search for "education" or "school" or "degree." Multiple groups may be relevant to your question — B15003 covers educational attainment for the population 25 and over, while B14001 covers school enrollment for the population 3 and over. Choosing between them requires reading the variable labels carefully and understanding which population denominator matches your research question.

A practical tip: when you find a variable that looks right, always retrieve it for a geography you know well and sanity-check the values. If you expect Boulder's median household income to be around $70,000 and the API returns 70000, you are on track. If it returns 7 or 70000000, you may have the wrong variable or the wrong unit. This kind of spot-checking catches errors early and builds confidence in your data pipeline before you scale it to dozens of geographies.

11.3.3 Querying for Specific Places

Census data is organized geographically using FIPS codes. Colorado is state 08; Boulder is place 07850; Longmont is place 45970. You can find FIPS codes at census.gov/library/reference/code-lists.

```
api_url = "https://api.census.gov/data/2022/acs/acs5"

# Housing age distribution for Boulder, CO
params = {
    "get": "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,B25034_005E",
    "for": "place:07850",
    "in": "state:08",
    "key": census_key
}

response = requests.get(api_url, params=params)
data = response.json()

# First row is column headers, remaining rows are data
df = pd.DataFrame(data[1:], columns=data[0])

# Convert numeric columns from strings
numeric_cols = [c for c in df.columns if c.startswith("B25034")]
```

```
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric)

print(df)
```

11.3.4 Comparing Multiple Locations

You can pass multiple FIPS codes to compare locations side by side:

```
# Compare Boulder and Longmont
params["for"] = "place:07850,45970"

response = requests.get(api_url, params=params)
data = response.json()
df_compare = pd.DataFrame(data[1:], columns=data[0])
df_compare[numeric_cols] = df_compare[numeric_cols].apply(pd.to_numeric)

# Reshape for visualization
import matplotlib.pyplot as plt

df_plot = df_compare.set_index("NAME")[numeric_cols].T
df_plot.index = ["Total", "2020+", "2010-2019", "2000-2009", "1990-1999"]

df_plot.plot(kind="bar", figsize=(10, 5))
plt.title("Housing Stock by Decade Built: Boulder vs. Longmont")
plt.ylabel("Number of Housing Units")
plt.xlabel("Decade Built")
plt.xticks(rotation=45)
plt.legend(title="City")
plt.tight_layout()
plt.show()
```

11.4 FRED API

The Federal Reserve Bank of St. Louis maintains FRED (Federal Reserve Economic Data), arguably the world's best free source of economic time series. It contains over 800,000 data series covering macroeconomics, banking, labor markets, prices, and more.

11.4.1 Finding Series IDs

FRED organizes data by "series" — each series is a specific time series with a unique ID. The easiest way to find series IDs is to search the FRED website interactively. For example, searching for "median household income Boulder County" will lead you to the series `MHIBS08013A052NCEN`.

```
# Retrieve median household income for Boulder County
url = "https://api.stlouisfed.org/fred/series/observations"

params = {
    "series_id": "MHIBS08013A052NCEN",
    "api_key": fred_key,
    "file_type": "json"
}

response = requests.get(url, params=params)
data = response.json()

df_income = pd.DataFrame(data["observations"])
df_income["date"] = pd.to_datetime(df_income["date"])
df_income["value"] = pd.to_numeric(df_income["value"], errors="coerce")

print(df_income.head())
```

11.4.2 Combining Multiple Series

The real power of FRED emerges when you combine multiple series to tell a richer story:

```
# Also get CPI data for comparison
params_cpi = {
    "series_id": "CPIAUCSL",  # Consumer Price Index, All Urban Consumers
    "api_key": fred_key,
    "file_type": "json",
    "observation_start": "2000-01-01"
}

response_cpi = requests.get(url, params=params_cpi)
df_cpi = pd.DataFrame(response_cpi.json()["observations"])
df_cpi["date"] = pd.to_datetime(df_cpi["date"])
df_cpi["cpi"] = pd.to_numeric(df_cpi["value"], errors="coerce")
```

```
# Merge income and CPI data
df_merged = pd.merge_asof(
    df_income.sort_values("date"),
    df_cpi[["date", "cpi"]].sort_values("date"),
    on="date",
    direction="nearest"
)
```

11.4.3 Server-Side Transformations

FRED can compute transformations before returning data, saving you work:

```
# Get CPI as year-over-year percent change
params_transform = {
    "series_id": "CPIAUCSL",
    "api_key": fred_key,
    "file_type": "json",
    "units": "pc1",  # Percent change from 1 year ago
    "observation_start": "2020-01-01"
}

response = requests.get(url, params=params_transform)
df_inflation = pd.DataFrame(response.json()["observations"])
df_inflation["date"] = pd.to_datetime(df_inflation["date"])
df_inflation["inflation_pct"] = pd.to_numeric(df_inflation["value"], errors="coerce")

plt.figure(figsize=(12, 5))
plt.plot(df_inflation["date"], df_inflation["inflation_pct"])
plt.axhline(y=2, color="red", linestyle="--", label="2% target")
plt.title("Year-over-Year Inflation Rate (CPI)")
plt.ylabel("Percent Change")
plt.legend()
plt.tight_layout()
plt.show()
```

Other available transformations include `chg` (change from previous value), `log` (natural log), `pca` (compounded annual rate of change), and frequency aggregation (converting monthly data to quarterly or annual).

11.4.4 Comparing CPI Across Locations

National economic indicators often mask significant regional variation. The Denver metro area, for instance, experienced housing cost increases that diverged sharply from the national average during the post-pandemic period. FRED provides regional CPI series that let you measure this divergence directly.

```
# Retrieve Denver metro CPI and national CPI
denver_params = {
    "series_id": "CUURS49BSA0",  # CPI, Denver-Aurora-Lakewood
    "api_key": fred_key,
    "file_type": "json",
    "units": "pc1",              # Year-over-year percent change
    "observation_start": "2019-01-01"
}

national_params = {
    "series_id": "CPIAUCSL",     # CPI, All Urban Consumers (national)
    "api_key": fred_key,
    "file_type": "json",
    "units": "pc1",
    "observation_start": "2019-01-01"
}

df_denver = pd.DataFrame(requests.get(url, params=denver_params).json()["observations"])
df_national = pd.DataFrame(requests.get(url, params=national_params).json()["observations"])

df_denver["date"] = pd.to_datetime(df_denver["date"])
df_denver["denver_cpi"] = pd.to_numeric(df_denver["value"], errors="coerce")
df_national["date"] = pd.to_datetime(df_national["date"])
df_national["national_cpi"] = pd.to_numeric(df_national["value"], errors="coerce")

# Merge and plot both series
df_cpi_compare = pd.merge(df_denver[["date", "denver_cpi"]],
                          df_national[["date", "national_cpi"]],
                          on="date", how="inner")

plt.figure(figsize=(12, 5))
plt.plot(df_cpi_compare["date"], df_cpi_compare["denver_cpi"], label="Denver Metro")
plt.plot(df_cpi_compare["date"], df_cpi_compare["national_cpi"], label="National Average")
plt.axhline(y=2, color="gray", linestyle="--", alpha=0.5, label="2% target")
plt.ylabel("Year-over-Year Inflation (%)")
plt.title("Regional vs. National Inflation: Denver Metro Area")
```

```
plt.legend()
plt.tight_layout()
plt.show()
```

What drives local inflation to diverge from national trends? Housing costs are often the primary factor — markets with constrained supply and strong demand (like Denver's Front Range corridor) see price increases that outpace the national average. Energy prices, labor market tightness, and local policy decisions (such as minimum wage increases or zoning restrictions) all contribute to regional variation. Comparing local and national series is a simple but powerful way to contextualize the economic conditions that affect your community specifically.

11.5 FEC API

The Federal Election Commission provides campaign finance data through a free API. This data enables accountability journalism and research on the influence of money in politics:

```
fec_key = keys["fec"]

# Search for a candidate
url = "https://api.open.fec.gov/v1/candidates/search/"
params = {
    "q": "Biden",
    "office": "P",
    "election_year": 2024,
    "api_key": fec_key,
    "per_page": 5
}

response = requests.get(url, params=params)
data = response.json()

for candidate in data["results"]:
    print(f"{candidate['name']} ({candidate['party_full']})")
    print(f"  ID: {candidate['candidate_id']}")
    print(f"  State: {candidate['state']}")
    print()
```

11.5.1 Getting Campaign Finance Data

```
# Get committee (campaign) information for a candidate
candidate_id = "P80000722"  # Biden's presidential committee

url = f"https://api.open.fec.gov/v1/candidate/{candidate_id}/totals/"
params = {
    "api_key": fec_key,
    "election_year": 2024
}

response = requests.get(url, params=params)
data = response.json()

for result in data["results"]:
    print(f"Cycle: {result.get('cycle')}")
    print(f"  Receipts: ${result.get('receipts', 0):,.0f}")
    print(f"  Disbursements: ${result.get('disbursements', 0):,.0f}")
```

11.6 Combining Census and FRED Data

The most compelling analyses often integrate data from multiple government sources to tell a story that no single dataset can tell alone. Census data provides snapshots of demographic and housing characteristics; FRED provides economic time series that capture the conditions under which those characteristics formed. Combining them reveals the relationship between macro-level economic forces and community-level outcomes.

```
# Census: housing vintage by decade for Boulder
housing_params = {
    "get": "NAME,B25034_001E,B25034_002E,B25034_003E,B25034_004E,"
           "B25034_005E,B25034_006E,B25034_007E",
    "for": "place:07850",
    "in": "state:08",
    "key": census_key
}
response = requests.get("https://api.census.gov/data/2022/acs/acs5",
                        params=housing_params)
housing_data = response.json()

decades = ["2020+", "2010s", "2000s", "1990s", "1980s", "1970s"]
```

```
counts = [int(housing_data[1][i]) for i in range(2, 8)]

# FRED: 30-year mortgage rates (annual average)
mortgage_params = {
    "series_id": "MORTGAGE30US",
    "api_key": fred_key,
    "file_type": "json",
    "frequency": "a",
    "observation_start": "1970-01-01"
}
df_mortgage = pd.DataFrame(
    requests.get(url, params=mortgage_params).json()["observations"]
)
df_mortgage["date"] = pd.to_datetime(df_mortgage["date"])
df_mortgage["rate"] = pd.to_numeric(df_mortgage["value"], errors="coerce")

# Multi-panel figure
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.bar(decades, counts, color="steelblue")
ax1.set_title("Boulder Housing Stock by Decade Built")
ax1.set_ylabel("Housing Units")
ax1.set_xlabel("Decade")

ax2.plot(df_mortgage["date"], df_mortgage["rate"], color="darkred")
ax2.set_title("30-Year Mortgage Rate (Annual Average)")
ax2.set_ylabel("Interest Rate (%)")
plt.tight_layout()
plt.show()
```

The juxtaposition is revealing. Decades with low mortgage rates tend to correspond to more construction activity, while high-rate periods see building slow down. Boulder's housing stock tells a specific story: significant construction in the 1970s and 1990s, with constraints imposed by the city's growth management policies limiting new development regardless of interest rate conditions. This is the value of integrating multiple government data sources — economic trends provide context that makes local patterns interpretable. The Census tells you *what* the housing stock looks like today; FRED tells you *why* it looks that way, by revealing the economic conditions that shaped building decisions over decades. Neither dataset alone answers the question as well as both together.

11.7 Exercises

1. **Census exploration.** Using the Census API, retrieve a variable group of your choice (browse at `api.census.gov/data/2022/acs/acs5/groups.html`) for at least five geographic areas. Create a comparative visualization and write a brief analytical narrative explaining what the data reveals.
2. **FRED time series.** Retrieve two related economic time series for your local area or state using the FRED API. Plot them on the same or side-by-side axes. What story does the data tell? Common pairings include housing prices and mortgage rates, unemployment and job creation, or income and cost of living.
3. **Campaign finance.** Using the FEC API, retrieve contribution data for a local congressional race. How much did each major candidate raise? Can you find differences in the geographic distribution or size distribution of contributions?
4. **Multi-source integration.** Combine data from at least two different government APIs (Census + FRED, Census + FEC, or FRED + FEC) into a single analysis. For example, compare median household income (Census) with campaign contributions (FEC) across congressional districts, or compare housing construction rates (Census) with interest rates (FRED).
5. **County-level analysis.** Retrieve a Census variable of your choice for all counties in Colorado using `for=county:*&in=state:08`. Create a ranked bar chart of the top 20 counties. Write a brief analysis of the geographic patterns you observe.

> **Public Interest Connection**
>
> Government data APIs embody the value of **openness** from Chapter 3 — they exist because of democratic commitments to transparency. But government data infrastructure is not immune to erosion. Datasets are occasionally withdrawn, APIs deprecated, and political pressure on statistical agencies can affect what data is collected and how it is presented. The fragility of government data infrastructure is itself a public interest concern.

11.8 Social History and Public Interest

The Census Bureau has been counting the American population since 1790, making the decennial census one of the oldest continuous data collection programs in the world. The American Community Survey, launched in 2005, replaced the census long form with continuous measurement, providing annual estimates of demographic, social, economic, and housing characteristics. The decision to make this data freely accessible through APIs reflects a

commitment to the idea that public data should be accessible to all, not just to researchers with institutional subscriptions or specialized software.

FRED was created by the Federal Reserve Bank of St. Louis with a similar philosophy: economic data should be freely accessible to anyone who wants it. Before FRED, accessing Federal Reserve data required specialized terminals or institutional subscriptions. By making the data freely available and well-documented, FRED democratized access to economic information that was previously the province of Wall Street and government insiders.

The FEC's public disclosure requirements enable some of the most important accountability journalism in American democracy. News organizations, research groups, and citizens use FEC data to track the flow of money in politics, identify potential conflicts of interest, and hold elected officials accountable. This data exists because of the Federal Election Campaign Act (1971) and subsequent legislation mandating transparency in campaign finance.

11.9 Common Issues to Debug

- **Census variable codes**: The naming convention is opaque. Bookmark the variable catalog and use it as a reference. Variables ending in `E` are estimates; those ending in `M` are margins of error.
- **FIPS codes**: Geographic identifiers like `08` (Colorado) and `07850` (Boulder) are strings, not integers. Do not strip leading zeros.
- **FRED series IDs**: These are alphanumeric strings with no consistent pattern. Use the FRED website search to find them, then copy the ID into your code.
- **API key exposure**: The most common mistake. Add `api_keys.json` to `.gitignore` *before* your first commit, or the key will persist in your git history even after deletion.
- **Rate limiting**: Government APIs generally have generous limits (several hundred requests per minute), but stay well within them, especially during automated collection.

11.10 Key Takeaways

Government APIs provide access to some of the most valuable, reliable, and stable data sources available. The Census Bureau, Federal Reserve, and FEC each serve a specific transparency mandate, and their APIs reflect this mission. The key skills — navigating complex variable catalogs, managing API keys securely, combining data from multiple sources, and understanding the institutional context behind the data — apply to every authenticated API you will encounter. These APIs exist because of public interest mandates, and using them effectively is itself a form of civic engagement.

11.11 Further Reading

- U.S. Census Bureau API: https://www.census.gov/data/developers.html
- FRED API documentation: https://fred.stlouisfed.org/docs/api/fred/
- FEC API documentation: https://api.open.fec.gov/developers/
- @salganik2018bit — Chapter 2: Observing Behavior
- @munzert2014automated — Chapter on working with web APIs

12 Social and Media Platform APIs

Learning Objectives

- Explain OAuth authentication flows and implement authenticated API access
- Retrieve and analyze subreddit metadata, submissions, comment trees, and user histories from Reddit
- Retrieve artist, album, track, and audio feature data from Spotify
- Access decentralized social network data from Bluesky and Mastodon
- Compare API design patterns across platforms and assess research implications of different access models

12.1 Platforms, Data, and Power

Social and media platform APIs mediate access to an enormous share of public discourse. Unlike the government APIs in Chapter 11 — which exist because of transparency mandates — platform APIs exist at the discretion of private companies. This means access can change overnight, as the Reddit and Twitter cases illustrate.

This chapter teaches authenticated API access through four platforms that represent different points on the spectrum from proprietary to open: Reddit (PRAW), Spotify (spotipy), Bluesky (AT Protocol), and Mastodon (Mastodon.py). Pay attention to the structural differences — they embody the tensions between enclosure and openness discussed in Chapter 3.

12.2 Reddit via PRAW

Reddit provides a free API tier for non-commercial use (100 queries per minute). The PRAW (Python Reddit API Wrapper) library simplifies authentication and data retrieval.

12.2.1 Authentication

```python
# Install: pip install praw
import praw

reddit = praw.Reddit(
    client_id="your_client_id",
    client_secret="your_client_secret",
    user_agent="WebDataScience/1.0 by u/your_username"
)

# Verify authentication
print(f"Authenticated as: {reddit.user.me()}")
```

You will need to create a Reddit app at https://www.reddit.com/prefs/apps/ and obtain your client ID and secret. Store these in a separate file, not in your notebook.

12.2.2 Exploring Subreddits

```python
import pandas as pd
from datetime import datetime

subreddit = reddit.subreddit("news")

# Subreddit metadata
print(f"Subscribers: {subreddit.subscribers:,}")
print(f"Created: {datetime.fromtimestamp(subreddit.created_utc)}")
print(f"Description: {subreddit.public_description[:200]}")

# Subreddit rules
for rule in subreddit.rules:
    print(f"- {rule['short_name']}")
```

12.2.3 Retrieving Submissions

```python
# Get the top 25 submissions
submissions = []
for submission in subreddit.top(limit=25, time_filter="week"):
    submissions.append({
        "title": submission.title,
```

```
        "score": submission.score,
        "num_comments": submission.num_comments,
        "author": str(submission.author),
        "created_utc": datetime.fromtimestamp(submission.created_utc),
        "url": submission.url
    })

df = pd.DataFrame(submissions)

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.scatter(df["score"], df["num_comments"], alpha=0.7)
plt.xlabel("Score")
plt.ylabel("Number of Comments")
plt.title("r/news: Score vs. Comments (Top 25 Weekly)")
plt.tight_layout()
plt.show()
```

12.2.4 Comment Trees

```
# Get a submission and its comments
submission = reddit.submission(url="https://www.reddit.com/r/AskReddit/...")
submission.comments.replace_more(limit=0)  # Flatten the tree

comments = []
for comment in submission.comments.list():
    comments.append({
        "body": comment.body[:200],
        "score": comment.score,
        "depth": comment.depth,
        "author": str(comment.author)
    })

df_comments = pd.DataFrame(comments)
print(f"Total comments: {len(df_comments)}")
```

12.2.5 Analyzing Comment Engagement

With the comment tree flattened into a DataFrame, you can start asking quantitative questions about how engagement varies with comment structure and position. One natural question: do deeper comments — replies to replies to replies — receive less attention than top-level responses? The `depth` field PRAW provides makes this straightforward to investigate.

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Scatter plot: comment depth vs. score
axes[0].scatter(df_comments["depth"], df_comments["score"], alpha=0.4)
axes[0].set_xlabel("Comment Depth")
axes[0].set_ylabel("Score (Upvotes - Downvotes)")
axes[0].set_title("Comment Depth vs. Score")

# Histogram of comment lengths (word count)
df_comments["word_count"] = df_comments["body"].apply(lambda x: len(x.split()))
axes[1].hist(df_comments["word_count"], bins=30, edgecolor="black", alpha=0.7)
axes[1].set_xlabel("Word Count")
axes[1].set_ylabel("Frequency")
axes[1].set_title("Distribution of Comment Lengths")

plt.tight_layout()
plt.show()
```

```
# Summary statistics by depth
depth_summary = df_comments.groupby("depth").agg(
    mean_score=("score", "mean"),
    mean_length=("word_count", "mean"),
    count=("score", "size")
).reset_index()

print(depth_summary)
# depth  mean_score  mean_length  count
# 0        45.2        28.1       312
# 1        12.8        22.5       487
# 2         5.3        18.9       201
# ...
```

You can also compute the correlation between depth and score to quantify this relationship. A Spearman rank correlation is appropriate here because the score distribution is heavily skewed

— a handful of top-level comments accumulate hundreds or thousands of votes while most comments hover near zero.

The pattern you will typically observe is a clear decay: top-level comments (depth 0) receive the highest scores because they are the most visible, appearing directly below the submission. Each additional level of nesting reduces both visibility and engagement. This is not necessarily a reflection of comment quality — it is an artifact of Reddit's interface design, where nested comments are progressively collapsed and harder to reach.

Comment length tells a complementary story. Most Reddit comments are short — a few sentences at most — but the distribution has a long right tail of substantive, multi-paragraph responses. These longer comments often appear in subreddits with stricter moderation norms (like r/AskHistorians or r/science), where low-effort replies are actively removed. The relationship between platform governance and the data you can collect is not incidental; it is fundamental to interpreting what you find.

> **Public Interest Connection**
>
> Reddit's 2023 API pricing changes — from free to $0.24 per 1,000 calls — and the shutdown of the Pushshift archive are a live example of **enclosure** in action. The free tier still supports research use, but the death of third-party apps and Pushshift fundamentally changed what data is available and on whose terms [@baumgartner2020pushshift].

12.3 Spotify via spotipy

Spotify provides rich metadata and audio analysis data through its API:

```
# Install: pip install spotipy
import spotipy
from spotipy.oauth2 import SpotifyClientCredentials

auth = SpotifyClientCredentials(
    client_id="your_spotify_client_id",
    client_secret="your_spotify_client_secret"
)
sp = spotipy.Spotify(auth_manager=auth)
```

12.3.1 Searching and Exploring

```
# Search for an artist
results = sp.search(q="Queen", type="artist", limit=5)
for artist in results["artists"]["items"]:
    print(f"{artist['name']} - Followers: {artist['followers']['total']:,}")

# Get the top artist's ID
queen_id = results["artists"]["items"][0]["id"]

# Get top tracks
top_tracks = sp.artist_top_tracks(queen_id)
for track in top_tracks["tracks"][:5]:
    print(f"  {track['name']} ({track['album']['name']})")
```

12.3.2 Audio Features

One of the most powerful features of the Spotify API is access to audio analysis data:

```
# Get audio features for top tracks
track_ids = [t["id"] for t in top_tracks["tracks"][:10]]
features = sp.audio_features(track_ids)

df_features = pd.DataFrame(features)
print(df_features[["danceability", "energy", "tempo", "valence"]].describe())
```

12.3.3 Comparing Artists by Audio Profile

Audio features become especially revealing when you compare artists across genres. Spotify's numerical representations of musical characteristics let you quantify differences that listeners perceive intuitively. Let us retrieve audio features for two contrasting artists — one from classical music and one from electronic dance music — and see whether the API's features capture the distinction.

```
def get_artist_audio_profile(sp, artist_name):
    """Retrieve mean audio features for an artist's top tracks.

    Parameters
    ----------
    sp : spotipy.Spotify
        Authenticated Spotify client
    artist_name : str
```

```
        Name of the artist to search for

    Returns
    -------
    dict
        Mean values for key audio features
    """
    results = sp.search(q=artist_name, type="artist", limit=1)
    artist_id = results["artists"]["items"][0]["id"]

    top_tracks = sp.artist_top_tracks(artist_id)
    track_ids = [t["id"] for t in top_tracks["tracks"][:10]]
    features = sp.audio_features(track_ids)

    df = pd.DataFrame(features)
    return df[["danceability", "energy", "valence", "tempo"]].mean().to_dict()

# Compare two contrasting artists
profile_classical = get_artist_audio_profile(sp, "Yo-Yo Ma")
profile_electronic = get_artist_audio_profile(sp, "Daft Punk")

comparison = pd.DataFrame({
    "Yo-Yo Ma": profile_classical,
    "Daft Punk": profile_electronic
})
print(comparison)
```

```
import numpy as np

# Grouped bar chart comparing audio profiles
features_to_plot = ["danceability", "energy", "valence"]
x = np.arange(len(features_to_plot))
width = 0.35

fig, ax = plt.subplots(figsize=(8, 5))
bars1 = ax.bar(x - width/2, [profile_classical[f] for f in features_to_plot],
               width, label="Yo-Yo Ma", color="#4C72B0")
bars2 = ax.bar(x + width/2, [profile_electronic[f] for f in features_to_plot],
               width, label="Daft Punk", color="#DD8452")

ax.set_ylabel("Mean Feature Value (0-1)")
ax.set_title("Audio Feature Comparison: Classical vs. Electronic")
```

```
ax.set_xticks(x)
ax.set_xticklabels([f.capitalize() for f in features_to_plot])
ax.legend()
ax.set_ylim(0, 1)
plt.tight_layout()
plt.show()
```

The differences are stark. Daft Punk's tracks score high on danceability and energy — characteristics that define electronic dance music — while Yo-Yo Ma's cello performances register low on both dimensions. Valence, Spotify's measure of musical positivity, may be more surprising: classical pieces often score in the middle range, neither clearly "happy" nor "sad" by the algorithm's standards, while electronic tracks tend toward higher valence.

This kind of comparison scales naturally. You could extend it to dozens of artists, building a feature matrix that maps the audio landscape of an entire genre or era. Clustering algorithms applied to these profiles can reveal subgenre structure that mirrors musicological categories — or, more interestingly, identifies groupings that cut across traditional genre boundaries.

These audio features are not ground truth about music — they are Spotify's proprietary algorithmic interpretations. Tempo is measured in beats per minute and is relatively objective; danceability and valence are learned from training data and reflect particular cultural assumptions about what makes music "danceable" or "positive." When you use these features in research, you are analyzing Spotify's model of music as much as you are analyzing the music itself. This is a recurring theme in web data science: the data you collect from platforms is always mediated by the platform's own interpretive layer.

12.4 Bluesky via AT Protocol

Bluesky, built on the AT Protocol, represents a structural alternative to proprietary platforms. Its open protocol means that API access is an architectural feature, not a business decision:

```
# Install: pip install atproto
from atproto import Client

client = Client()
client.login("your-handle.bsky.social", "your-app-password")

# Get your own profile
profile = client.app.bsky.actor.get_profile({"actor": client.me.did})
print(f"Handle: {profile.handle}")
print(f"Followers: {profile.followers_count}")
```

```
print(f"Following: {profile.follows_count}")

# Get your timeline
timeline = client.app.bsky.feed.get_timeline({"limit": 10})
for item in timeline.feed:
    post = item.post
    print(f"@{post.author.handle}: {post.record.text[:80]}")
```

The key difference from Reddit and Spotify: Bluesky's protocol is designed so that no single company controls API access. Your identity, social graph, and data are portable across any service that implements the AT Protocol.

12.5 Mastodon via Mastodon.py

Mastodon is a decentralized social network built on the ActivityPub protocol. Each server (instance) hosts its own API [@diener2026mastodonpy]:

```
# Install: pip install Mastodon.py
from mastodon import Mastodon

# Register your app (one-time setup)
# Mastodon.create_app(
#     "WebDataScience",
#     api_base_url="https://mastodon.social",
#     to_file="app_cred.secret"
# )

mastodon = Mastodon(
    client_id="app_cred.secret",
    api_base_url="https://mastodon.social"
)

# mastodon.log_in("your-email", "your-password", to_file="user_cred.secret")
mastodon = Mastodon(access_token="user_cred.secret", api_base_url="https://mastodon.social")

# Get the public timeline
timeline = mastodon.timeline_public(limit=10)
for toot in timeline:
    print(f"@{toot['account']['username']}: {toot['content'][:80]}")
```

12.6 Exercises

1. **Reddit analysis.** Compare two subreddits on the same broad topic (e.g., r/science and r/askscience). How do submission patterns, comment engagement, and moderation structures differ?
2. **Audio features.** Using spotipy, analyze the audio features of a playlist or an artist's full discography. How do tempo, energy, and danceability vary across albums or over the artist's career?
3. **Open vs. closed.** Attempt to retrieve the same type of information (user profile, social graph, recent posts) from both Bluesky and Reddit. Compare the ease of access, the richness of the data, and the authentication requirements. What does this comparison tell you about platform governance?
4. **Genre fingerprints.** Choose two Spotify playlists representing different genres (e.g., country vs. electronic). Retrieve audio features for the tracks in each playlist using `sp.playlist_tracks()` and `sp.audio_features()`. Create overlapping distribution plots (using histograms or kernel density estimates) comparing danceability, energy, and valence across genres. Can you distinguish the genres algorithmically based on audio features alone?
5. **Bluesky analysis.** Retrieve the 50 most recent posts from a Bluesky account of your choice using the AT Protocol SDK. Compute summary statistics: average post length, posting frequency (posts per day), fraction of posts that are replies vs. original posts, and the most common words used. How does this activity profile compare to what you might expect from the same account on Twitter/X? What limitations of the AT Protocol API did you encounter during data collection?

12.7 Social History and Public Interest

The contrast between Reddit, Spotify, Bluesky, and Mastodon illustrates the governance spectrum described in Chapter 3. Reddit's 2023 changes demonstrate how a platform can unilaterally restrict access that researchers depended on. Spotify's API represents a productive commercial arrangement — rich data in exchange for engagement. Bluesky and Mastodon represent structural alternatives where open protocols make enclosure architecturally harder.

The PRAW library and Mastodon.py continue to work because their platforms maintain documented APIs. But the history of Twitter's API — which went from the most important data source in computational social science to essentially inaccessible within a year of its acquisition — is a reminder that platform goodwill is not infrastructure.

The Fediverse offers two distinct architectural responses to this problem. ActivityPub, the protocol underlying Mastodon, creates a federated network where each instance is independently

operated but can communicate with others — much like email servers. No single company controls the protocol, so no single company can revoke access. The AT Protocol, which powers Bluesky, takes a different approach centered on portable identity: your account data, social graph, and content can move between providers, preventing the lock-in that makes platform migration so costly. Both represent structural alternatives to centralized platforms, distributing control rather than concentrating it. This is the architectural embodiment of **ownership** (Chapter 3) — users and communities, not corporations, control the infrastructure. For researchers, these open protocols mean that data access is a design feature rather than a business concession, offering more durable foundations for longitudinal research than any corporate API ever could. The trade-off is fragmentation: collecting comprehensive data from a federated network requires engaging with multiple instances, each with its own norms, moderation policies, and technical configurations. Decentralization solves the single-point-of-failure problem but introduces new challenges for research design that centralized platforms never posed.

12.8 Common Issues to Debug

- **PRAW `OAuthException`**: Double-check your `client_id`, `client_secret`, and `user_agent`. The user_agent must be unique and descriptive.
- **Spotify client credentials vs. authorization code**: Client credentials flow cannot access user-specific data like listening history. It works for public search, artist, and track data.
- **Bluesky `atproto` SDK**: Still pre-1.0; API may change between versions. Check the changelog before upgrading.
- **Mastodon instance differences**: Different instances have different rate limits, rules, and may block automated access. Always register your app properly.

12.9 Key Takeaways

Platform APIs provide access to behavioral data at unprecedented scale, but that access is conditional, costly, and fragile. The pattern across all four platforms is the same: authenticate, construct requests, parse nested JSON, convert to DataFrames. The difference is in governance: who controls the data, on what terms, and what happens when they change the rules. Build skills across both proprietary and open platforms so that the closure of any single source does not shut down your research.

12.10 Further Reading

- PRAW documentation: https://praw.readthedocs.io/

- spotipy documentation: https://spotipy.readthedocs.io/
- AT Protocol SDK: https://atproto.blue/
- Mastodon.py documentation: https://mastodonpy.readthedocs.io/
- @baumgartner2020pushshift — the Pushshift Reddit dataset
- @diener2026mastodonpy — the Mastodon.py library

12.11 Comparing Platform Architectures

The four platforms in this chapter represent fundamentally different governance architectures, and these differences shape what data is available, on what terms, and how durable the access is.

Platform	Protocol	Authentication	Rate Limit (free)	Data Ownership
Reddit	Proprietary REST API	OAuth2	100 QPM	Platform-controlled
Spotify	Proprietary REST API	OAuth2 / Client Credentials	~180 RPM	Platform-controlled
Bluesky	AT Protocol (open)	App password	Generous, per-PDS	User-portable
Mastodon	Activity-Pub (open)	OAuth2, per-instance	Varies by instance	Instance-governed

The structural difference matters: Reddit and Spotify can unilaterally change their API terms because they control both the platform and the protocol. Bluesky and Mastodon cannot — or at least, no single entity can — because the protocols are open standards that anyone can implement. This is the architectural embodiment of the **ownership** value from Chapter 3.

12.11.1 Building a Bluesky Social Network

The AT Protocol makes social graph data directly accessible, enabling network analysis that would be difficult on more restricted platforms:

```
def get_social_graph(client, handle, direction="followers", limit=100):
    """Get followers or following for a Bluesky user.

    Parameters
    ----------
    client : atproto.Client
```

```
        Authenticated Bluesky client
    handle : str
        The user's handle (e.g., 'username.bsky.social')
    direction : str
        'followers' or 'following'
    limit : int
        Maximum number of accounts to retrieve

    Returns
    -------
    list
        List of dictionaries with handle and display name
    """
    results = []
    cursor = None

    while len(results) < limit:
        if direction == "followers":
            response = client.app.bsky.graph.get_followers(
                {"actor": handle, "limit": 50, "cursor": cursor}
            )
            accounts = response.followers
        else:
            response = client.app.bsky.graph.get_follows(
                {"actor": handle, "limit": 50, "cursor": cursor}
            )
            accounts = response.follows

        for account in accounts:
            results.append({
                "handle": account.handle,
                "display_name": account.display_name or "",
                "description": (account.description or "")[:100]
            })

        cursor = response.cursor
        if not cursor or not accounts:
            break

    return results[:limit]

# Get followers for a user
```

```
followers = get_social_graph(client, "your-handle.bsky.social", "followers")
following = get_social_graph(client, "your-handle.bsky.social", "following")

print(f"Followers: {len(followers)}")
print(f"Following: {len(following)}")

# Find mutual connections
follower_handles = {f["handle"] for f in followers}
following_handles = {f["handle"] for f in following}
mutuals = follower_handles & following_handles
print(f"Mutual connections: {len(mutuals)}")
```

12.11.2 Mastodon: Working Across Instances

A key difference between Mastodon and centralized platforms is that there is no single API endpoint for "all of Mastodon." Each instance maintains its own API, and you can only directly access the data on instances you authenticate with. This federated architecture means that comprehensive data collection requires accessing multiple instances — a fundamentally different research design challenge:

```
# Search for accounts across the instance you're connected to
results = mastodon.account_search("data science", limit=10)
for account in results:
    print(f"@{account['acct']} - {account['followers_count']} followers")
    print(f"  {account['note'][:100]}")
    print()

# Get a user's recent statuses (toots)
user_id = results[0]["id"]
statuses = mastodon.account_statuses(user_id, limit=20)

for status in statuses[:5]:
    # Strip HTML tags from content
    from bs4 import BeautifulSoup
    text = BeautifulSoup(status["content"], "html.parser").get_text()
    print(f"  [{status['created_at'][:10]}] {text[:100]}")

# Track a hashtag
tagged = mastodon.timeline_hashtag("datascience", limit=20)
print(f"\nRecent posts tagged #datascience: {len(tagged)}")
```

13 AI and Language Model APIs

Learning Objectives

- Set up authenticated access to LLM APIs (OpenAI, Anthropic)
- Construct chat completion requests with system, user, and assistant roles
- Use few-shot prompting and structured output to extract data from unstructured text
- Compute text embeddings for document similarity and clustering
- Assess the costs, reproducibility challenges, and appropriate uses of LLMs in research

13.1 LLMs as Web Data Science Tools

Large language model APIs represent a fundamentally new category of tool for web data science. They can transform the unstructured text outputs of Chapters 6–9 (scraped web pages, PDF extractions, archived documents) into structured, analyzable data — performing tasks like sentiment analysis, entity extraction, content coding, and summarization at scale.

This chapter treats LLMs as another API to learn: the same patterns of authentication, request construction, and response parsing apply. But LLMs have distinctive properties — non-determinism, cost, and limited reproducibility — that require careful handling.

13.2 The OpenAI API

```
# Install: pip install openai
from openai import OpenAI

# Store your key securely (see Missing Manual Ch. 29)
import os
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))
```

13.2.1 Basic Chat Completion

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is web scraping in one sentence?"}
    ]
)

# Extract the response text
answer = response.choices[0].message.content
print(answer)
```

13.2.2 Customizing Roles

The `system` message sets the assistant's behavior. The `user` message is your input. The `assistant` message lets you simulate prior conversation turns:

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a legislative analyst specializing in Colorado
        {"role": "user", "content": "Summarize the key provisions of this bill in three bulle
        {"role": "assistant", "content": "I'd be happy to analyze that bill. Could you share
        {"role": "user", "content": bill_text}  # Variable containing bill text
    ]
)
```

13.2.3 Structured Output and Few-Shot Prompting

For research applications, you typically want structured data, not free text. Use few-shot prompting — providing examples of the desired input-output format:

```
prompt = """Classify the following bill summary into one of these categories:
education, healthcare, environment, taxation, criminal_justice, other.

Examples:
Summary: "Increases funding for K-12 teacher salaries across the state."
Category: education
```

```
Summary: "Establishes new emission standards for power plants."
Category: environment

Now classify:
Summary: "{bill_summary}"
Category:"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Respond with only the category label, nothing else."}
        {"role": "user", "content": prompt.format(bill_summary=bill_summary)}
    ],
    temperature=0  # Reduce randomness for classification tasks
)

category = response.choices[0].message.content.strip()
print(f"Category: {category}")
```

13.3 Text Embeddings

Embeddings are vector representations that encode the semantic meaning of text. You can use them to measure document similarity:

```
def get_embedding(text, model="text-embedding-3-small"):
    """Get the embedding vector for a text string."""
    response = client.embeddings.create(input=text, model=model)
    return response.data[0].embedding

# Compute embeddings for bill summaries
import numpy as np
from scipy.spatial.distance import cosine

bill_summaries = ["Summary of bill 1...", "Summary of bill 2...", "Summary of bill 3..."]
embeddings = [get_embedding(s) for s in bill_summaries]

# Compute pairwise similarity
embedding_matrix = np.array(embeddings)
n = len(bill_summaries)
similarity_matrix = np.zeros((n, n))
```

```
for i in range(n):
    for j in range(n):
        similarity_matrix[i][j] = 1 - cosine(embedding_matrix[i], embedding_matrix[j])

# Visualize as a heatmap
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 5))
sns.heatmap(similarity_matrix, annot=True, fmt=".2f", cmap="YlOrRd",
            xticklabels=range(1, n+1), yticklabels=range(1, n+1))
plt.title("Bill Summary Similarity (Cosine)")
plt.tight_layout()
plt.show()
```

13.4 Applied Example: Legislative Bill Analysis

Let us apply these tools to a concrete web data science task. Suppose you have scraped bill summaries from the Colorado General Assembly website (building on the scraping techniques from Chapter 6). You want to classify each bill by policy area, extract the key stakeholders mentioned, and measure how similar bills are to each other.

13.4.1 Step 1: Classifying Bills

```
def classify_bill(summary, client):
    """Classify a bill summary into a policy category.

    Parameters
    ----------
    summary : str
        The text of the bill summary
    client : OpenAI
        An authenticated OpenAI client

    Returns
    -------
    str
        The policy category
```

```
    """
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": (
                    "You are a legislative analyst. Classify bill summaries into "
                    "exactly one category: education, healthcare, environment, "
                    "taxation, criminal_justice, transportation, housing, labor, "
                    "technology, or other. Respond with only the category label."
                )
            },
            {"role": "user", "content": summary}
        ],
        temperature=0
    )
    return response.choices[0].message.content.strip().lower()

# Apply across a list of bills
bills = [
    {"id": "HB24-1001", "summary": "Increases funding for K-12 teacher salaries..."},
    {"id": "HB24-1042", "summary": "Establishes new emission standards for power plants..."}
    {"id": "SB24-087", "summary": "Requires health insurers to cover annual mental health...
]

for bill in bills:
    bill["category"] = classify_bill(bill["summary"], client)
    print(f"{bill['id']}: {bill['category']}")

import time
time.sleep(1)  # Rate limit between API calls
```

13.4.2 Step 2: Extracting Structured Information

You can use few-shot prompting to extract multiple fields at once:

```
import json

extraction_prompt = """Extract the following fields from this bill summary as JSON:
- primary_topic: one phrase describing the main topic
```

```
- stakeholders: list of groups affected
- fiscal_impact: "yes", "no", or "unclear"
- geographic_scope: "statewide", "local", or "specific_region"

Example input: "Requires all school districts to provide free breakfast and lunch to students
Example output: {{"primary_topic": "school meals", "stakeholders": ["students", "school distr

Now extract from: "{summary}"
"""

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Respond with valid JSON only. No markdown formatting."
        {"role": "user", "content": extraction_prompt.format(summary=bills[0]["summary"])}
    ],
    temperature=0
)

try:
    extracted = json.loads(response.choices[0].message.content)
    print(json.dumps(extracted, indent=2))
except json.JSONDecodeError:
    print("Failed to parse response as JSON")
    print(response.choices[0].message.content)
```

13.4.3 Step 3: Measuring Similarity with Embeddings

Embeddings let you move beyond keyword matching to semantic similarity. Two bills about "school funding" and "educational resource allocation" would have low keyword overlap but high embedding similarity:

```
import numpy as np
from scipy.spatial.distance import cosine

def get_embeddings(texts, client, model="text-embedding-3-small"):
    """Get embedding vectors for a list of texts."""
    embeddings = []
    for text in texts:
        response = client.embeddings.create(input=text, model=model)
        embeddings.append(response.data[0].embedding)
```

```
        time.sleep(0.5)  # Rate limit
    return np.array(embeddings)

# Get embeddings for all bill summaries
summaries = [b["summary"] for b in bills]
embedding_matrix = get_embeddings(summaries, client)

# Compute pairwise cosine similarity
n = len(summaries)
similarity = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        similarity[i][j] = 1 - cosine(embedding_matrix[i], embedding_matrix[j])

# Visualize
import matplotlib.pyplot as plt
import seaborn as sns

labels = [b["id"] for b in bills]
plt.figure(figsize=(8, 6))
sns.heatmap(similarity, annot=True, fmt=".2f", cmap="YlOrRd",
            xticklabels=labels, yticklabels=labels)
plt.title("Bill Summary Similarity (Cosine Distance of Embeddings)")
plt.tight_layout()
plt.show()
```

13.5 Fine-Tuning for Domain-Specific Tasks

Sometimes a general model does not produce consistent results for your specific task. Fine-tuning lets you train the model on your own examples:

```
# Prepare training data in JSONL format
# Each line is a complete conversation demonstrating the desired behavior
training_data = [
    {
        "messages": [
            {"role": "system", "content": "Classify Colorado bill summaries by policy area."
            {"role": "user", "content": "Increases the minimum wage to $15/hour by 2026."},
            {"role": "assistant", "content": "labor"}
        ]
```

```
    },
    {
        "messages": [
            {"role": "system", "content": "Classify Colorado bill summaries by policy area."
            {"role": "user", "content": "Requires electric vehicle charging at new buildings
            {"role": "assistant", "content": "environment"}
        ]
    },
    # ... at least 10 examples, ideally 50+
]

# Save to JSONL file
import json

with open("training_data.jsonl", "w") as f:
    for example in training_data:
        f.write(json.dumps(example) + "\n")

# Upload the training file
training_file = client.files.create(
    file=open("training_data.jsonl", "rb"),
    purpose="fine-tune"
)

# Start a fine-tuning job
job = client.fine_tuning.jobs.create(
    training_file=training_file.id,
    model="gpt-4o-mini-2024-07-18"
)

print(f"Fine-tuning job started: {job.id}")
print(f"Status: {job.status}")
```

Fine-tuning takes time (minutes to hours depending on dataset size). You can check the status periodically and then use the fine-tuned model by specifying its ID in place of the base model name.

13.6 Cost Management

LLM API calls cost money. Before running a large batch, estimate the cost:

```python
# Rough cost estimation for GPT-4o-mini
# Input: ~$0.15 per 1M tokens; Output: ~$0.60 per 1M tokens
# A typical bill summary is ~200 tokens; a classification response is ~5 tokens

num_bills = 500
input_tokens = num_bills * 200
output_tokens = num_bills * 5

estimated_cost = (input_tokens * 0.15 / 1_000_000) + (output_tokens * 0.60 / 1_000_000)
print(f"Estimated cost for classifying {num_bills} bills: ${estimated_cost:.4f}")
# Typically a few cents for small jobs, but can add up with larger models
```

Use `gpt-4o-mini` for development and testing. Switch to `gpt-4o` only when you need higher quality and have validated your prompts on a smaller sample.

13.7 Reproducibility and Documentation

LLM-based research faces reproducibility challenges that are fundamentally different from those of traditional computational methods. When you write a Python script to parse HTML, anyone with the same input file will get the same output. LLMs break this guarantee in multiple ways. Even with `temperature=0`, model outputs can vary slightly between calls due to floating-point non-determinism in the inference process. More significantly, model versions change over time: a prompt that works reliably with GPT-4 in early 2024 may produce different results with the same model name in late 2024, because the underlying weights have been updated. And when a model is deprecated entirely — as GPT-3.5 eventually will be — exact replication becomes impossible. These properties do not disqualify LLMs as research tools, but they demand a level of documentation that most researchers are not yet accustomed to providing.

At a minimum, every research project that uses LLM APIs should record the following for each API call: the exact model name and version (e.g., `gpt-4o-mini-2024-07-18`, not just `gpt-4o-mini`), the complete prompt text including the system message, all generation parameters (`temperature`, `max_tokens`, `top_p`, `frequency_penalty`), the date and time of the API call, and any post-processing or validation steps applied to the output. Store this metadata alongside your results, not in a methods section written months later from memory. A simple practice is to log each API call to a JSONL file that includes both the request parameters and the full response object — this creates an audit trail that supports both reproducibility and debugging.

This documentation burden connects to the broader norms of scientific reproducibility discussed in Chapter 16. The AI disclosure framework presented there provides a structured template for reporting LLM use in research. The core principle is transparency: another researcher

should be able to read your methods section and understand exactly what model, prompt, and parameters produced your results — even if they cannot reproduce them identically, they should be able to reproduce them approximately and assess whether the differences are meaningful. Treating LLM documentation as a first-class methodological concern — not an afterthought — is what separates rigorous research from ad hoc analysis.

13.8 The Anthropic API

Anthropic's Claude provides an alternative LLM API with a similar interface:

```
# Install: pip install anthropic
from anthropic import Anthropic

client_anthropic = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

response = client_anthropic.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "What is the AT Protocol in one sentence?"}
    ]
)

print(response.content[0].text)
```

13.8.1 Comparing Model Outputs

When you use LLMs for research tasks like content classification, a critical question is whether your results depend on which model you chose. The answer, almost always, is yes — and the degree of that dependence matters for the credibility of your findings. You can test this directly by sending the same prompt to both APIs and comparing their responses.

```
# Send the same classification prompt to both models
bill_text = "Establishes new emission standards for power plants operating in Colorado."

system_prompt = (
    "You are a legislative analyst. Classify this bill summary into exactly one "
    "category: education, healthcare, environment, taxation, criminal_justice, "
    "transportation, housing, labor, technology, or other. "
    "Respond with only the category label."
```

```
)

# OpenAI classification
openai_response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": bill_text}
    ],
    temperature=0
)
openai_label = openai_response.choices[0].message.content.strip()

# Anthropic classification
anthropic_response = client_anthropic.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=50,
    system=system_prompt,
    messages=[
        {"role": "user", "content": bill_text}
    ]
)
anthropic_label = anthropic_response.content[0].text.strip()

print(f"OpenAI:    {openai_label}")     # environment
print(f"Anthropic: {anthropic_label}")   # environment
print(f"Agreement: {openai_label == anthropic_label}")
```

For straightforward cases like the example above, both models will likely agree. The disagreements emerge at the boundaries — a bill about "electric vehicle incentives" might be classified as `environment` by one model and `transportation` or `taxation` by the other. These boundary cases are not failures; they reflect genuine ambiguity in the classification scheme. But they mean that your results are partially an artifact of your model choice, not purely a reflection of the underlying data.

This makes validation against human judgment essential. If two state-of-the-art models disagree on 15 percent of your cases, you need to know whether one of them aligns better with how a domain expert would classify the same text. Running a cross-model comparison on a human-labeled sample is the minimum standard for any research that uses LLM-based classification. Treat the model as a measurement instrument, and like any instrument, it needs calibration.

Missing Manual Reference

For a deeper understanding of how language models work internally — tokens, embeddings, attention, context windows — see *Missing Manual* Chapter 30: How Language Models Work.

13.9 Exercises

1. **Structured extraction.** Use an LLM API to extract structured metadata (topic, stakeholders, geographic scope) from a set of at least 10 web-scraped documents. Design your prompt, test it, and assess the quality of the results.
2. **Embedding analysis.** Compute embeddings for a corpus of at least 20 related documents and use cosine similarity to identify clusters. Visualize the results. Do the clusters match your intuitive groupings?
3. **Human-AI agreement.** Design a classification task for a set of documents. Manually classify 20 documents, then have an LLM classify the same documents. Compute the agreement rate. Where does the LLM disagree with your judgment?
4. **Cross-model comparison.** Code 20 text samples manually according to a classification scheme of your choice. Have both GPT-4o-mini and Claude classify the same 20 samples. Compute the agreement rate between each model and your human labels, and between the two models. Where do the models disagree with each other, and what does that tell you about the task?
5. **Hierarchical clustering.** Generate embeddings for 30 Wikipedia article titles spanning three different topic areas (10 per topic). Use `scipy.cluster.hierarchy` to create a dendrogram. Do the embeddings cluster by topic? How does the clustering compare to your intuitive groupings?

13.10 Social History and Public Interest

LLMs have accelerated the enclosure dynamics described in Chapter 3 — the enormous value of user-generated content as training data has given platforms new incentives to restrict access. At the same time, LLMs provide researchers with powerful analytical tools that can partially offset restricted data access: if you can retrieve even a small sample of text, LLMs can help you code, classify, and extract information from it at scales that would be impossible manually [@ziems2024can].

The key challenge is reproducibility. LLM outputs are non-deterministic (even with `temperature=0`, there is some variation), model versions change, and the cost structure

means that replication requires financial investment. These properties require new norms for documentation and disclosure — see Chapter 16.

There is also a cost-based access inequality shaping who can use these tools for research. GPT-4-class models cost orders of magnitude more per token than GPT-3.5-class models, creating a tier system where well-funded labs can afford the most capable models while under-resourced researchers are limited to cheaper, less capable alternatives. Open-weight models like LLaMA, Mistral, and Qwen represent a partial counterbalance — their weights are freely available for download and local inference. But running these models requires significant GPU infrastructure, creating a different kind of barrier rooted in compute rather than API fees. The result is a landscape where access to the most capable AI tools is stratified by institutional resources, raising questions about whose research questions get answered and whose analytical methods are considered state-of-the-art.

> **Public Interest Connection**
>
> The cost structure of LLM APIs creates a form of **oversight** asymmetry (Chapter 3). Well-resourced organizations can afford to run large-scale analyses that smaller teams cannot replicate or verify. Open-weight models and transparent pricing help, but the fundamental tension between capability and accessibility remains unresolved.

13.11 Common Issues to Debug

- **API key exposure**: Use environment variables. Never commit keys to version control.
- **Rate limiting and costs**: Estimate costs before large batch runs. Use `gpt-4o-mini` for development, `gpt-4o` for production.
- **JSON parsing failures**: When requesting structured output, the model may not follow your format exactly. Use `try/except` around JSON parsing.
- **Model version changes**: Pin your model version in code. Document which model you used in your methods section.

13.12 Key Takeaways

LLM APIs are powerful tools for processing web data, but they require careful handling: non-determinism, cost, and reproducibility challenges are real. Use them for tasks where their strengths — scale, multilingual capability, structured extraction — outweigh their weaknesses. Always validate outputs against manual inspection. And treat the API like any other: read the documentation, manage your keys, and parse your responses carefully.

13.13 Further Reading

- OpenAI API documentation: https://platform.openai.com/docs/
- Anthropic API documentation: https://docs.anthropic.com/
- @ziems2024can — LLMs and computational social science

Part IV

Practice

14 Automating Data Collection

Learning Objectives

- Convert a Jupyter Notebook into a standalone Python script
- Configure headless Selenium for server-side browser automation
- Write GitHub Actions workflow files (YAML) to schedule scraping jobs
- Manage secrets (API keys, credentials) in GitHub repository settings
- Implement end-to-end automated scraping pipelines
- Assess the maintenance commitments of sustained data collection

14.1 From Exploration to Production

Interactive notebooks are for exploration; production data collection requires automation. This chapter moves from the interactive scraping techniques of Parts II and III into scheduled, unattended data pipelines that run in the cloud.

The progression is: **notebook** (where you develop and test your code) → **script** (a standalone `.py` file that runs without interaction) → **scheduled pipeline** (a script that runs automatically on a regular schedule). By the end of this chapter, you will have data collection infrastructure that runs every hour without your intervention.

Missing Manual Reference

For a full treatment of converting notebooks to scripts, command-line arguments, and `if __name__ == "__main__"` patterns, see *Missing Manual* Chapter 17: Scripting. For version control with Git, see *Missing Manual* Chapter 26: Version Control.

14.2 Why Automate?

The techniques you have learned in earlier chapters — scraping web pages, querying APIs, parsing documents — are powerful for capturing a snapshot of data at a single point in time. But many of the most interesting research questions in web data science are fundamentally

temporal. How do Rotten Tomatoes scores evolve during a film's theatrical release? How does the composition of a Spotify playlist shift with the seasons? Which Wikipedia articles surge in pageviews when a news event breaks, and how quickly does attention decay? These questions require not a single extraction but a time series: the same data collected at regular intervals over days, weeks, or months.

Automation transforms web data science from a one-time extraction exercise into a longitudinal research methodology. A scraper that runs once gives you a cross-sectional dataset. A scraper that runs every six hours for three months gives you a panel dataset that can reveal trends, cycles, and responses to external events. The infrastructure cost is modest — GitHub Actions provides free compute for public repositories — but the analytical payoff is substantial.

This chapter teaches you how to build that infrastructure from the tools you already know. The progression mirrors a real development workflow: you start with a working notebook, convert it to a standalone script, wrap it in a GitHub Actions workflow, and then monitor the pipeline over time. Each step introduces new skills, but the underlying logic remains the same code you have already written. The goal is not to learn a new programming paradigm but to move your existing skills from the interactive notebook environment into a production context where they run unattended. By the end, you will have built data collection infrastructure that connects directly to the research design considerations in Chapter 15.

14.3 Step 1: Notebook to Script

Let us convert a Rotten Tomatoes score scraper from notebook format to a standalone script. The key changes: remove interactive exploration, add a `main()` function, handle imports and configuration at the top, and write output to a file.

```
# scraper.py - Rotten Tomatoes Score Scraper
"""Scrape Rotten Tomatoes audience and critic scores for a film."""

import csv
import os
from datetime import datetime
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from bs4 import BeautifulSoup

def get_scores(url):
    """Retrieve audience and critic scores from a Rotten Tomatoes page.
```

```
    Parameters
    ----------
    url : str
        The Rotten Tomatoes movie URL

    Returns
    -------
    dict
        Dictionary with 'timestamp', 'url', 'critic_score', 'audience_score'
    """
    # Configure headless Chrome
    options = Options()
    options.add_argument("--headless")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")

    driver = webdriver.Chrome(options=options)

    try:
        driver.get(url)
        soup = BeautifulSoup(driver.page_source, "html.parser")

        # Extract scores from metadata tags
        scores = {}
        for tag in soup.find_all("score-icon-critic-deprecated"):
            scores["critic_score"] = tag.get("percentage", "")
        for tag in soup.find_all("score-icon-audience-deprecated"):
            scores["audience_score"] = tag.get("percentage", "")

        scores["timestamp"] = datetime.utcnow().isoformat()
        scores["url"] = url
        return scores
    finally:
        driver.quit()


def save_scores(scores, filepath="scores.csv"):
    """Append scores to a CSV file, creating it if needed."""
    file_exists = os.path.exists(filepath)

    with open(filepath, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=scores.keys())
```

```
        if not file_exists:
            writer.writeheader()
        writer.writerow(scores)


def main():
    url = "https://www.rottentomatoes.com/m/the_wild_robot"
    scores = get_scores(url)
    save_scores(scores)
    print(f"Saved scores at {scores['timestamp']}")


if __name__ == "__main__":
    main()
```

Notice the headless Chrome configuration: `--headless` runs the browser without a visible window, `--no-sandbox` and `--disable-dev-shm-usage` are required for running in CI/CD environments like GitHub Actions.

14.3.1 Testing Your Script Locally

Before deploying your script to GitHub Actions, always test it locally from the command line. Running a script in the terminal is different from running cells in a notebook — import errors, missing dependencies, and file path issues that were invisible in Jupyter will surface immediately.

```
# From your terminal, run the script directly:
# $ python scraper.py
# Saved scores at 2024-11-15T14:23:07.482361

# Verify the output file was created and contains data:
# $ head scores.csv
# timestamp,url,critic_score,audience_score
# 2024-11-15T14:23:07.482361,https://www.rottentomatoes.com/m/the_wild_robot,97,95

# Check the file size to confirm it is not empty:
# $ ls -la scores.csv
# -rw-r--r--  1 user  staff  142 Nov 15 14:23 scores.csv
```

Debugging a Python script on your own machine is straightforward: you can add `print()` statements, inspect variables, and re-run instantly. Debugging the same script inside a GitHub

Actions YAML workflow is far more painful — each attempt requires a commit, a push, and waiting for the runner to spin up, install dependencies, and execute. The few minutes you spend testing locally will save you hours of frustrating CI debugging later. If the script runs cleanly on your machine and produces the expected output file, you can be reasonably confident it will work in the Actions environment, provided the dependencies match.

14.4 Step 2: GitHub Actions

GitHub Actions lets you run scripts on a schedule using GitHub's cloud infrastructure. You define a workflow in a YAML file placed in `.github/workflows/`:

```yaml
# .github/workflows/scraper-action.yml
name: Scrape Rotten Tomatoes Scores

on:
  schedule:
    - cron: "0 */6 * * *"  # Every 6 hours
  workflow_dispatch:  # Allow manual triggering

jobs:
  scrape:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Install Chrome
        uses: browser-actions/setup-chrome@v1

      - name: Run scraper
        run: python scraper.py

      - name: Commit and push results
```

```
        run: |
          git config user.name "GitHub Actions Bot"
          git config user.email "actions@github.com"
          git add scores.csv
          git diff --staged --quiet || git commit -m "Update scores $(date -u)"
          git push
```

14.4.1 Python-Free Scraping

Not every automated scraper needs Python. The Wikimedia REST API returns JSON, so you can retrieve data with `curl` directly in the YAML workflow:

```
# .github/workflows/wiki-top-pages.yml
name: Top Wikipedia Pages

on:
  schedule:
    - cron: "0 12 * * *"  # Daily at noon UTC

jobs:
  fetch:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Fetch top pages
        run: |
          DATE=$(date -u -d "yesterday" +%Y/%m/%d)
          curl -s "https://wikimedia.org/api/rest_v1/metrics/pageviews/top/en.wikipedia/all-a
            -H "User-Agent: WebDataScience/1.0" \
            -o "data/top_pages_$(date -u -d yesterday +%Y%m%d).json"

      - name: Commit results
        run: |
          git config user.name "GitHub Actions Bot"
          git config user.email "actions@github.com"
          git add data/
          git diff --staged --quiet || git commit -m "Top pages $(date -u -d yesterday +%Y-%m
          git push
```

14.4.2 Understanding Cron Syntax

The `cron` field in your workflow file controls when the scraper runs. Cron expressions use five fields separated by spaces: `minute hour day-of-month month day-of-week`. Each field can be a specific number, a range, a list, or a wildcard (`*` for "every"). The `*/N` syntax means "every N units." All times are in UTC, not your local timezone.

Expression	Meaning
`0 */6 * * *`	Every 6 hours (at minute 0)
`0 12 * * 1-5`	Weekdays at noon UTC
`30 8 * * *`	Daily at 8:30 AM UTC
`0 0 1 * *`	First day of each month at midnight
`0 0 * * 0`	Weekly on Sunday at midnight

The interactive tool at crontab.guru lets you build and verify cron expressions visually — type in an expression and it will show you in plain English when the job will run. Bookmark it; you will use it more often than you expect. Note that GitHub Actions does not guarantee exact execution times for scheduled workflows. Your job may run several minutes after the scheduled time, especially during periods of high demand on GitHub's infrastructure. For most data collection tasks, this jitter is inconsequential.

14.4.3 Secrets Management

For authenticated APIs (like Spotify from Chapter 12), store credentials as GitHub repository secrets:

```yaml
      - name: Run Spotify scraper
        env:
          SPOTIPY_CLIENT_ID: ${{ secrets.SPOTIPY_CLIENT_ID }}
          SPOTIPY_CLIENT_SECRET: ${{ secrets.SPOTIPY_CLIENT_SECRET }}
        run: python spotify_scraper.py
```

> **Missing Manual Reference**
>
> For a full treatment of automation with GitHub Actions, cron syntax, and CI/CD concepts, see *Missing Manual* Chapter 28: Automation. For secrets management, see *Missing Manual* Chapter 29: Environment Variables and Secrets.

14.4.4 Monitoring and Debugging Actions

Once your workflow is running on a schedule, you need to monitor it. GitHub's Actions tab shows the status of every workflow run — green check for success, red X for failure. Click into a failed run to read the step-by-step log output, which will show you exactly where the error occurred. Common failure modes include expired API keys (the scraper worked for weeks, then the token expired), package version conflicts (a dependency updated and broke your import), and Chrome driver version mismatches (Selenium's ChromeDriver must match the installed Chrome version, and both update frequently).

A particularly insidious failure mode is the "silent success": the workflow runs to completion without errors, but the scraper did not actually collect any data — perhaps the target website changed its HTML structure, or an API endpoint started returning empty results. Your workflow reports success because no Python exception was raised, but your CSV file has a new row full of empty values. To catch this, add a validation step to your workflow that checks whether the data actually grew.

```
      - name: Validate scrape results
        run: |
          # Count rows in the CSV (excluding the header)
          ROW_COUNT=$(tail -n +2 scores.csv | wc -l)
          echo "Current row count: $ROW_COUNT"

          # Check if the latest row has non-empty score fields
          LAST_LINE=$(tail -1 scores.csv)
          if echo "$LAST_LINE" | grep -q ",,"; then
            echo "ERROR: Latest row contains empty fields"
            exit 1
          fi

          echo "Validation passed"
```

Adding this step after your scraping step means the workflow will fail with a clear error message when the data is incomplete, rather than silently committing garbage to your repository. You can also configure GitHub to send email notifications on workflow failures, so you learn about problems promptly rather than discovering weeks of missing data when you finally sit down to analyze your dataset.

14.4.5 Handling Failures Gracefully

Even with validation in place, your scraper will occasionally fail due to circumstances beyond your control: the target website goes down for maintenance, a rate limit is temporarily enforced,

or a network timeout occurs. A well-designed scraping pipeline handles these failures without losing previously collected data or crashing the workflow.

The simplest approach is to wrap your scraping logic in a try/except block that logs the error and exits gracefully rather than raising an unhandled exception:

```
import sys
import traceback

def main():
    url = "https://www.rottentomatoes.com/m/the_wild_robot"
    try:
        scores = get_scores(url)
        save_scores(scores)
        print(f"Saved scores at {scores['timestamp']}")
    except Exception as e:
        print(f"Scraper failed: {e}", file=sys.stderr)
        traceback.print_exc()
        # Exit with code 1 so the GitHub Actions step reports failure
        sys.exit(1)
```

This pattern ensures that a single failed scrape does not corrupt your data file. The error is logged, the workflow step is marked as failed, and the next scheduled run will try again. Over a multi-week collection period, you should expect a small number of failures — the important thing is that they are visible, documented, and do not cascade into data loss.

14.5 Step 3: Analyze Collected Data

After running your scraper for a period, you will have accumulated data to analyze:

```
import pandas as pd

df = pd.read_csv("scores.csv")
df["timestamp"] = pd.to_datetime(df["timestamp"])

import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5))
plt.plot(df["timestamp"], df["critic_score"], label="Critics", marker="o")
plt.plot(df["timestamp"], df["audience_score"], label="Audience", marker="s")
plt.title("Rotten Tomatoes Score Tracking")
```

```
plt.ylabel("Score (%)")
plt.legend()
plt.tight_layout()
plt.show()
```

14.6 Exercises

1. **Build a scraper.** Set up a GitHub Actions scraper for a data source of your choice. Run it for at least three days and analyze the results. Good candidates include weather APIs, news site headlines, government dashboards, or e-commerce prices.
2. **Script conversion.** Take one of your scraping projects from earlier chapters, convert it to a standalone `.py` script with headless execution, and verify it runs from the command line.
3. **Monitoring.** Design a simple monitoring strategy for your scraper: a check that validates the output file size, row count, or data freshness. Add it as a step in your GitHub Actions workflow.
4. **Government data collector.** Set up a daily GitHub Actions workflow that retrieves data from a free API (Open-Meteo weather, OpenAQ air quality, or USGS earthquake data). Run it for at least one week. Download the collected data and create a visualization showing how the measurements changed over the collection period.
5. **Staleness monitor.** Write a `check_freshness.py` script that reads your CSV file, checks whether the most recent timestamp is less than 24 hours old, and prints a warning if the data is stale. Add it as a step in your GitHub Actions workflow that runs after the scraping step. Test it by temporarily disabling the scraper and verifying the freshness check fails.

> **Public Interest Connection**
>
> Automated data collection is how you build the independent infrastructure that Chapter 3 describes as **ownership**. A GitHub Actions scraper does not depend on platform goodwill — it runs on your terms, on your schedule, collecting data you control. But ownership comes with responsibility: the ongoing labor of monitoring, maintaining, and updating your scrapers is itself a form of stewardship.

14.7 Social History and Public Interest

Automated data collection powers some of the most important public interest projects in data science. Air quality monitoring networks, transit performance trackers, and legislative activity dashboards all depend on regular, automated data retrieval. Community data projects — where citizens build their own monitoring infrastructure rather than depending on platforms or government agencies — embody the ownership value described in Chapter 3.

But automation also raises the ethical stakes from Chapter 2. A manual scraper that you run once makes a few requests. An automated scraper running every hour, indefinitely, accumulates a much larger footprint. The commitment to maintaining an automated scraper responsibly — monitoring for errors, respecting rate limits, adjusting to site changes — is a form of ongoing stewardship.

The Internet Archive's Wayback Machine is perhaps the most ambitious automated data collection project in history — it has been systematically crawling and archiving the web since 1996, preserving billions of pages that would otherwise be lost as websites change and disappear. On a smaller scale, projects like the GDELT Project monitor global news media in real time, and the Pushshift archive (before its shutdown) collected every public Reddit post and comment for over a decade. These projects demonstrate both the power and the responsibility of automated collection: the data they preserved has enabled thousands of research studies, but maintaining the infrastructure required sustained funding, technical labor, and institutional commitment. When you build your own automated scraper, you are participating in this same tradition of data stewardship — on a smaller scale, but with the same obligations.

14.8 Common Issues to Debug

- **YAML indentation errors**: YAML is whitespace-sensitive. Use a YAML validator before committing.
- **GitHub Actions runner environment**: The Ubuntu runner may differ from your local setup. Pin Python and Chrome versions explicitly.
- **Git conflicts**: If the Action runs while you are also pushing changes, you will get merge conflicts. Add `git pull` before `git push` in your workflow.
- **Cron syntax**: The five fields are `minute hour day-of-month month day-of-week`. Use crontab.guru to verify your schedule.
- **Stale ChromeDriver**: Selenium requires a ChromeDriver version that matches your Chrome version. The `browser-actions/setup-chrome@v1` action in GitHub Actions handles this automatically, but local setups may drift. Use `webdriver-manager` for automatic local driver management.

14.9 Key Takeaways

Interactive notebooks are for exploration; production data collection requires automation. The progression is notebook → script → scheduled pipeline. GitHub Actions provides free cloud-based scheduling for public repositories. But automation is not "set and forget" — scrapers break, APIs change, and data quality drifts. Building your own data infrastructure means accepting responsibility for its upkeep. That ongoing commitment is a form of the stewardship and ownership that defines public interest data science.

14.10 Further Reading

- GitHub Actions documentation: https://docs.github.com/en/actions
- Cron syntax reference: https://crontab.guru/
- Selenium headless mode: https://www.selenium.dev/documentation/webdriver/drivers/options/

15 Research Design with Web Data

Learning Objectives

- Develop a research question suited to web data and match it to appropriate data sources
- Design a multi-stage data collection plan combining scraping, APIs, and supplementary data
- Evaluate the sampling, representativeness, and validity challenges specific to web data
- Build a reproducible research pipeline from data collection through analysis
- Communicate findings to different audiences with appropriate evidentiary standards
- Apply the values of openness, oversight, and ownership to your own research practice

15.1 Technical Skill Is Not Enough

You now command a substantial technical toolkit: you can scrape static and dynamic web pages, parse PDFs, query APIs for platforms from Wikipedia to Spotify to the Federal Reserve, apply NLP and LLM-based analysis, and automate your data collection with GitHub Actions. These are powerful capabilities. But technical skill without research design produces data, not knowledge.

This chapter steps back from code to address the questions that determine whether your technically-excellent scraping produces genuine insight: What is the right question for this data source? What can this data actually tell you? What are the threats to the validity of your conclusions? How do you communicate your findings responsibly?

15.2 From Question to Data Source

The best web data science projects start with a question, not a dataset. The data source serves the question, not the other way around.

15.2.1 The Proposal Arc

A research proposal — whether for a class project, a thesis, a journal article, or a grant application — follows a consistent structure:

1. **Motivation**: What is the question or problem? Why does it matter? Who benefits from the answer?
2. **Data source**: What specific web data could address this question? What constructs does the data measure? What are the gaps?
3. **Methods**: How will you collect the data? What tools and techniques from this book will you use? What analytical approach will you apply?
4. **Expected results**: What kinds of analyses or visualizations will you produce? What patterns do you expect to find?
5. **Implications**: What should stakeholders do if your findings hold? What are the practical consequences?
6. **Limitations**: What can this data *not* tell you? What are the threats to validity? What alternative explanations exist?

Notice that limitations are not an afterthought — they are a core component of the design. Acknowledging what your data cannot do is as important as demonstrating what it can.

15.2.2 Matching Questions to Methods

Research Design	Web Data Methods	Example
Longitudinal tracking	Wayback Machine (Chapter 7), GitHub Actions (Chapter 14)	How has a platform's privacy policy changed over time?
Cross-sectional snapshot	API queries (Chapter 10, Chapter 11, Chapter 12)	How does Reddit moderation differ across subreddits?
Content analysis	HTML scraping (Chapter 6) + LLM coding (Chapter 13)	What topics dominate legislative activity in Colorado?
Network analysis	Link extraction (Chapter 10), social graphs (Chapter 12)	How connected are Wikipedia articles about climate change?
Natural experiment	Event-triggered collection (Chapter 14)	How did a policy change affect web search patterns?

15.3 Sampling and Representativeness

Web data is not a random sample of anything. It is a convenience sample shaped by who uses the platform, what the API exposes, and what your scraper can reach. Understanding these biases is essential for interpreting your findings.

Who is missing? Reddit users skew young, male, and American. Wikipedia editors skew educated, male, and from the Global North. Bluesky's early adopters skew toward journalists and technologists. The data you retrieve from any platform reflects its user base, not the broader population.

What is hidden? APIs expose what platforms choose to expose. Reddit's API does not reveal deleted posts, shadow-banned users, or the internal ranking algorithms that determine what content becomes visible. Spotify's audio features are proprietary measurements, not raw audio data. What you can measure is constrained by what the platform makes available.

What has been lost? Longitudinal studies face the erosion problem from Chapter 3. Articles are deleted, accounts are suspended, pages are redesigned. The data available today may not be available tomorrow, and the data available yesterday may already be gone.

15.4 Data Quality and Validation

Web data is messy. Scrapers fail silently, APIs return incomplete results, and the same information can appear in different formats across sources. Build validation into your pipeline:

```
def validate_dataset(df, expected_columns, min_rows=10):
    """Basic validation checks for a scraped dataset.

    Parameters
    ----------
    df : pd.DataFrame
        The dataset to validate
    expected_columns : list
        Columns that should be present
    min_rows : int
        Minimum expected number of rows

    Returns
    -------
    bool
        True if all checks pass
    """
```

```
    checks = []

    # Check column presence
    missing_cols = set(expected_columns) - set(df.columns)
    if missing_cols:
        print(f"WARNING: Missing columns: {missing_cols}")
        checks.append(False)
    else:
        checks.append(True)

    # Check minimum rows
    if len(df) < min_rows:
        print(f"WARNING: Only {len(df)} rows (expected >= {min_rows})")
        checks.append(False)
    else:
        checks.append(True)

    # Check for excessive missing data
    null_pct = df.isnull().mean()
    high_null = null_pct[null_pct > 0.5]
    if not high_null.empty:
        print(f"WARNING: High null rates: {dict(high_null)}")
        checks.append(False)
    else:
        checks.append(True)

    return all(checks)
```

15.5 Reproducibility and Documentation

A reproducible research pipeline means that someone else — or you, six months from now — can re-run your code and get the same results. This requires:

- **Version control**: Your code, configuration, and documentation should be in a git repository. See *Missing Manual* Chapter 26.
- **Data documentation**: For each dataset, record what it contains, how it was collected, when it was collected, and what its known quality issues are [@gebru2021datasheets].
- **Environment specification**: A `requirements.txt` or `environment.yml` file that pins the exact versions of your dependencies.
- **Collection parameters**: Document not just what data you collected but how: the API endpoints, query parameters, rate limiting, date ranges, and any filters you applied.

15.6 Communicating Findings

Web data science serves different audiences, each with different expectations:

- **Academic papers** require methodological precision, explicit limitations, and engagement with prior scholarship. Your data collection methods should be replicable from the description alone.
- **Policy briefs** need actionable findings, clear implications, and accessible language. Lead with the "so what?" rather than the methodology.
- **Data journalism** demands narrative structure, visual evidence, and accountability framing. The question is not just "what did you find?" but "what should the public know?"

In all cases, the visualizations you create with matplotlib and seaborn are not decorations — they are arguments. Every axis label, every color choice, every title should serve your analytical purpose.

15.6.1 Visualization as Argument

Consider the difference between a default plot and one designed to communicate a finding. A default `plt.plot()` call with no labels, no title, and no context forces the reader to guess what they are looking at. A well-designed figure makes its argument legible at a glance: the title states a claim, the annotations connect data patterns to real-world causes, and the axis labels include units so the reader can evaluate the evidence independently.

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd

# Suppose you have monthly Wikipedia pageviews for "Affordable Care Act"
# with columns: 'month' (datetime) and 'pageviews' (int)
df = pd.read_csv('aca_pageviews.csv', parse_dates=['month'])

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(df['month'], df['pageviews'], color='#2c3e50', linewidth=1.5)

# Annotate key events with vertical lines and text
ax.axvline(pd.Timestamp('2017-07-28'), color='#e74c3c', linestyle='--', alpha=0.7)
ax.text(pd.Timestamp('2017-08-15'), df['pageviews'].max() * 0.92,
        '"Skinny repeal"\nvote fails',
        fontsize=9, color='#e74c3c')
```

```
ax.axvline(pd.Timestamp('2020-03-11'), color='#2980b9', linestyle='--', alpha=0.7)
ax.text(pd.Timestamp('2020-04-01'), df['pageviews'].max() * 0.78,
        'WHO declares\nCOVID-19 pandemic',
        fontsize=9, color='#2980b9')

# Labels that include units and a title that states a finding
ax.set_title('Public attention to the ACA spikes around legislative and health crises',
             fontsize=12, fontweight='bold')
ax.set_xlabel('Month')
ax.set_ylabel('Monthly pageviews (thousands)')
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout()
plt.savefig('aca_pageviews_annotated.png', dpi=150)
# plt.show()
```

Notice that the title does not say "Wikipedia Pageviews for Affordable Care Act Over Time" — that merely describes the axes. Instead, it states a finding: public attention spikes around legislative and health crises. The annotations connect specific peaks to specific events, transforming a line chart into an argument about the relationship between political events and public information-seeking behavior. The clean axis formatting and removal of chart junk (top and right spines) keep the reader's attention on the data rather than on visual noise. When you build a figure, ask yourself: if a reader saw only this image and its title, would they understand the claim you are making?

15.7 Public Interest Framing

As you design your research, return to the values introduced in Chapter 3:

- **Openness**: Will you publish your code, data (to the extent ethically permissible), and methods so others can evaluate and build on your work?
- **Oversight**: Does your research contribute to independent accountability — helping citizens, journalists, or policymakers understand systems that affect them?
- **Ownership**: Are you building infrastructure (datasets, tools, pipelines) that others can use, or are you extracting value from public resources without contributing back?

These questions do not have formulaic answers. But asking them is itself a practice of responsible web data science.

15.7.1 Data Journalism as a Model

Some of the most compelling examples of web data science in practice come not from academia but from data journalism. Consider ProPublica's "Dollars for Docs" investigation: it began with a straightforward question — which doctors receive payments from pharmaceutical companies? — and pursued it by scraping publicly available but practically inaccessible disclosure databases maintained by drug manufacturers. The team cleaned, standardized, and linked records across companies, then published a searchable database that allowed local journalists across the country to investigate payment patterns in their own communities. The project moved from question to data collection to analysis to public accountability, following the same arc that a strong research proposal describes.

Academic researchers can learn several things from this model. Data journalists are accountable to an audience that will scrutinize their methods and challenge their conclusions. This pressure produces methodological transparency: the best data journalism outlets publish their code, describe their scraping and cleaning procedures, and explain the limitations of their data. They also structure their findings as narratives rather than tables of regression coefficients, making complex analyses accessible to non-specialist readers without sacrificing rigor. The journalistic commitment to showing your work — publishing the underlying data, linking to source documents, and explaining analytical choices — aligns directly with the reproducibility standards discussed earlier in this chapter.

The practitioners who do this work well — computational social scientists, civic technologists, data journalists — share a common profile: they combine data fluency with deep domain expertise. They understand not just how to scrape a website or query an API but why the data matters, who it affects, and what questions it can and cannot answer. This integration of technical and substantive knowledge is precisely what this book aims to develop. The tools you have learned are necessary but not sufficient; pairing them with domain knowledge, ethical reflection, and a commitment to public accountability is what transforms data collection into genuine contribution.

15.8 Exercises

1. **Research proposal.** Write a 1,500-word research proposal for a web data science project. Include all six components: motivation, data source, methods, expected results, implications, and limitations. Be specific about which tools from this book you would use and why.
2. **Validity threats.** Identify three potential threats to validity in a hypothetical study that uses scraped social media data to measure public opinion. For each threat, propose a mitigation strategy.

3. **Data documentation.** Take a dataset you collected in an earlier chapter and write a "datasheet" following @gebru2021datasheets: what does each variable measure, how was it collected, what are the known quality issues, and how should it be cited?
4. **Data limitations assessment.** Choose a dataset you collected in an earlier chapter. Write a 500-word assessment of its limitations, covering: sampling bias (what is included and excluded?), measurement validity (does your variable actually measure what you claim?), temporal scope (what period does the data cover, and what might have changed?), and platform constraints (how does the platform's design shape the data you can collect?).
5. **Multi-method design.** Design a research study that combines at least three techniques from this book (e.g., API data collection, web scraping, PDF extraction, text analysis, or automated collection). Write a one-page methods section that describes your data sources, collection procedures, analysis plan, and anticipated limitations. This should read like a methods section in an academic paper.

15.9 Key Takeaways

Technical skill without research design produces data, not knowledge. Every project needs a clear question, a defensible data collection strategy, honest assessment of limitations, and responsible communication of findings. The web is the richest data source humanity has ever created; using it well is both a technical and an ethical challenge. The skills in this book — scraping, API access, PDF extraction, NLP, LLM analysis, automation — are means, not ends. The end is producing knowledge that serves openness, oversight, and ownership.

15.10 Further Reading

- @salganik2018bit — comprehensive treatment of digital-age research design
- @gebru2021datasheets — a framework for documenting datasets
- @lazer2020computational — obstacles and opportunities in computational social science
- @vanatteveldt2022computational — methods for computational analysis of communication
- @rogers2019doing — qualitative and critical approaches to digital methods

15.11 Common Research Designs with Web Data

The technical toolkit in this book supports several distinct research designs. Choosing the right one depends on your question, your data source, and the claims you want to make.

15.11.1 Longitudinal Tracking

Question type: How does X change over time?

You observed something on the web and want to track how it evolves — the content of a policy document, the composition of a playlist, the frequency of a term in news coverage, or the price of a product. Longitudinal tracking requires either archival data (the Wayback Machine in Chapter 7) or forward-looking automated collection (GitHub Actions in Chapter 14).

Design considerations: The Wayback Machine captures snapshots opportunistically, not on a fixed schedule, so your temporal resolution is uneven. Automated scrapers give you consistent intervals but require maintenance. In both cases, you need to account for changes in the underlying structure (a website redesign, an API version change) that might affect your measurements without reflecting genuine change in the phenomenon you are studying.

15.11.2 Cross-Sectional Snapshot

Question type: How does X vary across cases at a point in time?

You want to compare legislative activity across states, subreddit moderation policies across communities, or audio features across genres. Cross-sectional designs collect data at one point in time and analyze variation across units.

Design considerations: The main threat is selection bias — you are limited to the platforms, communities, and data sources that exist and are accessible. If a platform's API does not expose the variable you need, you cannot include it. If a community is private or a government website does not publish the data you want, it is missing from your sample.

15.11.3 Content Analysis

Question type: What is the substantive content of a corpus?

You have collected a body of text — scraped web pages, PDF documents, social media posts — and want to characterize what it contains. Content analysis can be manual (human coding), automated (NLP in Chapter 7, LLM-assisted coding in Chapter 13), or a combination.

Design considerations: The reliability of automated coding should be validated against manual coding on a random sample. LLM-based classification is powerful but non-deterministic — run the same prompt twice and you may get different results. Always report inter-coder reliability or human-AI agreement rates.

15.11.4 Natural Experiments

Question type: What was the causal effect of an event or policy change?

Sometimes external events — a platform policy change, a natural disaster, a legislative decision — create natural variation that allows you to estimate causal effects. For example, you might compare Wikipedia editing behavior before and after a major news event, or Reddit community dynamics before and after a moderation policy change.

Design considerations: Causal claims from observational data are always tentative. You need a clear "treatment" event, a defensible comparison group, and controls for confounding variables. Web data makes it easy to observe change but does not, by itself, establish causation.

15.12 Managing the Research Process

15.12.1 Version Control for Research Projects

Your code, data documentation, and analysis notebooks should be version-controlled with git from the start. This is not just good practice — it is essential for reproducibility. When you revise your analysis after a reviewer's feedback, you need to be able to trace exactly what changed and when.

```
my-project/
   .gitignore           # Exclude api_keys.json, large data files
   README.md            # Project overview and setup instructions
   requirements.txt     # Python dependencies with pinned versions
   data/
       raw/             # Original scraped data (never modify)
       processed/       # Cleaned and transformed data
       README.md        # Data documentation (provenance, variables)
   notebooks/
       01_collection.ipynb
       02_cleaning.ipynb
       03_analysis.ipynb
   scripts/
       scraper.py       # Production scraping scripts
   output/
        figures/
        tables/
```

Missing Manual Reference

For a complete guide to version control with Git and GitHub — commits, branches, pull requests, and collaboration workflows — see *Missing Manual* Chapter 26: Version Control.

15.12.2 Data Documentation

For every dataset you create, write a datasheet [@gebru2021datasheets] that answers:

- **What**: What does each variable measure? What are the units? What do missing values mean?
- **How**: What tools did you use? What parameters and filters did you apply? What date range does the data cover?
- **Limits**: What is not captured? What biases are likely present? What quality issues are you aware of?
- **Ethics**: Does the data contain personally identifiable information? Was it collected in accordance with the platform's terms? Are there privacy risks if the dataset is shared?

This documentation is not busywork — it is the difference between a dataset that can be used by others (including future-you) and one that is a black box.

Part V

Appendices

16 AI Coauthorship and Responsible Disclosure

Generative AI tools are increasingly integrated into research, data analysis, and academic writing. This appendix provides practical guidance on when and how to disclose AI use in work produced for courses, publications, and professional practice.

16.1 When Disclosure Is Required

You should disclose AI use when:

- You used an LLM (ChatGPT, Claude, Gemini, etc.) to generate code, text, analysis, or visualizations that appear in your submitted work
- You used LLM APIs (as in Chapter 13) as analytical tools — for sentiment analysis, content coding, structured extraction, or similar tasks
- You used AI-powered code completion tools (GitHub Copilot, Cursor) for significant code generation beyond individual line completions

16.2 When Disclosure Is Typically Not Required

- Using AI for spell-checking, grammar correction, or basic proofreading
- Using AI to understand error messages or debug syntax errors (analogous to searching Stack Overflow)
- Using standard autocomplete features in code editors

16.3 How to Disclose: A Practical Framework

An effective disclosure statement answers four questions:

1. **What tool?** Name the specific model and version (e.g., "GPT-4o via the OpenAI API, accessed in October 2025")
2. **What task?** Describe what the AI was used for (e.g., "sentiment classification of 200 bill summaries," "initial draft of the data collection section")

3. **What oversight?** Describe how you reviewed, verified, and modified the output (e.g., "all generated code was tested independently," "LLM classifications were validated against a 20% manual sample")
4. **What limitations?** Note any concerns about the output's accuracy or appropriateness

16.4 Example Disclosure Statements

For a notebook assignment:

> AI tools used: GitHub Copilot was used for code autocompletion throughout the notebook. ChatGPT (GPT-4o) was consulted to debug a BeautifulSoup parsing error in Section 3. All code was tested and validated independently.

For a research paper:

> Text embeddings were computed using OpenAI's `text-embedding-3-small` model. Sentiment classifications were generated by Claude 3.5 Sonnet using the prompting strategy described in Section 3.2. A random sample of 50 classifications (25%) was manually validated, achieving 88% agreement with human coding.

For a final project:

> This project used AI tools in three ways: (1) GitHub Copilot for code autocompletion, (2) GPT-4o-mini for classifying 500 legislative bill summaries into policy categories using the few-shot prompt in Appendix B, and (3) Claude for copy-editing the final report. All classifications were spot-checked against a 50-item manual sample (92% agreement). The scraping code, analysis pipeline, and visualization code were written by the author.

16.5 Publisher and Funder Requirements

Major academic publishers and funding agencies have established disclosure requirements:

- **Elsevier, Springer Nature, Wiley, and Taylor & Francis** all require disclosure of generative AI use but do not permit AI as a listed author. The consensus: AI tools can assist research, but humans bear full responsibility for accuracy and integrity.
- **NIH** (NOT-OD-23-149) prohibits peer reviewers from using AI tools to analyze grant applications, citing confidentiality concerns. Researchers are encouraged to disclose AI use in project descriptions.
- **NSF** suggests disclosure of AI use in proposals and will formalize requirements in future Proposal and Award Policies and Procedures Guides.

The landscape of publisher and funder policies continues to evolve. Always check the current requirements of your target journal or funding agency before submission.

16.6 Core Principles

Three principles underlie responsible AI disclosure:

1. **Accountability is unchanged.** AI tools do not change the fundamentals of scholarly accountability. You are responsible for the accuracy, originality, and integrity of everything you submit, regardless of what tools you used to produce it.
2. **Disclosure is about transparency, not permission.** The goal is not to ask whether you are "allowed" to use AI, but to ensure that readers can evaluate your work knowing how it was produced. Could someone replicate your results knowing what tools you used and how?
3. **The failure to disclose is the problem.** Using AI tools responsibly and transparently is generally acceptable. Using them without disclosure, or in ways that misrepresent the origins of your work, compromises research integrity.

Missing Manual Reference

For the full AI disclosure statement used in this book's companion resource, see *Missing Manual* Appendix B: Disclosure — Use of AI in Authoring.

17 Where to Go from Here

This book has introduced the core tools and concepts of web data science. This appendix points you toward advanced topics, tools, and communities for continuing your education beyond this textbook.

17.1 Advanced Scraping and Data Collection

Scrapy is a full-featured web crawling framework for large-scale, production-grade scraping. If your projects outgrow the `requests` + BeautifulSoup approach of this book, Scrapy provides built-in support for following links, handling pagination, managing request queues, and exporting data. Documentation: https://scrapy.org/

Playwright is a modern alternative to Selenium developed by Microsoft. It is faster, more reliable, and has better async support. If you work extensively with dynamic pages (as in Chapter 8), Playwright is worth learning. Documentation: https://playwright.dev/python/

Cloud-based scraping with AWS Lambda, Google Cloud Functions, or similar serverless platforms lets you run scrapers without maintaining servers. This extends the GitHub Actions approach of Chapter 14 for larger-scale or more complex workflows.

17.2 Advanced APIs and Data Access

GraphQL APIs represent a different approach to API design than the REST APIs covered in this book. Instead of multiple endpoints returning fixed data structures, GraphQL lets you specify exactly what data you want in a single query. GitHub's API v4 is a prominent example.

Websockets and streaming APIs provide real-time data rather than request-response interactions. If you need to monitor live events — breaking news, financial markets, or social media feeds — streaming APIs are the appropriate tool.

The EU Digital Services Act (DSA) mandates researcher access to data from very large online platforms. As this framework matures, it may create new, regulated pathways for the kind of research data access that has been restricted by the enclosure dynamics described in Chapter 3.

17.3 Text Analysis and NLP

spaCy is an industrial-strength NLP library for tasks like named entity recognition, part-of-speech tagging, and dependency parsing. It complements the bag-of-words approach from Chapter 7 and the LLM-based analysis from Chapter 13. Documentation: https://spacy.io/

Hugging Face Transformers provides access to thousands of pre-trained models for text classification, summarization, translation, and embedding computation. If you need more control than the OpenAI API provides, or want to run models locally, Transformers is the standard library. Documentation: https://huggingface.co/docs/transformers/

Topic modeling with BERTopic or Latent Dirichlet Allocation (LDA) can help you discover the thematic structure of large text corpora — a natural extension of the content analysis techniques in Chapter 13.

17.4 Network Analysis

NetworkX is the standard Python library for creating, analyzing, and visualizing graphs. The link structures from Chapter 10 and social graphs from Chapter 12 are natural inputs for network analysis. Documentation: https://networkx.org/

Gephi provides interactive network visualization for larger graphs. It can import data from NetworkX and produce publication-quality network visualizations. Download: https://gephi.org/

17.5 Geospatial Analysis

GeoPandas extends pandas with support for spatial data types and operations. Combined with Census geographic data from Chapter 11, it enables spatial analysis and mapping of web-scraped data. Documentation: https://geopandas.org/

Folium creates interactive maps using Leaflet.js. It is useful for visualizing scraped data with geographic components (business locations, legislative districts, environmental monitoring stations). Documentation: https://python-visualization.github.io/folium/

17.6 Data Management and Infrastructure

SQLite and **PostgreSQL** provide database storage for scraped data that outgrows flat CSV files. If your automated scrapers (Chapter 14) accumulate large datasets, migrating from CSV to a database improves performance and query flexibility.

Apache Airflow orchestrates complex data pipelines with dependencies between tasks. It is the production-grade equivalent of the GitHub Actions workflows in Chapter 14, suited for pipelines with many steps, conditional logic, and failure recovery.

Docker provides reproducible computing environments that bundle your code, dependencies, and configuration into a portable container. This solves the "works on my machine" problem for collaborative projects.

DVC (Data Version Control) extends git to handle large data files, tracking datasets alongside code. It addresses the challenge of versioning scraped data that is too large for standard git.

17.7 Communities and Continuing Education

Summer Institutes in Computational Social Science (SICSS) are intensive two-week programs held at universities around the world. They cover advanced topics in computational social science with hands-on training and collaborative research. Apply at: https://sicss.io/

The Carpentries (Software Carpentry, Data Carpentry, Library Carpentry) offer workshops on foundational computing and data skills. Their instructor training program is also an excellent way to improve your own teaching. Information: https://carpentries.org/

GESIS Computational Social Science Resource List maintains a curated collection of courses, textbooks, and tools: https://github.com/gesiscss/awesome-computational-social-science

17.8 Recommended Textbooks

- @salganik2018bit — the methodological companion to this book's technical focus; essential reading on observing behavior, asking questions, and running experiments in the digital age
- @vanatteveldt2022computational — deeper treatment of text analysis, NLP methods, and communication research with computational tools
- @vandenbroucke2018practical — more advanced scraping patterns, anti-detection techniques, and production considerations
- @rogers2019doing — qualitative and critical approaches to web data, complementing this book's quantitative emphasis
- @munzert2014automated — covers similar ground to this book but using R; useful for multilingual researchers
- @lazer2009computational — the foundational manifesto for computational social science as a field